

Contents

第十四册简介: PyTorch 与深度学习工程	8
本书定位	8
为什么需要这本书	8
适合读者	8
学完后应具备的能力	8
阅读建议	8
与其他书的关系	8
第十四册: PyTorch 与深度学习工程	8
本书定位	8
章节文件	9
第一章: PyTorch 张量基础	9
0. 本讲资料边界与第二轮精修口径	9
1.1 Tensor 是什么	9
1.1.1 关键 shape 公式与张量调试速查	10
1.2 Shape 是第一优先级	11
1.3 Dtype: 精度、性能和数值稳定性	12
1.4 Device: CPU 和 GPU 必须一致	12
1.5 创建 Tensor 的常用方式	13
1.6 索引与切片	13
1.7 Broadcasting: 自动扩展维度	13
1.8 unsqueeze、squeeze 和 expand	14
1.9 matmul: 大模型里最常见的计算	14
1.10 bmm、mm 和 matmul 的区别	15
1.11 einsum: 把维度关系写清楚	15
1.12 reshape、view 和 flatten	16
1.13 transpose、permute 和 contiguous	16
1.14 Stride: 理解 contiguous 的关键	17
1.15 cat 和 stack	17
1.16 mask 的常见写法	18
1.17 in-place 操作的风险	18
1.18 调试 Tensor 的实用清单	19
1.19 最小可运行 PyTorch 张量审计 demo	19
1.20 常见面试题	22
1.21 本章小结	22
第二章: Autograd 与反向传播	23
0. 本讲资料边界与第二轮精修口径	23
2.1 为什么需要 Autograd	23
2.1.1 关键公式与 Autograd 调试速查	24
2.2 动态计算图: 前向时构建, 反向后释放	25
2.3 requires_grad 决定是否跟踪梯度	25
2.4 叶子张量、非叶子张量和.grad	26
2.5 backward 做了什么	27
2.6 梯度会累积, 不会自动清零	27
2.7 detach: 切断计算图	28
2.8 no_grad 和 inference_mode	29
2.9 in-place 操作为什么容易破坏反向传播	30
2.10 常见 Autograd 报错与定位方法	31
2.10.1 grad 是 None	31
2.10.2 loss 不下降	31
2.10.3 backward through graph a second time	31
2.10.4 element does not require grad	31

2.11 autograd.grad、二阶梯度和高阶求导	32
2.12 hook: 观察和修改梯度	32
2.13 训练循环里的正确顺序	33
2.14 大模型工程中的 Autograd 边界	34
2.14.1 LoRA 和参数冻结	34
2.14.2 梯度检查点	34
2.14.3 混合精度和 GradScaler	34
2.14.4 分布式训练	35
2.15 最小可运行 Autograd 审计 demo	35
2.16 Debug 清单	37
2.17 面试官会怎么问	38
问题一: PyTorch 的 autograd 是怎么工作的?	38
问题二: 为什么每次训练都要 zero_grad?	38
问题三: detach 和 no_grad 有什么区别?	38
问题四: 为什么中间 tensor 的.grad 是 None?	38
问题五: 为什么验证时要同时写 model.eval() 和 no_grad()?	38
问题六: in-place 操作为什么可能导致 backward 报错?	38
2.18 常见误区	39
2.19 小练习	39
2.20 本章总结	39
第三章: nn.Module 与模型组织	39
0. 本讲资料边界与第二轮精修口径	40
3.1 nn.Module 解决什么问题	40
3.1.1 关键机制公式与 Module 调试速查	41
3.2 最小 Module 写法	42
3.3 forward 和 call 的关系	43
3.4 Parameter: 什么会被 optimizer 更新	43
3.5 子模块注册: 赋值不是普通赋值	44
3.6 Sequential、ModuleList、ModuleDict 怎么选	46
3.7 ParameterList 和 ParameterDict	46
3.8 buffer: 不是参数, 但属于模型状态	47
3.9 parameters、named_parameters 和参数分组	48
3.10 state_dict 保存什么	48
3.11 保存和加载模型	49
3.12 strict=True 和 strict=False	50
3.13 train() 和 eval() 到底改变什么	50
3.14 Module.to、device 和 dtype 管理	51
3.15 apply、children、modules 和递归遍历	52
3.16 自定义初始化	53
3.17 大模型中的 Module 组织方式	54
3.18 forward 返回值怎么设计	55
3.19 hooks: 观察和修改中间过程	56
3.20 register_module、setattr 和动态添加层	56
3.21 常见 bug: 参数没有注册	57
3.22 常见 bug: eval 后结果仍然变化	57
3.23 常见 bug: 加载 checkpoint 报错	58
3.24 LoRA、Adapter 与可训练参数筛选	58
3.25 DDP、FSDP 与 Module 边界	59
3.26 torch.compile 对 Module 的影响	59
3.27 最小可运行 Module 审计 demo	60
3.28 面试高频问题	64
3.29 本章小练习	64
3.30 本章总结	64
第四章: Dataset 与 DataLoader	65

0. 本讲资料边界与第二轮精修口径	65
4.1 为什么数据管线很重要	65
4.1.1 关键 shape 公式与数据管线调试速查	66
4.2 Dataset 的两种基本范式	66
4.2.1 map-style Dataset	66
4.2.2 iterable-style Dataset	67
4.3 一个最小可训练文本 Dataset	67
4.4 DataLoader 解决什么问题	68
4.5 batch 是怎么拼起来的	69
4.6 collate_fn: batch 组装的关键	69
4.6.1 变长序列 padding	69
4.6.2 labels 的 padding	70
4.6.3 一个更完整的 collate_fn	70
4.7 padding 的方向和坑	70
4.7.1 right padding	71
4.7.2 left padding	71
4.7.3 常见坑	71
4.8 packing: 把多个短样本拼进一个长序列	71
4.8.1 为什么需要 packing	71
4.8.2 简化 packing 示例	71
4.8.3 packing 的风险	72
4.9 shuffle、sampler 和 batch_sampler	72
4.9.1 shuffle	72
4.9.2 sampler	72
4.9.3 batch_sampler	72
4.10 按长度分桶: 减少 padding 浪费	73
4.11 DataLoader 的性能参数	73
4.11.1 num_workers	73
4.11.2 pin_memory	73
4.11.3 persistent_workers	74
4.11.4 prefetch_factor	74
4.12 分布式训练中的数据切分	74
4.13 一个完整的数据管线例子	74
4.14 常见 bug 与排查顺序	75
4.14.1 collate_fn 报错	75
4.14.2 数据加载特别慢	76
4.14.3 每个 epoch 顺序都一样	76
4.14.4 loss 异常偏小	76
4.14.5 多卡训练样本重复	76
4.15 最小可运行 DataLoader 审计 demo	76
4.16 面试官会怎么问	79
问题一: Dataset 和 DataLoader 有什么区别?	79
问题二: 为什么变长序列不能直接用默认 DataLoader?	79
问题三: collate_fn 的作用是什么?	79
问题四: num_workers、pin_memory、persistent_workers 分别有什么作用?	79
问题五: 为什么分布式训练要用 DistributedSampler?	79
问题六: 为什么 collate_fn 通常比在 Dataset 里 padding 更合适?	79
问题七: DataLoader 里如何保证随机性可复现?	79
4.17 常见误区	79
4.18 小练习	80
4.19 本章总结	80
第五章: 训练循环工程	80
0. 本讲资料边界与第二轮精修口径	80
5.1 训练循环解决什么问题	80
5.1.1 关键公式与训练循环调试速查	81

5.2 最小训练 step	82
5.3 batch 到 device	82
5.4 loss 在哪里计算	83
5.4.1 模型外部计算 loss	83
5.4.2 模型内部计算 loss	83
5.5 语言模型 loss 的标准形态	84
5.6 梯度裁剪	84
5.7 梯度累积	85
5.8 optimizer 和 scheduler 的 step 时机	85
5.9 日志记录: 不要只看 loss	86
5.10 validation 循环	86
5.11 checkpoint 保存什么	87
5.12 checkpoint 加载和恢复训练	87
5.13 保存 best checkpoint 和 last checkpoint	88
5.14 可复现性	88
5.15 训练中的异常检测	89
5.15.1 NaN 和 Inf	89
5.15.2 loss spike	89
5.15.3 OOM	89
5.16 一个工程化训练循环骨架	89
5.17 最小可运行训练循环审计 demo	91
5.18 训练循环常见 bug	94
5.18.1 loss 不下降	94
5.18.2 训练速度很慢	94
5.18.3 恢复训练后 loss spike	94
5.18.4 验证结果不稳定	94
5.19 面试官会怎么问	95
问题一: 一个标准训练 step 包含哪些步骤?	95
问题二: 梯度累积为什么要把 loss 除以 accum_steps?	95
问题三: 验证循环和训练循环有什么区别?	95
问题四: checkpoint 应该保存什么?	95
问题五: 训练 loss 出现 NaN 怎么排查?	95
5.20 常见误区	95
5.21 小练习	95
5.22 本章总结	96

第六章: 混合精度与显存优化 96

0. 本讲资料边界与第二轮精修口径	96
6.1 显存到底花在什么地方	96
6.1.1 关键公式与显存调试速查	97
6.2 为什么需要混合精度	97
6.3 FP16 和 BF16 的区别	98
6.3.1 FP16	98
6.3.2 BF16	98
6.4 autocast: 自动混合精度	98
6.5 GradScaler: 解决 fp16 的数值问题	99
6.6 mixed precision 训练的推荐模板	99
6.7 activation checkpointing	100
6.8 梯度累积和显存	101
6.9 影响显存的关键因素	101
6.10 optimizer 状态为什么也占很多显存	101
6.11 训练中常见的显存优化顺序	102
6.12 显存泄漏的常见来源	102
6.13 OOM 排查方法	102
6.13.1 先看是训练还是验证	102
6.13.2 先缩小 batch 和 sequence length	103

6.13.3 检查是否在累积图	103
6.13.4 检查验证流程	103
6.14 一个完整 AMP + checkpointing 的训练骨架	103
6.15 最小可运行 AMP 与显存审计 demo	104
6.16 工程中怎么选 fp16 和 bf16	106
6.17 面试官会怎么问	106
问题一: FP16 和 BF16 有什么区别?	106
问题二: autocast 和 GradScaler 分别做什么?	106
问题三: activation checkpointing 为什么能省显存?	106
问题四: 为什么 Adam 比 SGD 更占显存?	106
问题五: 训练 OOM 时你怎么排查?	106
6.18 常见误区	106
6.19 小练习	107
6.20 本章总结	107
第七章: 分布式训练入门	107
0. 本讲资料边界与第二轮精修口径	107
7.1 为什么需要分布式训练	107
7.1.1 关键公式与分布式调试速查	108
7.2 rank、local rank 和 world size	109
7.2.1 world size	109
7.2.2 rank	109
7.2.3 local rank	109
7.3 process group 是什么	109
7.4 数据并行的核心直觉	110
7.5 all-reduce 是什么	110
7.6 DDP 的基本用法	110
7.7 torchrun 启动训练	111
7.8 DistributedSampler	111
7.9 global batch size 怎么算	112
7.10 DDP 和 DataParallel 的区别	112
7.11 DDP 的通信时机	112
7.12 梯度累积下的 no_sync	113
7.13 FSDP 的基本直觉	113
7.14 分布式 checkpoint	113
7.15 分布式训练常见 bug	114
7.15.1 程序卡住	114
7.15.2 某些参数没有梯度	114
7.15.3 每张卡数据重复	114
7.15.4 只有 rank 0 正常, 其他 rank 报错	114
7.16 分布式 debug 清单	114
7.17 一个最小 DDP 训练骨架	115
7.18 最小可运行分布式训练审计 demo	116
7.19 面试官会怎么问	118
问题一: DDP 的工作原理是什么?	118
问题二: rank、local rank、world size 分别是什么?	118
问题三: 为什么 DDP 要用 DistributedSampler?	119
问题四: DDP 和 FSDP 有什么区别?	119
问题五: 分布式训练卡住怎么排查?	119
7.20 常见误区	119
7.21 小练习	119
7.22 本章总结	119
第八章: Debug 与 Profiling	120
0. 本讲资料边界与第二轮精修口径	120
8.1 Debug 的基本原则	120

8.1.1 关键公式与 debug/profiling 速查	120
8.2 打印 tensor 元信息	121
8.3 Shape debug	122
8.4 Dtype debug	122
8.5 Device debug	123
8.6 NaN 和 Inf debug	123
8.7 anomaly detection	124
8.8 梯度 debug	124
8.9 hook 调试中间激活	125
8.10 OOM debug	125
8.11 DataLoader 性能 debug	126
8.12 性能 profiling 的基本思路	126
8.13 torch.profiler 入门	127
8.14 常见性能瓶颈	127
8.14.1 小 batch 导致 GPU 利用率低	127
8.14.2 DataLoader 慢	127
8.14.3 CPU-GPU 拷贝慢	127
8.14.4 通信慢	128
8.15 训练结果异常的系统排查	128
8.15.1 loss 不下降	128
8.15.2 train loss 降, val loss 不降	128
8.15.3 训练很快但效果很差	128
8.16 分布式 debug	128
8.17 一个实用 debug checklist	129
8.18 一个最小可复现 bug 的写法	129
8.19 最小可运行 debug/profiling 审计 demo	130
8.20 面试官会怎么问	133
问题一: loss 不下降你怎么排查?	133
问题二: 训练出现 NaN 怎么排查?	133
问题三: GPU 利用率低怎么排查?	133
问题四: OOM 怎么排查?	133
问题五: torch.profiler 能看什么?	133
8.21 常见误区	133
8.22 小练习	133
8.23 本章总结	134

第九章: 实现 Transformer 组件	134
0. 本讲资料边界与第二轮精修口径	134
9.1 Transformer 组件总览	134
9.1.1 关键公式与 Transformer 组件速查	135
9.2 配置对象	136
9.3 Token Embedding 和 LM Head	137
9.4 手写 RMSNorm	137
9.5 Causal Mask	137
9.6 Scaled Dot-Product Attention	138
9.7 Multi-Head Self-Attention	139
9.8 MLP 和 SwiGLU	139
9.9 Decoder Block	140
9.10 RoPE 的最小实现	140
9.11 加入 RoPE 的 Attention	141
9.12 最小 Decoder-only LM	142
9.13 KV Cache 的直觉	142
9.14 简化 KV Cache Attention	143
9.15 最小可运行 Transformer 组件审计 demo	144
9.16 常见实现 bug	149
9.16.1 head_dim 不整除	149

9.16.2 mask 方向错	149
9.16.3 transpose 后忘记 contiguous	149
9.16.4 loss 没 shift	149
9.16.5 padding 参与 loss	150
9.17 面试官会怎么问	150
问题一: Self-attention 的 shape 怎么变化?	150
问题二: 为什么需要 causal mask?	150
问题三: RMSNorm 和 LayerNorm 有什么区别?	150
问题四: RoPE 作用在哪里?	150
问题五: KV Cache 为什么能加速推理?	150
9.18 小练习	150
9.19 本章总结	150

第十章: 工程面试题

151

0. 本讲资料边界与第二轮精修口径	151
10.0.1 工程面试公式与排查速查	151
10.1 Tensor 与 Shape	152
问题 1: PyTorch tensor 最重要的元信息有哪些?	152
问题 2: view 和 reshape 有什么区别?	152
问题 3: 为什么 transpose 后经常要 contiguous()?	152
问题 4: Broadcasting 规则是什么?	152
10.2 Autograd 与梯度	153
问题 5: PyTorch autograd 是怎么工作的?	153
问题 6: 为什么每个 step 要 zero_grad()?	153
问题 7: 为什么中间 tensor 的 .grad 是 None?	153
问题 8: detach() 和 no_grad() 有什么区别?	153
10.3 nn.Module 与模型组织	153
问题 9: nn.Module 的作用是什么?	153
问题 10: 为什么子模块不能放普通 list?	153
问题 11: Parameter 和 buffer 的区别是什么?	153
问题 12: state_dict 保存什么?	153
10.4 Dataset、DataLoader 与训练数据	153
问题 13: Dataset 和 DataLoader 有什么区别?	153
问题 14: 为什么变长序列需要自定义 collate_fn?	154
问题 15: padding token 为什么不能直接参与 loss?	154
10.5 训练循环	154
问题 16: 一个标准训练 step 包含哪些步骤?	154
问题 17: 梯度累积为什么要除以 accumulation steps?	154
问题 18: 验证循环和训练循环有什么区别?	154
问题 19: checkpoint 应该保存什么?	154
10.6 混合精度与显存	154
问题 20: 训练显存主要花在哪里?	154
问题 21: FP16 和 BF16 有什么区别?	154
问题 22: autocast 和 GradScaler 分别做什么?	154
问题 23: activation checkpointing 为什么能省显存?	155
10.7 分布式训练	155
问题 24: DDP 的工作原理是什么?	155
问题 25: rank、local rank、world size 分别是什么?	155
问题 26: 为什么 DDP 要用 DistributedSampler?	155
问题 27: DDP 和 FSDP 有什么区别?	155
10.8 Debug 与 Profiling	155
问题 28: loss 不下降怎么排查?	155
问题 29: 训练出现 NaN 怎么排查?	155
问题 30: OOM 怎么排查?	155
问题 31: GPU 利用率低怎么排查?	155
10.9 Transformer 实现	156

问题 32: Self-attention 的 shape 如何变化?	156
问题 33: 为什么需要 causal mask?	156
问题 34: RoPE 作用在哪里?	156
问题 35: KV Cache 为什么能加速推理?	156
10.10 系统性开放题	156
问题 36: 如何设计一个可靠的训练脚本?	156
问题 37: 从单卡训练扩展到多卡训练要改什么?	156
问题 38: 如何把一个教学 Transformer 改得更接近真实 LLM?	156
10.11 最小可运行 PyTorch 工程面试复盘 demo	156
10.12 本章总结	159

第十四册简介：PyTorch 与深度学习工程

本书定位

第十四册训练工程实现能力，帮助读者从“能看懂代码”升级到“能写训练脚本、能调试模型、能实现核心组件、能组织实验工程”。

为什么需要这本书

大模型算法岗不只考理论，还会考能否手写 attention、loss、训练循环、mask、debug、profiling 和分布式基础。PyTorch 是把理论变成实验的主要工具。

适合读者

1. PyTorch 不熟、训练脚本写得不稳的读者。
2. 想准备手写代码和工程 debug 面试的人。
3. 想为第三册实战打好工程基础的人。

学完后应具备的能力

1. 能熟练操作 tensor、autograd、nn.Module、Dataset 和 DataLoader。
2. 能写完整训练循环、验证循环、checkpoint 和日志。
3. 能理解混合精度、显存优化、分布式训练入门。
4. 能实现 Transformer 核心组件并排查 shape、grad 和 loss 问题。

阅读建议

建议和第三册前两部分同步学习，边看边写代码。

与其他书的关系

第十四册是第三册实战、第十三册数学和第五册训练系统之间的工程桥梁。

第十四册：PyTorch 与深度学习工程

本书定位

本书专门训练工程实现能力，帮助读者从能看懂代码升级到能写训练脚本、调试模型、实现核心算子和组织实验工程。

章节文件

1. chapters/01-pytorch 张量基础.md
2. chapters/02-autograd 与反向传播.md
3. chapters/03-nn-module 与模型组织.md
4. chapters/04-dataset-dataloader.md
5. chapters/05-训练循环工程.md
6. chapters/06-混合精度与显存优化.md
7. chapters/07-分布式训练入门.md
8. chapters/08-debug 与 profiling.md
9. chapters/09-实现 transformer 组件.md
10. chapters/10-工程面试题.md

第一章：PyTorch 张量基础

PyTorch 的核心对象是 tensor。大模型工程里的输入 token、embedding、hidden states、attention scores、logits、loss mask、梯度和参数，本质上都是不同 shape、dtype、device 和 stride 的 tensor。

很多人学 PyTorch 时只记住了几个 API，但一到面试或调试训练脚本，就会卡在更底层的问题上：为什么 view 报错、为什么广播后 shape 不对、为什么 matmul 得到的维度和预期不同、为什么 transpose 后要 contiguous、为什么同一段代码在 CPU 上能跑到 GPU 上就报 device mismatch。

本章目标不是罗列 PyTorch API，而是建立大模型工程中最常用的 tensor 基础：shape、dtype、device、broadcast、矩阵乘法、einsum、索引、reshape、view、transpose、contiguous，以及常见调试方法。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修时，参考了 PyTorch 官方 tensor tutorial、tensor attributes、broadcasting semantics、torch.matmul、torch.bmm、torch.einsum、Tensor.view、torch.reshape、Tensor.contiguous、Tensor.stride、torch.nn.functional.cross_entropy 和 CUDA device 相关文档，并结合前序 Transformer、attention、LM loss、mask 和数学基础章节的 shape 推导口径。

本章只聚焦大模型工程最常见的 tensor 基础：shape、dtype、device、broadcasting、matmul / bmm / einsum、索引切片、view / reshape / flatten、transpose / permute / contiguous、stride、mask、loss reshape 和 tensor debug。它不展开 CUDA kernel、autograd graph、distributed tensor、Tensor Core、编译优化或 profiler 细节；这些会在后续章节中处理。

1.1 Tensor 是什么

Tensor 可以理解为多维数组，但在深度学习框架里，它不只是数组。一个 PyTorch tensor 至少包含几类关键信息：

1. 数据本身。
2. Shape，也就是每个维度的长度。
3. Dtype，也就是数据类型。
4. Device，也就是数据存放在 CPU 还是 GPU。
5. Stride，也就是如何从底层存储中按维度访问数据。
6. 是否参与梯度计算。

例如：

```
import torch

x = torch.randn(2, 3, 4)

print(x.shape)  # torch.Size([2, 3, 4])
print(x.dtype)  # torch.float32
```

```
print(x.device) # cpu
print(x.stride())
```

在大模型中，一个最常见的 hidden states shape 是：

X : $[B, T, d]$

其中：

1. B 是 batch size。
2. T 是 sequence length。
3. d 是 hidden size。

如果你能稳定地推导每一步 tensor 的 shape，大部分模型实现和 debug 都会变简单。

1.1.1 关键 shape 公式与张量调试速查

语言模型中最常见的张量主线：

$$input_ids \in \mathbb{Z}^{B \times T}$$

$$X = Emb(input_ids), \quad X \in \mathbb{R}^{B \times T \times d}$$

LM head 输出：

$$logits \in \mathbb{R}^{B \times T \times V}$$

next-token prediction 的 shift：

$$shift_logits = logits[:, 0 : T - 1, :]$$

$$shift_labels = labels[:, 1 : T]$$

交叉熵前展平：

$$shift_logits \rightarrow \mathbb{R}^{B(T-1) \times V}, \quad shift_labels \rightarrow \mathbb{Z}^{B(T-1)}$$

多头注意力拆头：

$$d = Hd_h$$

$$Q, K, V \in \mathbb{R}^{B \times H \times T \times d_h}$$

attention score：

$$S = \frac{QK^T}{\sqrt{d_h}}, \quad S \in \mathbb{R}^{B \times H \times T \times T}$$

attention mask 常见广播：

$$mask_{pad} : [B, T] \rightarrow [B, 1, 1, T]$$

$$mask_{causal} : [T, T] \rightarrow [1, 1, T, T]$$

broadcasting 从右往左对齐维度：

$$[B, T, d] + [d] \rightarrow [B, T, d]$$

LoRA 或线性层常见矩阵乘法：

$$X[B, T, d_{in}] \times W[d_{in}, d_{out}] \rightarrow Y[B, T, d_{out}]$$

view / reshape / contiguous 的面试口径：

1. view: 只在当前 stride 能兼容目标 shape 时直接改视图。
2. reshape: 能返回 view 就返回 view, 不行时可能创建拷贝。
3. transpose / permute: 通常只改 stride, 不重排底层数据, 因此结果常常不是 contiguous。

这些公式不是为了背维度, 而是为了形成一个调试顺序: 先看 shape, 再看 dtype, 再看 device, 再看 stride / contiguous, 最后检查 mask 方向和数值稳定性。

1.2 Shape 是第一优先级

写深度学习代码时, 最重要的习惯是先看 shape, 再看数值。

以语言模型为例, 输入通常是 token id:

input_ids: [B, T]

经过 embedding 层后:

hidden_states: [B, T, d]

经过 LM head 后:

logits: [B, T, vocab_size]

训练时 labels 通常是:

labels: [B, T]

如果做 next-token prediction, 常见处理是:

shift_logits = logits[:, :-1, :]

shift_labels = labels[:, 1:]

此时:

shift_logits: [B, T - 1, vocab_size]

shift_labels: [B, T - 1]

很多 loss 报错都来自 shape 没有对齐。比如 cross_entropy 通常希望输入是 [N, C], 标签是 [N], 因此语言模型训练里常写成:

```
loss = torch.nn.functional.cross_entropy(
    shift_logits.reshape(-1, shift_logits.size(-1)),
    shift_labels.reshape(-1),
)
```

这里的含义是把 [B, T - 1, vocab_size] 展平为 [B * (T - 1), vocab_size], 把 [B, T - 1] 展平为 [B * (T - 1)]。

面试回答:

我写 PyTorch 模型时会优先跟踪 shape。语言模型里 input_ids 是 [B, T], embedding 后是 [B, T, d], LM head 后是 [B, T, vocab]。

1.3 Dtype: 精度、性能和数值稳定性

Tensor 的 dtype 决定数值精度和计算性能。

常见 dtype:

1. `torch.float32`: 默认浮点类型, 精度较高, 显存占用较大。
2. `torch.float16`: 半精度, 显存更省, 吞吐更高, 但更容易溢出或下溢。
3. `torch.bfloat16`: 常用于大模型训练, 动态范围接近 `fp32`, 精度低于 `fp32`。
4. `torch.int64`: 常用于 token id、labels、索引。
5. `torch.bool`: 常用于 mask。

示例:

```
input_ids = torch.tensor([[1, 2, 3]], dtype=torch.long)
mask = torch.tensor([[True, True, False]])
x = torch.randn(2, 3, dtype=torch.float32)
y = x.to(torch.bfloat16)
```

大模型工程中要特别注意:

1. Embedding 输入必须是整数 token id, 通常是 `torch.long`。
2. Attention mask 通常是 bool 或可以加到 logits 上的浮点 mask。
3. 模型参数和激活可能是 `fp32`、`fp16` 或 `bf16`。
4. Loss 计算、softmax、归一化等操作可能需要更稳定的 dtype。

常见错误:

`RuntimeError: expected scalar type Long but found Float`

这类错误通常说明你把浮点 tensor 传给了需要整数索引的模块, 例如 `nn.Embedding`。

1.4 Device: CPU 和 GPU 必须一致

Tensor 的 device 表示它在哪个设备上。

```
x = torch.randn(2, 3)
print(x.device) # cpu
```

```
if torch.cuda.is_available():
    x = x.cuda()
    print(x.device) # cuda:0
```

PyTorch 不会自动在 CPU 和 GPU 之间搬运参与同一次计算的 tensor。下面代码会报错:

```
x = torch.randn(2, 3, device="cuda")
y = torch.randn(2, 3, device="cpu")
z = x + y
```

正确做法是让参与计算的 tensor 在同一个 device 上:

```
y = y.to(x.device)
z = x + y
```

训练脚本里常见写法:

```
device = next(model.parameters()).device
batch = {k: v.to(device) for k, v in batch.items()}
```

注意, 不是所有 batch 字段都一定是 tensor。如果 batch 里混有字符串、列表或元数据, 需要先判断类型:

```
batch = {
    k: v.to(device) if torch.is_tensor(v) else v
    for k, v in batch.items()
}
```

1.5 创建 Tensor 的常用方式

常见创建方式：

```
torch.tensor([1, 2, 3])
torch.zeros(2, 3)
torch.ones(2, 3)
torch.randn(2, 3)
torch.arange(0, 10)
torch.empty(2, 3)
```

需要注意 `torch.tensor` 和 `torch.as_tensor` 的区别：

```
data = [1, 2, 3]
x = torch.tensor(data)
y = torch.as_tensor(data)
```

`torch.tensor` 通常会拷贝数据并创建新 tensor。`torch.as_tensor` 在某些输入类型下会尽量共享数据，避免额外拷贝。

`torch.empty` 只分配内存，不初始化数值：

```
x = torch.empty(2, 3)
```

它的内容是不确定的，不能当作全 0 使用。只有你马上会覆盖所有元素时，才适合用 `empty`。

1.6 索引与切片

Tensor 支持类似 NumPy 的索引和切片。

```
x = torch.arange(2 * 3 * 4).reshape(2, 3, 4)
```

```
print(x[0].shape)      # [3, 4]
print(x[:, 1].shape)   # [2, 4]
print(x[:, :, -1].shape) # [2, 3]
```

语言模型里常见切片：

```
shift_logits = logits[:, :-1, :]
shift_labels = labels[:, 1:]
```

含义是：用第 0 到第 T-2 个位置的 logits 预测第 1 到第 T-1 个 token。

如果要保留维度，可以使用范围切片而不是单点索引：

```
x[:, 0, :].shape      # [B, d]
x[:, 0:1, :].shape    # [B, 1, d]
```

这在拼接、广播和 attention mask 处理中很常见。

1.7 Broadcasting：自动扩展维度

Broadcasting 是 PyTorch 自动对齐 shape 的规则。它允许某些不同 shape 的 tensor 做逐元素运算。

规则可以简化为：从右往左对齐维度，每一维要么相等，要么其中一个为 1，要么其中一个维度不存在。

示例：

```
x = torch.randn(2, 3, 4)
bias = torch.randn(4)
y = x + bias
print(y.shape) # [2, 3, 4]
```

这里 `bias: [4]` 会被看成 `[1, 1, 4]`，再广播到 `[2, 3, 4]`。

LayerNorm 中常见类似行为：

```
x:      [B, T, d]
weight: [d]
bias:   [d]
```

`weight` 和 `bias` 会沿着 `batch` 和 `sequence` 维度广播。

Attention mask 里也经常使用 broadcasting：

```
scores = torch.randn(2, 4, 8, 8)    # [B, H, T, T]
mask = torch.ones(2, 1, 1, 8).bool() # [B, 1, 1, T]
scores = scores.masked_fill(~mask, float("-inf"))
```

`mask` 会沿着 `head` 维和 `query` 维广播。

常见坑是某个维度刚好可以广播，但语义不对。例如你想让 `mask` 对齐 `token` 维，却把它写成了 `[B, T, 1]`，代码可能跑，但实际 `mask` 的方向错了。

1.8 unsqueeze、squeeze 和 expand

为了让 tensor 满足 broadcasting，需要经常增加或删除长度为 1 的维度。

```
x = torch.randn(2, 3)
```

```
x1 = x.unsqueeze(1)
print(x1.shape) # [2, 1, 3]
```

```
x2 = x1.squeeze(1)
print(x2.shape) # [2, 3]
```

`unsqueeze(dim)` 会在指定位置插入一个长度为 1 的维度。

`squeeze(dim)` 会删除指定位置上长度为 1 的维度。

注意不要随便使用无参数 `squeeze()`：

```
x = torch.randn(1, 8, 1, 64)
y = x.squeeze()
print(y.shape) # [8, 64]
```

它会删除所有长度为 1 的维度，可能把 `batch` 维也删掉。训练脚本中更安全的写法是显式指定维度：

```
y = x.squeeze(2)
```

`expand` 可以创建广播视图：

```
x = torch.randn(1, 3)
y = x.expand(4, 3)
print(y.shape) # [4, 3]
```

`expand` 不会真正复制数据，而是通过 `stride` 让多个位置指向同一份底层存储。因此不能把它理解成物理拷贝。如果需要真实拷贝，可以使用 `repeat`，但会占更多内存。

1.9 matmul：大模型里最常见的计算

矩阵乘法是 Transformer 的核心。

二维矩阵乘法规则：

```
A: [m, n]
B: [n, p]
C = A @ B
```

C: [m, p]

PyTorch 中:

```
A = torch.randn(3, 4)
B = torch.randn(4, 5)
C = A @ B
print(C.shape) # [3, 5]
```

对更高维 tensor, `torch.matmul` 会把最后两维当作矩阵维度, 前面的维度按 batch 维处理并尝试 broadcast。

```
Q = torch.randn(2, 4, 8, 64) # [B, H, T, d_h]
K = torch.randn(2, 4, 8, 64) # [B, H, T, d_h]
```

```
scores = Q @ K.transpose(-2, -1)
print(scores.shape) # [2, 4, 8, 8]
```

这里:

1. Q 的最后两维是 [T, d_h]。
2. K.transpose(-2, -1) 的最后两维是 [d_h, T]。
3. 相乘后得到 [T, T]。
4. 前面的 [B, H] 作为 batch 维保留下来。

这就是 attention score 的 shape 来源。

1.10 bmm、mm 和 matmul 的区别

PyTorch 有多个矩阵乘法 API:

1. `torch.mm`: 只处理两个二维矩阵。
2. `torch.bmm`: 处理两个三维 tensor 的 batch 矩阵乘法。
3. `torch.matmul`: 更通用, 支持一维、二维和高维 batch 矩阵乘法。
4. `@`: 通常等价于调用 `matmul` 语义。

示例:

```
A = torch.randn(10, 3, 4)
B = torch.randn(10, 4, 5)
C = torch.bmm(A, B)
print(C.shape) # [10, 3, 5]
```

`bmm` 不做 batch 维广播, 要求 batch size 相同。`matmul` 更灵活, 能处理更多 broadcasting 场景。

工程里常用建议:

1. 写普通二维矩阵乘法时, 用 `@` 或 `matmul`。
2. 写 attention 这种高维 batch 矩阵乘法时, 用 `@` 或 `matmul`。
3. 如果明确是三维 batch 矩阵乘法, 并且不需要广播, 可以用 `bmm`。

1.11 einsum: 把维度关系写清楚

`einsum` 可以用字符串显式描述维度之间的计算关系, 适合表达复杂张量运算。

例如矩阵乘法:

```
A = torch.randn(3, 4)
B = torch.randn(4, 5)
C = torch.einsum("mn,np->mp", A, B)
print(C.shape) # [3, 5]
```

Attention score 可以写成:

```
Q = torch.randn(2, 4, 8, 64) # [B, H, T, D]
K = torch.randn(2, 4, 8, 64) # [B, H, T, D]
```

```
scores = torch.einsum("bhtd,bhsd->bhts", Q, K)
print(scores.shape) # [2, 4, 8, 8]
```

这里 `t` 表示 query position, `s` 表示 key position, `d` 是被求和消掉的 head dimension。

Value 加权求和可以写成：

```
attn = torch.softmax(scores, dim=-1) # [B, H, T, S]
V = torch.randn(2, 4, 8, 64) # [B, H, S, D]
```

```
context = torch.einsum("bhts,bhsd->bhtd", attn, V)
print(context.shape) # [2, 4, 8, 64]
```

`einsum` 的优点是表达清晰，尤其适合面试手写和解释维度。缺点是字符串写错时不如普通矩阵乘法直观，而且在某些场景下性能需要实际 profiling。

1.12 reshape、view 和 flatten

改变 tensor shape 时常用 `reshape`、`view` 和 `flatten`。

```
x = torch.randn(2, 3, 4)
```

```
y = x.reshape(6, 4)
z = x.view(6, 4)
w = x.flatten(0, 1)
```

它们都可以把 `[2, 3, 4]` 变成 `[6, 4]`，但底层语义不同。

`view` 要求 tensor 在内存布局上兼容目标 shape。`reshape` 更宽松，如果不能返回 `view`，可能会创建拷贝。

常见错误：

```
x = torch.randn(2, 3, 4)
y = x.transpose(1, 2)
z = y.view(2, 12) # 可能报错
```

原因是 `transpose` 后 tensor 通常不再 contiguous，`view` 无法按目标 shape 解释底层存储。

更稳妥的写法：

```
z = y.reshape(2, 12)
```

或者显式让 tensor 连续：

```
z = y.contiguous().view(2, 12)
```

工程建议：

1. 如果只是想安全改变 shape，优先用 `reshape`。
2. 如果明确知道 tensor 是 contiguous，并且想避免潜在拷贝，可以用 `view`。
3. 如果要合并连续维度，用 `flatten(start_dim, end_dim)` 可读性更好。

1.13 transpose、permute 和 contiguous

`transpose` 用来交换两个维度：

```
x = torch.randn(2, 3, 4)
y = x.transpose(1, 2)
print(y.shape) # [2, 4, 3]
```

`permute` 可以重新排列多个维度：


```
x = torch.randn(2, 3, 4, 5)
y = x.permute(0, 2, 1, 3)
print(y.shape) # [2, 4, 3, 5]
```

多头注意力中常见维度变换:

```
B, T, d_model = 2, 8, 64
num_heads = 4
head_dim = d_model // num_heads
```

```
x = torch.randn(B, T, d_model)
q = x.reshape(B, T, num_heads, head_dim).transpose(1, 2)
```

```
print(q.shape) # [B, H, T, d_h]
```

这一步把 $[B, T, d_model]$ 拆成 $[B, T, H, d_h]$, 再交换维度得到 attention 更方便计算的 $[B, H, T, d_h]$ 。

计算完 attention 后, 通常要变回 $[B, T, d_model]$:

```
context = torch.randn(B, num_heads, T, head_dim)
out = context.transpose(1, 2).contiguous().view(B, T, d_model)
print(out.shape) # [B, T, d_model]
```

这里的 `contiguous()` 很关键。`transpose` 只是改变 stride 视图, 不一定重新排列底层内存。后续使用 `view` 前, 常常需要先调用 `contiguous()`。

面试回答:

`transpose` 和 `permute` 通常不会真的复制数据, 而是改变 tensor 的 `stride`, 因此结果可能不是 `contiguous`。`view` 要求内存布局兼容。

1.14 Stride: 理解 contiguous 的关键

Stride 表示沿某个维度移动一步, 在底层存储中要跳过多少个元素。

```
x = torch.arange(2 * 3 * 4).reshape(2, 3, 4)
print(x.stride()) # 通常是 (12, 4, 1)
```

```
y = x.transpose(1, 2)
print(y.shape) # [2, 4, 3]
print(y.stride()) # 通常是 (12, 1, 4)
```

原始 x 中, 最后一维是连续的。`transpose` 后, `shape` 变了, 但底层数据没有真正重排, 只是 `stride` 变了。

这也是为什么有些 tensor 看起来 `shape` 没问题, 但 `view` 会失败。因为 `view` 想用新的 `shape` 直接解释同一块连续内存, 而当前 `stride` 不满足要求。

判断 tensor 是否连续:

```
print(x.is_contiguous())
print(y.is_contiguous())
```

让 tensor 变连续:

```
z = y.contiguous()
```

注意, `contiguous()` 可能触发真实内存拷贝。在性能敏感代码中, 不要无脑到处加; 但在模型原型、面试手写和调试阶段, 它是解决 layout 问题的常见手段。

1.15 cat 和 stack

`torch.cat` 和 `torch.stack` 都能拼接 tensor, 但语义不同。

`cat` 在已有维度上拼接:

```
a = torch.randn(2, 3)
b = torch.randn(2, 3)

c = torch.cat([a, b], dim=0)
print(c.shape) # [4, 3]
```

```
d = torch.cat([a, b], dim=1)
print(d.shape) # [2, 6]
```

stack 会新增一个维度:

```
s = torch.stack([a, b], dim=0)
print(s.shape) # [2, 2, 3]
```

数据集 collate 时, 常用 stack 把多个样本堆成 batch:

```
samples = [torch.randn(10) for _ in range(4)]
batch = torch.stack(samples, dim=0)
print(batch.shape) # [4, 10]
```

如果每个样本长度不同, 不能直接 stack, 需要先 padding 或自定义 collate function。

1.16 mask 的常见写法

Mask 是大模型代码里最容易出错的 tensor 之一。

常见 attention scores:

scores: [B, H, T, T]

Padding mask 可能来自输入:

attention_mask: [B, T]

其中 1 表示有效 token, 0 表示 padding。为了加到 scores 上, 常见转换方式是:

```
attention_mask = torch.tensor([[1, 1, 1, 0, 0]]) # [B, T]
mask = attention_mask[:, None, None, :].bool() # [B, 1, 1, T]
```

```
scores = torch.randn(1, 4, 5, 5)
scores = scores.masked_fill(~mask, float("-inf"))
```

Causal mask 用于禁止当前位置看未来 token:

```
T = 5
causal_mask = torch.tril(torch.ones(T, T, dtype=torch.bool))
print(causal_mask.shape) # [T, T]
```

扩展到 attention scores:

```
scores = scores.masked_fill(~causal_mask[None, None, :, :], float("-inf"))
```

真实工程里通常会把 padding mask 和 causal mask 结合起来。关键是始终确认 mask 的维度语义: 哪个维度是 batch, 哪个维度是 query position, 哪个维度是 key position。

1.17 in-place 操作的风险

PyTorch 中以下划线结尾的方法通常是 in-place 操作:

```
x.add_(1)
x.masked_fill_(mask, 0)
```

In-place 操作会直接修改原 tensor, 可能节省内存, 但也可能影响 autograd 或后续复用。

例如:

```
x = torch.randn(3, requires_grad=True)
```

```
y = x * 2
```

某些复杂场景下，对计算图中还需要的 tensor 做 in-place 修改会导致 backward 报错。

在训练代码中，除非你明确知道某个 in-place 操作不会破坏计算图，否则优先使用非 in-place 写法。

常见建议：

1. 面试手写时，少用 in-place，避免引入额外解释成本。
2. 写模型 forward 时，谨慎修改会参与梯度计算的中间激活。
3. 优化显存时，可以有意识地使用 in-place，但需要配合测试和 anomaly detection。

1.18 调试 Tensor 的实用清单

当 PyTorch 代码报错时，不要只看最后一行错误。优先打印关键 tensor 的元信息：

```
def debug_tensor(name, x):
    if not torch.is_tensor(x):
        print(name, type(x))
        return
    print(
        name,
        "shape=", tuple(x.shape),
        "dtype=", x.dtype,
        "device=", x.device,
        "contiguous=", x.is_contiguous(),
        "stride=", x.stride(),
    )
```

重点检查：

1. Shape 是否符合预期。
2. Dtype 是否符合模块要求。
3. Device 是否一致。
4. 是否有 NaN 或 Inf。
5. Mask 的方向是否正确。
6. view 前是否 contiguous。

检查 NaN 和 Inf：

```
torch.isnan(x).any()
torch.isinf(x).any()
torch.isfinite(x).all()
```

定位异常值：

```
bad = ~torch.isfinite(x)
print(bad.nonzero())
```

对大模型训练来说，tensor debug 的核心不是“会不会调用 API”，而是能不能快速判断错误属于 shape、dtype、device、layout、mask 还是数值稳定性问题。

1.19 最小可运行 PyTorch 张量审计 demo

下面用一段 demo 串起本章最重要的 tensor 操作：shape、dtype、device、broadcasting、matmul、einsum、mask、reshape、view、contiguous、stride、stack 和语言模型 loss reshape。

```
import math
import torch
import torch.nn.functional as F
```

```

torch.manual_seed(7)

B, T, d_model = 2, 5, 16
num_heads = 4
vocab_size = 32
head_dim = d_model // num_heads

input_ids = torch.tensor(
    [
        [1, 5, 9, 2, 0],
        [1, 7, 8, 0, 0],
    ],
    dtype=torch.long,
)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
input_ids = input_ids.to(device)

embedding_table = torch.randn(vocab_size, d_model, device=device)
x = embedding_table[input_ids] # [B, T, d_model]
bias = torch.arange(d_model, device=device).float() / 1000
x_with_bias = x + bias # [B, T, d_model]

Wq = torch.randn(d_model, d_model)
Wk = torch.randn(d_model, d_model)
Wv = torch.randn(d_model, d_model)
Wq = Wq.to(device)
Wk = Wk.to(device)
Wv = Wv.to(device)

q = x @ Wq # [B, T, d_model]
k = x @ Wk # [B, T, d_model]
v = x @ Wv # [B, T, d_model]

q = q.reshape(B, T, num_heads, head_dim).transpose(1, 2) # [B, H, T, d_h]
k = k.reshape(B, T, num_heads, head_dim).transpose(1, 2) # [B, H, T, d_h]
v = v.reshape(B, T, num_heads, head_dim).transpose(1, 2) # [B, H, T, d_h]

scores = q @ k.transpose(-2, -1) / math.sqrt(head_dim) # [B, H, T, T]
scores_einsum = torch.einsum("bhtd,bhsd->bhts", q, k) / math.sqrt(head_dim)

padding_mask = input_ids.ne(0) # [B, T]
causal_mask = torch.tril(torch.ones(T, T, dtype=torch.bool, device=device))
combined_mask = padding_mask[:, None, None, :] & causal_mask[None, None, :, :]
scores = scores.masked_fill(~combined_mask, -1e9)

attn = torch.softmax(scores, dim=-1) # [B, H, T, T]
context = attn @ v # [B, H, T, d_h]

transposed = context.transpose(1, 2) # [B, T, H, d_h]
view_failed = False
try:
    transposed.view(B, T, d_model)
except RuntimeError:
    view_failed = True

```

```

out = transposed.contiguous().view(B, T, d_model)          # [B, T, d_model]

lm_logits = torch.randn(B, T, vocab_size, device=device)
labels = input_ids
shift_logits = lm_logits[:, :-1, :].reshape(-1, vocab_size)
shift_labels = labels[:, 1:].reshape(-1)
loss = F.cross_entropy(shift_logits, shift_labels, ignore_index=0)

sample_a = torch.randn(d_model, device=device)
sample_b = torch.randn(d_model, device=device)
stacked = torch.stack([sample_a, sample_b], dim=0)
concatated = torch.cat([sample_a, sample_b], dim=0)

report = {
    "meta": {
        "input_shape": tuple(input_ids.shape),
        "input_dtype": str(input_ids.dtype),
        "hidden_shape": tuple(x.shape),
        "hidden_dtype": str(x.dtype),
        "device": str(x.device),
    },
    "broadcast": {
        "bias_shape": tuple(bias.shape),
        "x_plus_bias_shape": tuple(x_with_bias.shape),
    },
    "attention": {
        "q_shape": tuple(q.shape),
        "scores_shape": tuple(scores.shape),
        "einsum_matches_matmul": torch.allclose(scores_einsum.masked_fill(~combined_mask, -1e9), scores),
        "mask_shape": tuple(combined_mask.shape),
        "attn_row_sum": round(float(attn[0, 0, 2].sum().item()), 6),
        "out_shape": tuple(out.shape),
    },
    "layout": {
        "transposed_contiguous": transposed.is_contiguous(),
        "transposed_stride": transposed.stride(),
        "view_failed_before_contiguous": view_failed,
        "out_contiguous": out.is_contiguous(),
    },
    "loss": {
        "shift_logits_shape": tuple(shift_logits.shape),
        "shift_labels_shape": tuple(shift_labels.shape),
        "loss_is_finite": bool(torch.isfinite(loss).item()),
    },
    "cat_stack": {
        "stacked_shape": tuple(stacked.shape),
        "concatated_shape": tuple(concatated.shape),
    },
    "checks": {
        "embedding_rank_3": x.dim() == 3,
        "broadcast_shape_ok": x_with_bias.shape == x.shape,
        "attention_shape_ok": scores.shape == (B, num_heads, T, T),
        "mask_broadcast_shape_ok": combined_mask.shape == (B, 1, T, T),
        "view_requires_contiguous": view_failed,
    }
}

```

```

        "lm_loss_flatten_ok": shift_logits.shape[0] == shift_labels.shape[0],
    },
}

```

print(report)

这段代码覆盖了本章最核心的点：

1. input_ids 是 torch.long, embedding 后变成 [B,T,d_model] 的浮点 hidden states。
2. [d_model] 的 bias 可以 broadcast 到 [B,T,d_model]。
3. 线性投影是矩阵乘法，多头拆分依赖 reshape 和 transpose。
4. Attention scores 可以用 matmul 或 einsum 表达。
5. Padding mask 和 causal mask 通过 broadcasting 合成 [B,1,T,T]。
6. 多头合并时，transpose 后通常需要 contiguous().view(...)
7. LM loss 前要把 [B,T-1,V] 和 [B,T-1] 展平成 [B(T-1),V] 和 [B(T-1)]。

真正写 Transformer 时会使用 nn.Linear、更完整的 mask、dropout、输出投影和更严谨的初始化，但 tensor shape 主线是不变的。

1.20 常见面试题

问题一：view 和 reshape 有什么区别？

答：

view 要求 tensor 的内存布局和目标 shape 兼容，通常要求 contiguous 或 stride 可以直接解释；reshape 更灵活，如果不能返回 view

问题二：为什么 transpose 后经常要 contiguous？

答：

transpose 通常不会重排底层数据，而是改变 stride，所以结果可能不是 contiguous。后续如果用 view 合并维度，可能因为内存布

问题三：PyTorch broadcasting 规则是什么？

答：

Broadcasting 会从右往左对齐维度，每一维要么相等，要么其中一个是 1，要么其中一个维度不存在。它常用于 bias、LayerNorm 参

问题四：Attention score 的 shape 为什么是 [B, H, T, T]？

答：

多头注意力中 Q 和 K 的 shape 通常是 [B, H, T, d_h]。计算 Q @ K.transpose(-2, -1) 时，最后两维是 [T, d_h] 乘 [d_h, T]，得到 [

问题五：cat 和 stack 有什么区别？

答：

cat 是在已有维度上拼接，不增加新维度；stack 会先新增一个维度，再把多个 tensor 沿这个新维度堆起来。把多个样本组成 batch

1.21 本章小结

PyTorch 张量基础的重点不是背 API，而是形成稳定的工程判断：

1. 任何模型代码先看 shape。
2. Dtype 决定精度、性能和模块输入要求。
3. Device 必须一致，否则 CPU/GPU 混用会报错。
4. Broadcasting 很强大，但要确认语义方向正确。
5. matmul 和 einsum 是理解 attention 的核心工具。
6. reshape、view、transpose、permute、contiguous 背后是内存布局和 stride。
7. Mask、loss reshape、多头拆分和合并，是大模型工程最常见的 tensor 基础考点。

下一章会在 tensor 基础上进入 autograd，理解 PyTorch 如何构建计算图、保存中间结果、执行 backward，以及为什么 detach、no_grad、梯度累积和 in-place 操作会影响训练行为。

第二章：Autograd 与反向传播

Autograd 是 PyTorch 把数学公式变成可训练模型的核心机制。你写 `loss.backward()` 时，PyTorch 会沿着前向计算留下的计算图，从 loss 反向追踪到模型参数，自动计算每个参数的梯度。

很多人能写训练代码，但一遇到梯度为 None、显存莫名增长、Trying to backward through the graph a second time、one of the variables needed for gradient computation has been modified by an inplace operation 这类问题，就不知道从哪里查。原因通常不是反向传播公式不会，而是没有真正理解 PyTorch 的动态计算图、叶子张量、梯度累积、detach、no_grad 和 in-place 操作之间的关系。

本章目标不是重新推导所有神经网络反向传播公式，而是把 PyTorch Autograd 在工程里的工作方式讲清楚：哪些 tensor 会记录梯度，计算图什么时候创建和释放，为什么梯度会累积，为什么推理阶段要关掉梯度，什么时候应该 detach，什么时候不该 detach，以及如何排查训练脚本里的梯度问题。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修时，参考了 PyTorch 官方 automatic differentiation tutorial、autograd mechanics、leaf / non-leaf tensor tutorial、torch.autograd.grad、Tensor.backward、Tensor.detach、torch.no_grad、torch.inference_mode、detect_anomaly、Optimizer.zero_grad 和 activation checkpointing 相关文档，并结合前序数学基础、张量基础和训练循环章节的公式与工程口径。

本章只聚焦 PyTorch Autograd 的工程主线：动态计算图、链式法则、requires_grad、叶子张量、.grad、backward、vector-Jacobian product、梯度累积、zero_grad、detach、no_grad / inference_mode、in-place 风险、autograd.grad、hook、训练循环顺序和梯度 debug。它不展开完整自动微分理论、CUDA backward kernel、functorch / torch.func、分布式 autograd 或编译器级图优化。

2.1 为什么需要 Autograd

训练神经网络的目标是最小化 loss。以最简单的线性模型为例：

$$\hat{y} = wx + b$$

如果使用均方误差：

$$L = (\hat{y} - y)^2$$

训练时需要知道：

$$\frac{\partial L}{\partial w}, \quad \frac{\partial L}{\partial b}$$

手写小模型的梯度还可以接受，但真实大模型有 embedding、attention、MLP、layer norm、residual connection、dropout、mask、loss reshape、混合精度和分布式通信。如果每个算子都手写反向传播，几乎不可维护。

Autograd 解决的是这个问题：你只写前向计算，框架自动记录前向过程，并根据链式法则自动生成反向计算。

这就是 PyTorch 训练代码通常长这样：

```
logits = model(input_ids)
loss = criterion(logits, labels)
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

前两行是你显式写出的前向计算，backward() 会触发反向传播，optimizer.step() 使用梯度更新参数。

面试回答：

Autograd 的作用是自动求导。PyTorch 在前向计算时动态构建计算图，每个可求导操作都会记录反向函数。调用 loss.backward() 时

2.1.1 关键公式与 Autograd 调试速查

标量 loss 对参数的梯度：

$$g_{\theta} = \frac{\partial L}{\partial \theta}$$

链式法则：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

多层计算图中的反向传播：

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow L$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial x}$$

向量输出的 `backward(gradient)` 本质是 vector-Jacobian product:

$$v^T J, \quad J = \frac{\partial y}{\partial x}$$

梯度累积：

$$grad_t = grad_{t-1} + \frac{\partial L_t}{\partial \theta}$$

梯度累积模拟大 batch 时的平均 loss：

$$L_{\text{micro}}^{\text{scaled}} = \frac{L_{\text{micro}}}{K}$$

参数更新必须在不建图环境中做：

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

叶子张量与 `.grad` 的工程口径：

1. 模型参数通常是 leaf tensor, `backward()` 后梯度累积到参数的 `.grad`。
2. 中间 tensor 默认不保留 `.grad`, 需要调试时用 `retain_grad()` 或 `hook`。
3. `detach()` 切断某个 tensor 和当前计算图的连接。
4. `no_grad()` / `inference_mode()` 让一段代码不记录新的 autograd 图。
5. `retain_graph=True` 不是普通训练的默认解法, 通常应该重新 forward 或重构 loss。

2.2 动态计算图：前向时构建，反向后释放

PyTorch 使用动态计算图，也被称为 define-by-run。意思是：代码每执行一次前向，就根据实际执行路径构建一张新的计算图。

示例：

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = x * x + 3 * x + 1

print(y)          # tensor(11., grad_fn=<AddBackward0>)
print(y.grad_fn)  # 记录 y 是由什么反向函数产生的

y.backward()
print(x.grad)     # tensor(7.), 因为 dy/dx = 2x + 3 = 7
```

这里 x 是需要梯度的输入， y 是由一系列计算得到的结果。因为 $x.requires_grad=True$ ，所以 $x * x$ 、 $3 * x$ 、加法这些操作会被 autograd 记录下来。

动态计算图的特点是：

1. 每次 forward 都会重新构图。
2. Python 控制流可以自然参与建图，例如 if、for、递归。
3. backward 默认会释放中间激活和计算图，节省显存。
4. 如果需要对同一张图反向多次，必须显式指定 `retain_graph=True`，但工程中应谨慎使用。

例如：

```
x = torch.tensor(2.0, requires_grad=True)
y = x * x

y.backward()
y.backward()  # RuntimeError
```

第二次 backward() 会报错，典型信息是：

```
Trying to backward through the graph a second time
```

原因是第一次反向传播后，PyTorch 已经释放了计算图中保存的中间结果。通常正确做法不是随手加 `retain_graph=True`，而是重新执行一次 forward：

```
x = torch.tensor(2.0, requires_grad=True)

y = x * x
y.backward()

x.grad = None
y = x * x
y.backward()
```

`retain_graph=True` 适用于确实需要从同一张图上做多次 backward 的场景，例如某些多 loss 分开反传、梯度惩罚、二阶梯度实验。但在普通训练循环里，如果加它只是为了让报错消失，往往会造成显存持续增长。

2.3 requires_grad 决定是否跟踪梯度

`requires_grad` 表示这个 tensor 是否需要参与梯度计算。

```
x = torch.randn(3, requires_grad=True)
w = torch.randn(3, requires_grad=True)
```

```
y = (x * w).sum()
print(y.requires_grad) # True
```

```
y.backward()
print(x.grad)
print(w.grad)
```

只要一个操作的输入中有 tensor 需要梯度，并且这个操作本身可求导，输出通常也会带上 `requires_grad=True`。
对模型参数来说，`requires_grad=True` 通常由 `nn.Parameter` 自动设置：

```
import torch.nn as nn
```

```
linear = nn.Linear(4, 2)
```

```
for name, param in linear.named_parameters():
    print(name, param.requires_grad)
```

输出一般是：

```
weight True
bias True
```

如果要冻结某些参数，可以设置：

```
for param in model.backbone.parameters():
    param.requires_grad = False
```

冻结参数后，这些参数不会累积梯度，也不会被优化器更新。微调大模型时，常见做法是冻结 base model，只训练 LoRA adapter 或某些 head；或者先冻结底层，再逐步解冻。

需要注意：冻结参数不等于整段 forward 都不建图。如果后续仍有可训练参数依赖这些中间结果，PyTorch 仍可能需要保留部分计算用于梯度传递。真正的推理阶段应该使用 `torch.no_grad()` 或 `torch.inference_mode()`。

2.4 叶子张量、非叶子张量和.grad

理解 `.grad` 时，必须理解叶子张量。

简单说：

1. 用户直接创建、且没有由其他可求导操作产生的 tensor，通常是叶子张量。
2. `nn.Parameter` 是叶子张量。
3. 由计算产生的中间结果通常是非叶子张量。
4. 默认只有叶子张量的 `.grad` 会在 `backward()` 后被保留下来。

示例：

```
x = torch.tensor(2.0, requires_grad=True)
y = x * 3
z = y * y
```

```
print(x.is_leaf) # True
print(y.is_leaf) # False
print(z.is_leaf) # False
```

```
z.backward()
```

```
print(x.grad) # 有值
print(y.grad) # 默认是 None, 并可能出现 warning
```

这不是 y 没有梯度，而是 PyTorch 默认不保存非叶子张量的 `.grad`。原因很直接：中间激活太多，如果都保存 `.grad`，显存会爆。

如果确实想观察某个中间 tensor 的梯度，可以使用 `retain_grad()`：

```
x = torch.tensor(2.0, requires_grad=True)
y = x * 3
y.retain_grad()
z = y * y
```

```
z.backward()
```

```
print(y.grad)
```

面试中如果被问“为什么我的中间 tensor `.grad` 是 `None`”，可以这样答：

PyTorch 默认只把梯度累积到叶子张量的 `.grad` 上，模型参数就是典型叶子张量。中间 tensor 虽然参与反向传播，但默认不保留 `.grad`。

2.5 backward 做了什么

`loss.backward()` 的本质是从 loss 对自己梯度为 1 开始，沿计算图反向应用链式法则。

以标量函数为例：

```
y=x^2,
\quad z=3y
```

那么：

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} = 3 \cdot 2x$$

对应代码：

```
x = torch.tensor(2.0, requires_grad=True)
y = x * x
z = 3 * y
```

```
z.backward()
print(x.grad) # tensor(12.)
```

如果输出不是标量，就不能直接 `backward()`，除非提供外部梯度：

```
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x * x
```

```
# y.backward() # RuntimeError: grad can be implicitly created only for scalar outputs
y.backward(torch.ones_like(y))
```

```
print(x.grad) # tensor([2., 4., 6.])
```

为什么标量 loss 可以不传 gradient？因为默认是：

$$\frac{\partial L}{\partial L} = 1$$

而向量输出需要告诉 autograd 你想计算哪个向量-雅可比积。PyTorch 的反向传播不是默认显式构造完整 Jacobian，而是高效计算 vector-Jacobian product。这一点在大模型里很重要，因为完整 Jacobian 可能大到不可接受。

2.6 梯度会累积，不会自动清零

PyTorch 的梯度默认是累积的。

```

x = torch.tensor(1.0, requires_grad=True)

y = x * 2
y.backward()
print(x.grad) # tensor(2.)

y = x * 3
y.backward()
print(x.grad) # tensor(5.), 不是 tensor(3.)

```

这就是训练循环里每一步都要清梯度的原因：

```

for batch in dataloader:
    optimizer.zero_grad()

    logits = model(batch["input_ids"])
    loss = criterion(logits, batch["labels"])

    loss.backward()
    optimizer.step()

```

如果忘记 `zero_grad()`，梯度会把多个 `step` 的结果加在一起，训练会变得不可控。

但梯度累积也可以被主动利用。比如显存只能放下小 `batch`，但想模拟大 `batch`：

```

accum_steps = 4
optimizer.zero_grad()

for step, batch in enumerate(dataloader):
    logits = model(batch["input_ids"])
    loss = criterion(logits, batch["labels"])
    loss = loss / accum_steps
    loss.backward()

    if (step + 1) % accum_steps == 0:
        optimizer.step()
        optimizer.zero_grad()

```

这里把 `loss` 除以 `accum_steps` 是为了让累积后的梯度尺度接近真实大 `batch` 的平均梯度。如果不除，等价于把学习率放大了约 `accum_steps` 倍。

更推荐的清梯度写法是：

```
optimizer.zero_grad(set_to_none=True)
```

它会把 `.grad` 设为 `None`，通常比填零更省内存和更快。需要注意，某些手写逻辑如果假设 `param.grad` 一定是 `tensor`，就要处理 `None` 情况。

2.7 detach：切断计算图

`detach()` 会返回一个与原 `tensor` 共享数据、但不再连接当前计算图的新 `tensor`。

```

x = torch.tensor(2.0, requires_grad=True)
y = x * 3
z = y.detach()

```

```

print(y.requires_grad) # True
print(z.requires_grad) # False

```

如果后续 `loss` 只依赖 `z`，梯度不会再传回 `x`：

```
x = torch.tensor(2.0, requires_grad=True)
y = x * 3
z = y.detach()
loss = z * z
```

`loss.backward()` # 会报错, 因为 `loss` 不需要梯度

常见使用场景:

1. 日志记录时把 `tensor` 从图里拿出来, 避免保留整张计算图。
2. 生成样本、缓存 `hidden states`、更新指标时不需要梯度。
3. 某些算法需要阻断一条分支的梯度, 例如 `target network`、`stop-gradient`、对比学习里的某些 `teacher` 分支。
4. RNN 或长序列训练中做 `truncated backpropagation through time`。

典型日志写法:

```
loss_value = loss.detach().item()
```

或者:

```
metrics["loss"] += loss.item()
```

`.item()` 会把单元素 `tensor` 转成 Python 数字, 也会脱离计算图。不要把带图的 `loss` 直接长期放进 `list`:

```
losses = []
```

```
for batch in dataloader:
    loss = compute_loss(batch)
    losses.append(loss) # 风险: 可能保留每一步的计算图
```

更安全的是:

```
losses.append(loss.detach().cpu())
```

或只保存数值:

```
losses.append(loss.item())
```

`detach` 的风险也很大: 如果误用在模型输出、`loss` 或中间 `hidden states` 上, 梯度会被切断, 导致参数不更新或部分模块学不到东西。

2.8 `no_grad` 和 `inference_mode`

`torch.no_grad()` 用于关闭梯度记录, 常用于验证和推理:

```
model.eval()
```

```
with torch.no_grad():
    logits = model(input_ids)
    probs = torch.softmax(logits, dim=-1)
```

在 `no_grad` 作用域里, 即使输入或参数 `requires_grad=True`, 新产生的结果也不会被 `autograd` 记录。这样可以:

1. 减少显存占用。
2. 加快推理速度。
3. 避免验证阶段误保留计算图。

`torch.inference_mode()` 比 `no_grad()` 更激进, 进一步关闭版本计数等 `autograd` 相关机制, 推理性能可能更好:

```
model.eval()
```

```
with torch.inference_mode():
    logits = model(input_ids)
```

一般经验：

1. 验证、推理、离线 embedding 抽取，优先使用 `torch.inference_mode()`。
2. 如果代码中需要创建后续还可能参与 autograd 的 tensor，或者存在特殊的 view/in-place 交互，使用 `torch.no_grad()` 更保守。
3. 训练阶段不要用 `no_grad()` 包住 forward，否则 loss 无法反向传播到参数。

`model.eval()` 和 `torch.no_grad()` 不是一回事：

1. `model.eval()` 影响 dropout、batch norm 等模块行为。
2. `torch.no_grad()` 影响 autograd 是否记录计算图。

验证时通常两个都要用：

```
model.eval()
with torch.no_grad():
    val_loss = evaluate(model, val_loader)
model.train()
```

面试回答：

`model.eval()` 只是切换模块行为，比如关闭 dropout、使用 batch norm 的 running statistics；`no_grad()` 是关闭梯度记录，减少显存。

2.9 in-place 操作为什么容易破坏反向传播

PyTorch 中以下划线结尾的操作通常是 in-place 操作，例如：

```
x.add_(1)
x.relu_()
x.masked_fill_(mask, 0)
```

in-place 操作会直接修改原 tensor 的数据。它可能更省内存，但也可能破坏 autograd 需要保存的中间值。

典型报错：

one of the variables needed for gradient computation has been modified by an inplace operation

直觉上，反向传播需要用前向时的某些值计算梯度。如果你在 backward 之前把这些值原地改掉，PyTorch 就无法保证梯度正确。

示例：

```
x = torch.randn(4, requires_grad=True)
y = x * 2
z = y * y

y.add_(1) # 修改了反向传播可能需要的中间值
loss = z.sum()
loss.backward()
```

并不是所有 in-place 操作都会报错，也不是所有带下划线的操作都绝对不能用。但在训练代码里，尤其是刚开始写模型时，建议优先使用非 in-place 版本：

```
x = x + 1
x = torch.relu(x)
x = x.masked_fill(mask, 0)
```

Transformer 实现中还要特别注意 mask：

```
scores = scores.masked_fill(causal_mask, float("-inf"))
```

比下面这种写法更安全：

```
scores.masked_fill_(causal_mask, float("-inf"))
```

面试中可以这样说：

in-place 操作会修改 tensor 本身，而 autograd 反向传播可能依赖前向保存的中间值。PyTorch 会用版本计数检查某些 tensor 是否

2.10 常见 Autograd 报错与定位方法

2.10.1 grad 是 None

常见原因：

1. 参数没有参与 loss 计算。
2. 参数的 `requires_grad=False`。
3. 中间某处用了 `detach()`、`.item()`、`torch.no_grad()` 切断计算图。
4. 你查看的是非叶子 tensor 的 `.grad`。
5. 优化器里没有包含这个参数。
6. 分支逻辑导致某个模块在当前 batch 没有被调用。

排查代码：

```
for name, param in model.named_parameters():
    if param.requires_grad and param.grad is None:
        print("missing grad:", name)
```

如果某些参数长期没有梯度，要检查 forward 路径和 loss 是否真的依赖它。

2.10.2 loss 不下降

loss 不下降不一定是 autograd 问题，但可以先排除几类梯度问题：

```
total_norm = 0.0
for param in model.parameters():
    if param.grad is not None:
        total_norm += param.grad.detach().norm().item()

print("grad_norm", total_norm)
```

如果梯度全是 0 或 None，可能是图断了、loss 写错、mask 把所有 token 都忽略了、学习率过小、激活饱和或初始化有问题。

如果梯度是 nan 或 inf，要检查学习率、混合精度、loss scale、softmax 前是否出现极端值、mask 是否产生全 -inf 行。

2.10.3 backward through graph a second time

常见原因：

1. 对同一个 loss 调用了两次 `backward()`。
2. 多个 loss 共用同一张计算图，但分开 backward 时没有保留图。
3. 循环里保存了带图 tensor，并在后续 step 重复参与计算。

可能修复：

1. 重新 forward 后再 backward。
2. 把多个 loss 加起来一次 backward: `(loss1 + loss2).backward()`。
3. 如果确实需要多次 backward，再考虑 `retain_graph=True`。
4. 对跨 step 保存的状态使用 `detach()`。

2.10.4 element does not require grad

典型报错：

element 0 of tensors does not require grad and does not have a grad_fn

常见原因：

1. loss 是在 `torch.no_grad()` 里算出来的。
2. loss 经过 `.item()` 变成 Python 数字后又包装回 tensor。
3. 模型输出被 `detach()` 了。
4. 所有参与计算的 tensor 都不需要梯度。

错误写法:

```
loss_value = loss.item()
loss = torch.tensor(loss_value, requires_grad=True)
loss.backward()
```

这样新建的 loss 已经和原模型计算图没有关系, 即使能 backward, 也不会更新模型参数。

2.11 autograd.grad、二阶梯度和高阶求导

除了 `loss.backward()`, PyTorch 还提供 `torch.autograd.grad`, 用于直接返回某些输入的梯度, 而不是累积到 `.grad` 上。

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3
```

```
grad_x = torch.autograd.grad(y, x)
print(grad_x) # (tensor(12.),)
```

如果要继续对梯度求导, 需要 `create_graph=True`:

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3
```

```
grad_x = torch.autograd.grad(y, x, create_graph=True)[0]
grad2_x = torch.autograd.grad(grad_x, x)[0]
```

```
print(grad_x) # tensor(12., grad_fn=...)
print(grad2_x) # tensor(12.)
```

常见场景包括:

1. 梯度惩罚。
2. 元学习。
3. 隐式优化。
4. 某些物理信息神经网络。
5. 研究二阶优化或 Hessian 相关性质。

高阶梯度会显著增加显存和计算开销。大模型训练主流程中很少直接使用完整二阶梯度, 更多是使用一阶优化器, 或者用近似方法估计曲率信息。

2.12 hook: 观察和修改梯度

hook 可以在反向传播时观察梯度, 常用于 debug。

```
def print_grad(grad):
    print("grad mean:", grad.mean().item())
```

```
x = torch.randn(4, requires_grad=True)
y = (x * x).sum()
```

```
handle = x.register_hook(print_grad)
y.backward()
handle.remove()
```

对模型参数也可以注册 hook:


```

for name, param in model.named_parameters():
    if param.requires_grad:
        param.register_hook(lambda grad, name=name: print(name, grad.norm().item()))

```

hook 的用途：

1. 查看某层梯度是否为 0、nan 或异常大。
2. 定位梯度从哪一层开始消失或爆炸。
3. 实现简单的梯度裁剪、mask 或自定义调试逻辑。

但训练主逻辑中不要滥用 hook。hook 会让梯度路径变得隐蔽，降低代码可读性，也可能引入性能开销。正式工程里，优先用清晰的 forward/loss 写法和显式日志；hook 更适合作为临时 debug 工具。

2.13 训练循环里的正确顺序

标准训练 step 通常是：

```
model.train()
```

```

for batch in train_loader:
    optimizer.zero_grad(set_to_none=True)

    input_ids = batch["input_ids"].to(device)
    labels = batch["labels"].to(device)

    logits = model(input_ids)
    loss = criterion(logits, labels)

    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()

```

关键点：

1. zero_grad 通常放在 forward 前，避免旧梯度残留。
2. forward 和 loss 计算必须在梯度开启的环境里。
3. backward 后参数的 .grad 才有值。
4. gradient clipping 要放在 backward 后、optimizer.step 前。
5. optimizer.step 后参数发生更新。
6. 日志记录时用 loss.item() 或 loss.detach()。

验证循环则不同：

```

model.eval()
total_loss = 0.0

```

```

with torch.no_grad():
    for batch in val_loader:
        input_ids = batch["input_ids"].to(device)
        labels = batch["labels"].to(device)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        total_loss += loss.item()

```

```
model.train()
```

如果验证时忘记 no_grad()，结果通常仍然正确，但会构建计算图，导致显存更高、速度更慢。如果还把验证 loss tensor 存起来，可能造成显存持续增长。

2.14 大模型工程中的 Autograd 边界

在大模型训练里，autograd 问题通常不只出现在简单的 `loss.backward()`，而是出现在模块边界和工程优化里。

2.14.1 LoRA 和参数冻结

LoRA 微调时，常见目标是只训练 adapter 参数：

```
for name, param in model.named_parameters():
    param.requires_grad = "lora" in name
```

排查时要确认：

1. 冻结的 base 参数 `requires_grad=False`。
2. LoRA 参数 `requires_grad=True`。
3. 优化器只包含可训练参数。
4. LoRA 分支确实参与 forward。

```
trainable_params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.AdamW(trainable_params, lr=1e-4)
```

2.14.2 梯度检查点

Gradient checkpointing 会在 forward 时不保存部分中间激活，backward 时重新计算它们，以计算换显存。它仍然依赖 autograd，但改变了中间激活的保存策略。

直觉：

1. 普通训练：forward 保存激活，backward 直接使用。
2. checkpoint：forward 少保存激活，backward 重新执行部分 forward。

优点是省显存，缺点是训练更慢。大模型训练里经常用它换更长序列、更大 batch 或更大模型。

2.14.3 混合精度和 GradScaler

混合精度训练中，autograd 仍然负责求导，但部分 forward/backward 以 fp16/bf16 执行。fp16 容易出现 underflow，因此常和 loss scaling 配合。

典型 AMP 训练结构：

```
scaler = torch.cuda.amp.GradScaler()
```

```
for batch in train_loader:
    optimizer.zero_grad(set_to_none=True)

    with torch.cuda.amp.autocast():
        logits = model(batch["input_ids"])
        loss = criterion(logits, batch["labels"])
```

```
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
```

这里梯度裁剪前要先 `unscale_`，否则裁剪的是被 scale 过的梯度。bf16 动态范围更大，很多大模型训练中不一定需要 GradScaler。

2.14.4 分布式训练

在 DistributedDataParallel 中，每个 rank 前向和反向自己的 mini-batch。反向传播过程中，DDP 会通过 autograd hook 在梯度 ready 时触发 all-reduce，把各 rank 的梯度同步。

工程上常见问题：

1. 某些参数在某些 rank 没参与 forward，导致梯度同步异常。
2. 条件分支让不同 rank 走了不同计算路径。
3. 梯度累积时忘记使用 no_sync()，导致每个 micro-step 都同步，性能变差。
4. 手动 detach 或 no_grad 切断了需要同步的参数梯度。

这些内容会在后续分布式训练章节展开，本章只需要记住：DDP 的梯度同步也是挂在 autograd 反向过程上的。

2.15 最小可运行 Autograd 审计 demo

下面用一个 demo 同时检查标量梯度、向量输出的外部梯度、leaf / non-leaf、retain_grad、梯度累积、detach、no_grad、optimizer 训练顺序和缺失梯度排查。

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
torch.manual_seed(13)
```

```
# 1. Scalar backward:  $y = x^3 + 2x$ ,  $dy/dx$  at  $x=2$  is 14.
```

```
x = torch.tensor(2.0, requires_grad=True)
```

```
y = x ** 3 + 2 * x
```

```
y.backward()
```

```
scalar_grad = float(x.grad)
```

```
# 2. Vector-Jacobian product:  $y = x^2$ ,  $v=[1,2,3]$ .
```

```
vjp_x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
```

```
vjp_y = vjp_x ** 2
```

```
vjp_y.backward(torch.tensor([1.0, 2.0, 3.0]))
```

```
vjp_grad = vjp_x.grad.detach().tolist()
```

```
# 3. Leaf / non-leaf and retain_grad.
```

```
leaf = torch.tensor(3.0, requires_grad=True)
```

```
middle = leaf * 4
```

```
middle.retain_grad()
```

```
out = middle ** 2
```

```
out.backward()
```

```
leaf_grad = float(leaf.grad)
```

```
middle_grad = float(middle.grad)
```

```
middle_is_leaf = middle.is_leaf
```

```
# 4. Gradients accumulate unless cleared.
```

```
accum = torch.tensor(1.0, requires_grad=True)
```

```
(accum * 2).backward()
```

```
first_grad = float(accum.grad)
```

```
(accum * 3).backward()
```

```
second_grad = float(accum.grad)
```

```
accum.grad = None
```

```
(accum * 3).backward()
```

```
after_clear_grad = float(accum.grad)
```

```

# 5. detach cuts graph on that branch.
base = torch.tensor(2.0, requires_grad=True)
tracked = base * 5
stopped = tracked.detach()
detach_loss = tracked + stopped
detach_loss.backward()
detach_grad = float(base.grad)

# 6. Minimal training loop with optimizer and zero_grad.
model = nn.Linear(2, 1)
inputs = torch.randn(16, 2)
true_w = torch.tensor([[2.0], [-3.0]])
targets = inputs @ true_w + 0.5

optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
loss_trace = []
grad_norm_trace = []

for _ in range(8):
    pred = model(inputs)
    loss = F.mse_loss(pred, targets)

    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    grad_norm = sum(
        p.grad.detach().norm().item()
        for p in model.parameters()
        if p.grad is not None
    )
    optimizer.step()

    loss_trace.append(round(float(loss.detach()), 4))
    grad_norm_trace.append(round(grad_norm, 4))

# 7. no_grad / eval: validation should not build graph.
model.eval()
with torch.no_grad():
    val_pred = model(inputs)
    val_loss = F.mse_loss(val_pred, targets)
model.train()

missing_grad = []
for name, param in model.named_parameters():
    if param.requires_grad and param.grad is None:
        missing_grad.append(name)

report = {
    "scalar_grad": round(scalar_grad, 3),
    "vjp_grad": [round(v, 3) for v in vjp_grad],
    "leaf_nonleaf": {
        "middle_is_leaf": middle_is_leaf,
        "leaf_grad": round(leaf_grad, 3),
        "middle_grad_after_retain": round(middle_grad, 3),
    },
    "grad_accumulation": {

```

```

        "first": round(first_grad, 3),
        "second_without_clear": round(second_grad, 3),
        "after_clear": round(after_clear_grad, 3),
    },
    "detach": {
        "base_grad": round(detach_grad, 3),
        "expected_only_tracked_branch": 5.0,
    },
    "training": {
        "loss_trace": loss_trace,
        "grad_norm_trace": grad_norm_trace,
        "final_loss": round(float(val_loss), 4),
        "val_requires_grad": val_loss.requires_grad,
        "missing_grad_after_last_step": missing_grad,
    },
    "checks": {
        "scalar_grad_ok": abs(scalar_grad - 14.0) < 1e-6,
        "vjp_grad_ok": [round(v, 3) for v in vjp_grad] == [2.0, 8.0, 18.0],
        "nonleaf_grad_retained": middle_grad == 24.0,
        "grad_accumulates": second_grad == first_grad + 3.0,
        "detach_blocks_one_branch": detach_grad == 5.0,
        "training_loss_decreases": loss_trace[-1] < loss_trace[0],
        "no_grad_validation": val_loss.requires_grad is False,
    },
}

```

print(report)

这段代码的关键点：

1. 标量 backward() 默认使用 $dL/dL=1$ 。
2. 向量输出必须传入外部梯度，本质是 vector-Jacobian product。
3. 非叶子 tensor 默认不保留 .grad，调用 retain_grad() 后才能观察。
4. .grad 默认累积，清空后再 backward 才是当前图的梯度。
5. detach() 只切断被 detach 的分支，未 detach 的分支仍能反传。
6. 验证阶段 no_grad() 下的 loss 不应继续挂在计算图上。
7. 真实训练通常用 optimizer 更新参数，而不是在图里手动构造新 tensor。

2.16 Debug 清单

遇到梯度问题时，可以按下面顺序查：

1. loss.requires_grad 是否为 True。
2. loss.grad_fn 是否为 None。
3. 关键参数的 requires_grad 是否为 True。
4. loss 是否真的依赖目标参数。
5. 是否误用了 detach()、.item()、no_grad()。
6. 是否对同一张图 backward 了多次。
7. 是否忘记 optimizer.zero_grad()。
8. 是否在 backward() 前做了破坏计算图的 in-place 操作。
9. 是否把非叶子 tensor 的 .grad is None 误判成没有梯度。
10. optimizer 是否包含了需要训练的参数。
11. 梯度中是否出现 nan 或 inf。
12. 分布式训练中各 rank 是否走了相同的参数使用路径。

常用检查代码：

```
print("loss requires_grad:", loss.requires_grad)
```

```
print("loss grad_fn:", loss.grad_fn)
```

```
for name, param in model.named_parameters():  
    if param.requires_grad:  
        grad_status = None if param.grad is None else param.grad.norm().item()  
        print(name, grad_status)
```

如果怀疑有 in-place 问题，可以临时开启 anomaly detection：

```
torch.autograd.set_detect_anomaly(True)
```

或者：

```
with torch.autograd.detect_anomaly():  
    loss.backward()
```

它会让 PyTorch 尝试报告导致 backward 出错的前向位置，但会明显降低速度，只适合 debug，不适合长期训练开启。

2.17 面试官会怎么问

问题一：PyTorch 的 autograd 是怎么工作的？

回答模板：

PyTorch 使用动态计算图。每次 forward 时，如果输入 tensor 需要梯度，相关操作会被记录成计算图节点，并保存 backward 所需的

问题二：为什么每次训练都要 zero_grad？

回答模板：

因为 PyTorch 的 .grad 默认是累积的，不会在 backward 前自动清零。这样设计可以支持梯度累积、多 loss 反传等场景。普通训练中

问题三：detach 和 no_grad 有什么区别？

回答模板：

detach 是对某个 tensor 切断它和当前计算图的连接，返回一个不再追踪梯度的新 tensor，常用于日志、缓存或 stop-gradient。no_grad 是一个上下文管理器，在作用域内关闭新操作的梯度记录，常用于验证和推理。detach 更像是切断某条边，no_g

问题四：为什么中间 tensor 的.grad 是 None？

回答模板：

默认只有叶子 tensor 的 .grad 会被保留，模型参数就是叶子 tensor。中间 tensor 虽然有梯度流过，但为了节省显存，PyTorch 不会

问题五：为什么验证时要同时写 model.eval() 和 no_grad()？

回答模板：

model.eval() 改变模块行为，比如关闭 dropout、让 batch norm 使用 running statistics；no_grad() 关闭 autograd 记录，减少显

问题六：in-place 操作为什么可能导致 backward 报错？

回答模板：

反向传播可能需要前向时保存的中间值。in-place 操作会直接改写 tensor，如果改掉了 backward 需要的值，梯度就可能不正确。Py

2.18 常见误区

1. 以为 `model.eval()` 会关闭梯度。它不会，关闭梯度要用 `no_grad()` 或 `inference_mode()`。
2. 以为 `loss.item()` 后还能反向传播。`.item()` 得到的是 Python 数字，已经脱离计算图。
3. 以为 `.grad is None` 就一定没有梯度。中间 tensor 默认不保留 `.grad`。
4. 以为 `retain_graph=True` 是解决 backward 报错的常规方案。多数时候应该重新 forward 或修正图复用逻辑。
5. 以为冻结参数后整个 forward 都不建图。如果后面还有可训练参数，仍可能需要保留部分图。
6. 以为 in-place 一定更好。它可能省一点内存，但会增加 autograd 风险。
7. 以为梯度累积只要多次 backward 就行。通常还要按累积步数缩放 loss，并控制 optimizer step 的频率。
8. 以为手动 `param = param - lr * grad` 能更新模型参数。这样可能创建新 tensor 而不是更新原参数，正确做法是 optimizer 或在 `no_grad` 下原地更新参数数据。

2.19 小练习

1. 写一个标量函数 $y = x^3 + 2 * x$ ，用 autograd 验证在 $x=2$ 时的梯度。
2. 构造一个两层 MLP，打印每个参数的 `requires_grad`、`is_leaf` 和 backward 后的梯度范数。
3. 故意在训练循环中去掉 `optimizer.zero_grad()`，观察 loss 和梯度范数变化。
4. 写一个例子保存每一步的 loss tensor 到 list，再改成保存 `loss.item()`，比较显存或对象引用差异。
5. 在一个中间激活上调用 `retain_grad()`，观察它的 `.grad`。
6. 把验证循环分别写成只用 `model.eval()`、只用 `no_grad()`、两者都用，解释差异。
7. 构造一个误用 `detach()` 导致某层参数没有梯度的例子，并用 `named_parameters()` 排查。

2.20 本章总结

Autograd 的核心是动态计算图和链式法则。PyTorch 在 forward 时记录可求导操作，在 `loss.backward()` 时沿图反向传播，把梯度累积到叶子参数的 `.grad` 上。默认情况下，计算图在 backward 后释放，梯度不会自动清零，中间 tensor 的 `.grad` 不会被保留。

工程上最重要的不是背 API，而是知道 autograd 的边界：`requires_grad` 决定是否追踪梯度，`detach` 会切断某个 tensor 的图连接，`no_grad` 和 `inference_mode` 会关闭一段代码的建图，in-place 操作可能破坏反向传播，`retain_graph=True` 不能滥用。

如果能稳定回答“这个 loss 是否依赖这个参数”“这段代码有没有建图”“这个 tensor 是叶子还是中间结果”“梯度为什么是 None”“验证为什么要关梯度”，就已经具备了写训练脚本和排查大部分 PyTorch 梯度问题的基础。

第三章：nn.Module 与模型组织

`nn.Module` 是 PyTorch 组织模型的核心抽象。第一章讲的是 tensor，第二章讲的是 autograd；到了真实工程里，我们不会把所有参数、矩阵乘法和激活函数都散写在训练脚本中，而是把它们封装成一个个 module，再组合成完整模型。

很多 PyTorch 初学者会把 `nn.Module` 理解成“写神经网络时必须继承的类”，但这只是表层。`nn.Module` 真正重要的地方在于：它负责注册参数、注册子模块、管理 buffer、递归切换训练/推理模式、递归移动 device、导出和加载 `state_dict`，并为优化器、分布式训练、混合精度和模型保存提供统一接口。

如果你不理解这些机制，很容易遇到这类问题：明明定义了参数但 optimizer 没有更新、保存的 checkpoint 里缺字段、`model.eval()` 后结果仍然随机、加载权重时报 missing key、把 tensor 放进 list 里导致参数没有注册、推理时 batch norm 或 dropout 行为不对、LoRA 参数没有进入 optimizer、DDP 包装后 key 多了 `module`。前缀。

本章目标是把 `nn.Module` 在工程中的规则讲清楚：如何写 forward，什么是 Parameter，子模块如何注册，`state_dict` 保存什么，`train()` 和 `eval()` 到底影响什么，模型保存加载应该怎么做，以及如何组织一个可维护的大模型代码结构。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前,已核对 PyTorch 官方 `nn.Module`、`nn.Parameter`、`register_buffer`、`state_dict`、`load_state_dict`、`ModuleList`、`ModuleDict`、`ParameterList`、`ParameterDict`、`train()` / `eval()`、模型保存加载教程和 `torch.compile` 文档口径。

本章聚焦大模型工程最常用的 Module 组织问题: 参数注册、子模块注册、`buffer`、`state_dict`、保存加载、训练/推理模式、`device` / `dtype` 迁移、参数冻结、参数分组、`hook`、`DDP` / `FSDP` 依赖的模块边界, 以及最小可运行的 Module 审计 demo。

本章不展开 C++ dispatcher、底层 autograd engine、完整分布式训练实现、`FSDP wrap policy` 细节、`torch.export` / `AOTAutograd` 编译器内部机制或生产级 checkpoint shard 格式。这些内容会分别放到分布式训练、profiling、推理部署和 AI Infra 相关章节中讲。

3.1 nn.Module 解决什么问题

先看一个不用 `nn.Module` 的极简线性模型:

```
import torch

w = torch.randn(10, 1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)

x = torch.randn(4, 10)
y = x @ w + b
loss = y.pow(2).mean()
loss.backward()
```

这段代码可以求梯度, 但它不是一个好工程结构。问题在于:

1. 参数 `w`、`b` 散落在外部变量中。
2. 训练脚本需要手动知道哪些 `tensor` 是参数。
3. 保存加载时需要自己维护字段名。
4. 复杂模型会有大量层、`buffer` 和子结构, 无法靠几个变量管理。
5. 分布式、混合精度、设备迁移都需要统一遍历模型内部对象。

`nn.Module` 提供了统一容器:

```
import torch
from torch import nn

class LinearModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 1)

    def forward(self, x):
        return self.linear(x)

model = LinearModel()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

x = torch.randn(4, 10)
y = model(x)
loss = y.pow(2).mean()
loss.backward()
optimizer.step()
```


这里 `self.linear = nn.Linear(10, 1)` 不只是普通赋值。因为 `LinearModel` 继承了 `nn.Module`, PyTorch 会把 `linear` 识别为子模块并注册起来。后续 `model.parameters()`、`model.state_dict()`、`model.to(device)`、`model.train()` 都会递归作用到它。

面试回答：

`nn.Module` 是 PyTorch 中组织模型的核心抽象。它不仅封装 `forward`，还会自动注册 `Parameter`、子模块和 `buffer`，并支持递归遍历。

3.1.1 关键机制公式与 Module 调试速查

`nn.Module` 本身不是一个数学层，而是一套工程组织规则。面试和 debug 时，可以把它拆成几条稳定口径。

第一，参数量来自注册参数，而不是来自 Python 变量名。线性层参数量为：

$$N = d_{in}d_{out} + d_{out}$$

如果 `bias=False`，则只有：

$$N = d_{in}d_{out}$$

Embedding 表的参数量为：

$$N_{emb} = Vd$$

其中 `V` 是词表大小，`d` 是 hidden size。一个两层 MLP 的参数量常写成：

$$N_{mlp} = d_{in}d_h + d_h + d_hd_{out} + d_{out}$$

第二，optimizer 更新的是它拿到的 `Parameter`。单个参数的 SGD 更新可以写成：

$$\theta_{t+1} = \theta_t - \eta g_t$$

$$g_t = \frac{\partial L}{\partial \theta_t}$$

如果某个 tensor 没有注册成 `Parameter`，或者某个子模块藏在普通 list / dict 中，它即使参与 `forward`，也不会自动出现在 `model.parameters()` 里，optimizer 通常不会更新它。

第三，`state_dict` 不是保存整个 Python 对象，而是保存注册权重和持久化 `buffer`：

$$S(M) = P(M) \cup B_p(M)$$

其中 `P(M)` 表示 module 树上的注册参数，`B_p(M)` 表示 `persistent=True` 的 `buffer`。普通属性、临时 tensor、`persistent=False` 的 `buffer`、`forward` 代码和 Python 类定义都不在模型 `state_dict` 里。

第四，参数高效微调或冻结模型时，先看可训练参数比例：

$$r = \frac{\sum_{p \in P_{train}} |p|}{\sum_{p \in P_{all}} |p|}$$

其中 `|p|` 是一个参数 tensor 的元素个数。LoRA、adapter、prompt tuning 的核心工程检查就是：应该训练的参数是否都被注册、`requires_grad=True`、进入 optimizer，并且保存时能从 `state_dict` 或 `named_parameters()` 中筛出来。

第五, `train()` / `eval()` 改的是模块状态, 不是梯度开关:

```
model.eval()
with torch.no_grad():
    outputs = model(inputs)
```

`eval()` 递归设置 `training=False`, 影响 Dropout、BatchNorm 等层; `no_grad()` / `inference_mode()` 关闭 autograd 记录。验证和推理通常两者都要用。

最小 Module 审计清单:

1. `list(model.named_parameters())` 中是否出现目标层参数。
2. `list(model.named_buffers())` 中是否出现固定 mask、统计量或位置编码。
3. `model.state_dict().keys()` 是否包含该保存的权重, 是否排除了临时缓存。
4. `sum(p.numel() for p in model.parameters() if p.requires_grad)` 是否等于 optimizer 参数组里的元素数。
5. `model.train()` / `model.eval()` 是否递归影响所有子模块。
6. 保存、加载后同一输入在 eval 模式下输出是否一致。
7. `strict=False` 加载时的 missing / unexpected keys 是否符合预期, 而不是被忽略。

3.2 最小 Module 写法

一个标准的 module 通常包含两部分:

1. `__init__`: 定义层、参数、buffer 和配置。
2. `forward`: 定义前向计算逻辑。

示例:

```
from torch import nn
```

```
class MLP(nn.Module):
    def __init__(self, hidden_size, intermediate_size):
        super().__init__()
        self.fc1 = nn.Linear(hidden_size, intermediate_size)
        self.act = nn.GELU()
        self.fc2 = nn.Linear(intermediate_size, hidden_size)

    def forward(self, x):
        x = self.fc1(x)
        x = self.act(x)
        x = self.fc2(x)
        return x
```

几个关键习惯:

1. 必须先调用 `super().__init__()`。
2. 在 `__init__` 中创建需要长期持有的层和参数。
3. 在 `forward` 中写真实计算, 不要在里面反复创建可训练层。
4. `forward` 可以使用 Python 控制流, PyTorch 动态图会按实际路径构图。

常见错误是把层写在 `forward` 里:

```
class BadMLP(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size

    def forward(self, x):
        layer = nn.Linear(self.hidden_size, self.hidden_size)
        return layer(x)
```

这段代码的问题很严重：每次 forward 都会新建一组随机参数，这些参数不会被稳定训练，optimizer 也不知道它们应该被长期更新。除非你明确要创建临时无参数对象，否则可训练层应该放在 `__init__` 中。

正确写法：

```
class GoodMLP(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.layer = nn.Linear(hidden_size, hidden_size)

    def forward(self, x):
        return self.layer(x)
```

3.3 forward 和 call 的关系

使用模型时，我们通常写：

```
output = model(x)
```

而不是：

```
output = model.forward(x)
```

虽然最终会执行 forward，但两者不完全等价。`model(x)` 会调用 `nn.Module.__call__`，它在内部处理 hook、autograd 相关状态、编译包装以及其他框架机制，然后再进入 forward。

因此工程中建议：

1. 外部调用永远写 `model(inputs)`。
2. 不要直接调用 `model.forward(inputs)`，除非是在非常明确的内部调试场景。
3. 自定义逻辑写在 forward 中，不要重写 `__call__`。

例如：

```
class ToyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(4, 2)

    def forward(self, x):
        return self.linear(x)
```

```
model = ToyModel()
out = model(torch.randn(3, 4))
```

面试回答：

PyTorch 里应该调用 `model(x)`，而不是 `model.forward(x)`。`model(x)` 会走 `Module` 的 `__call__` 逻辑，里面会处理 hook、状态和框架

3.4 Parameter：什么会被 optimizer 更新

`nn.Parameter` 是一种特殊 tensor。它的作用是告诉 `nn.Module`：这个 tensor 是模型参数，需要被注册，应该出现在 `model.parameters()` 和 `state_dict()` 中。

示例：

```
import torch
from torch import nn
```

```
class ScaleLayer(nn.Module):
```

```

def __init__(self, hidden_size):
    super().__init__()
    self.scale = nn.Parameter(torch.ones(hidden_size))

def forward(self, x):
    return x * self.scale

```

检查参数:

```
layer = ScaleLayer(4)
```

```

for name, param in layer.named_parameters():
    print(name, param.shape, param.requires_grad)

```

输出类似:

```
scale torch.Size([4]) True
```

如果你只是写普通 tensor:

```

class BadScaleLayer(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.scale = torch.ones(hidden_size, requires_grad=True)

    def forward(self, x):
        return x * self.scale

```

这个 scale 虽然 requires_grad=True, 但它不是 Parameter, 不会出现在 model.parameters() 中, optimizer 通常也不会更新它。

可以验证:

```

bad = BadScaleLayer(4)
print(list(bad.named_parameters())) # []

```

核心规则:

1. 需要训练的长期权重, 用 nn.Parameter 或标准层如 nn.Linear。
2. 中间激活不要定义成 Parameter。
3. 不需要训练但要随模型保存的状态, 用 buffer。
4. requires_grad=False 的 Parameter 仍是参数, 只是不计算梯度。

3.5 子模块注册: 赋值不是普通赋值

当你把一个 nn.Module 赋值给另一个 module 的属性时, 它会被自动注册为子模块。

```

class Block(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.norm = nn.LayerNorm(hidden_size)
        self.mlp = nn.Sequential(
            nn.Linear(hidden_size, 4 * hidden_size),
            nn.GELU(),
            nn.Linear(4 * hidden_size, hidden_size),
        )

    def forward(self, x):
        return x + self.mlp(self.norm(x))

```

self.norm 和 self.mlp 都会被注册。self.mlp 里面的 Linear 也会被递归注册。

可以查看模块树:

```
block = Block(768)
```

```
for name, module in block.named_modules():  
    print(name, type(module).__name__)
```

输出类似:

```
Block  
norm LayerNorm  
mlp Sequential  
mlp.0 Linear  
mlp.1 GELU  
mlp.2 Linear
```

这里第一个空 name 表示根模块本身。

常见坑是把子模块放进普通 Python list:

```
class BadStack(nn.Module):  
    def __init__(self, hidden_size, num_layers):  
        super().__init__()  
        self.layers = [Block(hidden_size) for _ in range(num_layers)]  
  
    def forward(self, x):  
        for layer in self.layers:  
            x = layer(x)  
        return x
```

这段代码能 forward, 但 self.layers 里的 block 不会被注册为子模块。后果是:

1. model.parameters() 找不到这些参数。
2. model.to("cuda") 不会移动这些参数。
3. state_dict() 不会保存这些权重。
4. DDP 不会正确同步这些参数。

正确写法是 nn.ModuleList:

```
class GoodStack(nn.Module):  
    def __init__(self, hidden_size, num_layers):  
        super().__init__()  
        self.layers = nn.ModuleList([  
            Block(hidden_size) for _ in range(num_layers)  
        ])  
  
    def forward(self, x):  
        for layer in self.layers:  
            x = layer(x)  
        return x
```

如果是按名称组织, 可以用 nn.ModuleDict:

```
class MultiHeadModel(nn.Module):  
    def __init__(self, hidden_size):  
        super().__init__()  
        self.heads = nn.ModuleDict({  
            "classification": nn.Linear(hidden_size, 2),  
            "regression": nn.Linear(hidden_size, 1),  
        })
```

```
def forward(self, x, task):
    return self.heads[task](x)
```

3.6 Sequential、ModuleList、ModuleDict 怎么选

三者都能组织子模块，但语义不同。

`nn.Sequential` 表示线性流水线：前一个模块的输出直接作为后一个模块的输入。

```
mlp = nn.Sequential(
    nn.Linear(128, 512),
    nn.GELU(),
    nn.Linear(512, 128),
)
```

适合简单串联结构。缺点是不适合复杂控制流、多输入多输出或 residual connection。

`nn.ModuleList` 表示一组已注册模块，但不规定 forward 如何执行。

```
self.layers = nn.ModuleList([
    Block(hidden_size) for _ in range(num_layers)
])
```

适合 Transformer block 堆叠，因为 forward 通常要加入 cache、mask、gradient checkpointing、输出 hidden states 等逻辑。

`nn.ModuleDict` 表示按字符串 key 管理子模块。

```
self.adapters = nn.ModuleDict({
    "domain_a": Adapter(hidden_size),
    "domain_b": Adapter(hidden_size),
})
```

适合多任务 head、多 adapter、多分支结构。

选择建议：

1. 简单串联用 `Sequential`。
2. 多层堆叠但 forward 需要自己控制，用 `ModuleList`。
3. 按名称动态选择子模块，用 `ModuleDict`。
4. 不要用普通 list 或 dict 存放可训练子模块。

3.7 ParameterList 和 ParameterDict

如果你存的是参数而不是子模块，可以用 `nn.ParameterList` 和 `nn.ParameterDict`。

示例：

```
class WeightedSum(nn.Module):
    def __init__(self, num_inputs):
        super().__init__()
        self.weights = nn.ParameterList([
            nn.Parameter(torch.ones(())) for _ in range(num_inputs)
        ])

    def forward(self, xs):
        out = 0
        for weight, x in zip(self.weights, xs):
            out = out + weight * x
        return out
```

如果直接用普通 list：

```
self.weights = [nn.Parameter(torch.ones(())) for _ in range(num_inputs)]
```

这些参数不会被注册。

ParameterDict 适合按名字管理参数：

```
class TaskBias(nn.Module):
    def __init__(self, tasks, hidden_size):
        super().__init__()
        self.biases = nn.ParameterDict({
            task: nn.Parameter(torch.zeros(hidden_size))
            for task in tasks
        })

    def forward(self, x, task):
        return x + self.biases[task]
```

在大模型工程中，直接使用 ParameterList 和 ParameterDict 的机会不如 ModuleList 多，但在自定义 embedding、adapter、可学习 prompt、可学习加权融合时很有用。

3.8 buffer：不是参数，但属于模型状态

有些 tensor 不需要训练，但仍然属于模型状态，需要随模型移动 device、保存和加载。例如：

1. BatchNorm 的 running mean 和 running variance。
2. 固定的位置编码。
3. attention mask 的固定模板。
4. 量化时的 scale、zero point。
5. 一些统计量或缓存状态。

这类 tensor 应该注册为 buffer：

```
class PositionalEncoding(nn.Module):
    def __init__(self, max_len, hidden_size):
        super().__init__()
        pe = torch.zeros(max_len, hidden_size)
        self.register_buffer("pe", pe)

    def forward(self, x):
        seq_len = x.size(1)
        return x + self.pe[:seq_len]
```

buffer 的特点：

1. 不会出现在 model.parameters() 中。
2. 默认会出现在 state_dict() 中。
3. 会随 model.to(device) 移动。
4. 可以通过 persistent=False 设置为不保存。

例如：

```
self.register_buffer("causal_mask", mask, persistent=False)
```

persistent=False 适合可重新生成的大缓存，避免 checkpoint 过大。

查看 buffer：

```
for name, buf in model.named_buffers():
    print(name, buf.shape, buf.device)
```

面试回答：

Parameter 是需要被优化器更新的模型权重；buffer 是不训练但属于模型状态的 tensor，比如 BatchNorm running mean 或固定 mask。

3.9 parameters、named_parameters 和参数分组

`model.parameters()` 返回模型中所有注册参数，通常直接传给 `optimizer`：

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
```

但真实训练里经常需要参数分组，例如：

1. 对 `bias` 和 `LayerNorm` 不使用 `weight decay`。
2. `backbone` 和 `head` 使用不同学习率。
3. 只训练 LoRA 或 adapter 参数。
4. 冻结一部分参数。

这时需要 `named_parameters()`：

```
decay_params = []  
no_decay_params = []
```

```
for name, param in model.named_parameters():  
    if not param.requires_grad:  
        continue  
    if name.endswith("bias") or "norm" in name.lower():  
        no_decay_params.append(param)  
    else:  
        decay_params.append(param)
```

```
optimizer = torch.optim.AdamW([  
    {"params": decay_params, "weight_decay": 0.01},  
    {"params": no_decay_params, "weight_decay": 0.0},  
, lr=1e-4)
```

为什么 `LayerNorm` 和 `bias` 常常不做 `weight decay`？直觉上，`weight decay` 更适合同约束大矩阵权重，而归一化层的 `scale`、`bias` 和偏置参数承担的是平移缩放作用，强行衰减可能影响训练稳定性。不同项目会有不同策略，但面试中要能说明这是参数分组问题。

冻结参数：

```
for name, param in model.named_parameters():  
    if name.startswith("backbone"):  
        param.requires_grad = False
```

```
optimizer = torch.optim.AdamW(  
    [p for p in model.parameters() if p.requires_grad],  
    lr=1e-4,  
)
```

注意：修改 `requires_grad` 后，最好重新创建 `optimizer`，避免 `optimizer` 里仍然持有不该更新的参数组。

3.10 state_dict 保存什么

`state_dict()` 是 PyTorch 保存模型权重的标准方式。

```
state = model.state_dict()
```

```
for key, value in state.items():  
    print(key, value.shape)
```

它保存的是：

1. 所有注册参数。
2. 默认持久化的 `buffer`。

它不保存：

1. Python 类定义本身。
2. forward 代码逻辑。
3. optimizer 状态，除非单独保存 optimizer 的 state_dict()。
4. 未注册在 module 上的普通 tensor。
5. persistent=False 的 buffer。

以一个简单模型为例：

```
model = nn.Sequential(
    nn.Linear(4, 8),
    nn.LayerNorm(8),
    nn.Linear(8, 2),
)

print(model.state_dict().keys())
```

可能输出：

```
dict_keys([
    '0.weight', '0.bias',
    '1.weight', '1.bias',
    '2.weight', '2.bias'
])
```

key 来自模块层级路径。大模型里的 key 可能是：

```
model.embed_tokens.weight
model.layers.0.self_attn.q_proj.weight
model.layers.0.self_attn.k_proj.weight
model.layers.0.mlp.gate_proj.weight
model.norm.weight
lm_head.weight
```

理解 key 的层级非常重要，因为加载权重、排查 missing key、做 LoRA merge、切分 checkpoint 都依赖这些名称。

3.11 保存和加载模型

推荐保存方式是保存 state_dict，而不是直接保存整个模型对象。

保存：

```
torch.save(model.state_dict(), "model.pt")
```

加载：

```
model = MyModel(config)
state = torch.load("model.pt", map_location="cpu")
model.load_state_dict(state)
```

为什么不推荐直接保存整个模型？

```
torch.save(model, "model.pt")
```

这种方式依赖 Python pickle，需要加载时能找到完全一致的类路径和代码结构。工程迭代后很容易因为类名、文件路径或依赖变化加载失败。保存 state_dict 更稳定，也更符合跨版本和跨工程迁移习惯。

完整训练 checkpoint 通常包含：

```
checkpoint = {
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
}
```

```

    "scheduler": scheduler.state_dict(),
    "step": step,
    "epoch": epoch,
    "config": config,
}

```

torch.save(checkpoint, "checkpoint.pt")

恢复:

```

checkpoint = torch.load("checkpoint.pt", map_location="cpu")
model.load_state_dict(checkpoint["model"])
optimizer.load_state_dict(checkpoint["optimizer"])
scheduler.load_state_dict(checkpoint["scheduler"])
step = checkpoint["step"]

```

推理只需要模型权重时，通常不需要 optimizer 和 scheduler。

3.12 strict=True 和 strict=False

load_state_dict 默认 strict=True，表示 checkpoint 中的 key 必须和当前模型完全匹配。

model.load_state_dict(state, strict=True)

如果不匹配，会报 missing key 或 unexpected key:

1. missing key: 当前模型需要某些参数，但 checkpoint 没有。
2. unexpected key: checkpoint 有某些参数，但当前模型不需要。

常见原因:

1. 模型结构改了。
2. 层名称改了。
3. DDP 保存时带了 module. 前缀。
4. 加载 backbone 到带分类头的新模型。
5. 只加载 LoRA、adapter 或部分模块。

如果确实只想部分加载，可以用 strict=False:

```

missing, unexpected = model.load_state_dict(state, strict=False)
print("missing:", missing)
print("unexpected:", unexpected)

```

但不要把 strict=False 当成万能修复。正确做法是检查 missing 和 unexpected 是否符合预期。例如加载预训练 backbone 到下游分类模型时，分类头 missing 是合理的；但如果 Transformer block 大量 missing，说明 key 对不上或模型结构不一致。

处理 module. 前缀:

```

new_state = {}
for key, value in state.items():
    if key.startswith("module."):
        key = key[len("module."):]
    new_state[key] = value

```

model.load_state_dict(new_state)

更好的习惯是在 DDP 训练时保存未包装模型或保存 model.module.state_dict()。

3.13 train() 和 eval() 到底改变什么

model.train() 和 model.eval() 用于切换模块的训练/推理模式。

```
model.train()
```

等价于：

```
model.train(True)
```

推理模式：

```
model.eval()
```

等价于：

```
model.train(False)
```

它们会递归设置每个子模块的 `training` 标志：

```
print(model.training)
```

最受影响的层是：

1. Dropout。
2. BatchNorm。

Dropout 在训练时会随机置零部分激活，在 eval 时不再随机丢弃：

```
dropout = nn.Dropout(p=0.5)
x = torch.ones(10)
```

```
dropout.train()
print(dropout(x)) # 随机有些位置为 0
```

```
dropout.eval()
print(dropout(x)) # 通常保持不变
```

BatchNorm 在训练时使用当前 batch 的统计量并更新 running statistics，在 eval 时使用保存的 running statistics。

重要区别：`model.eval()` 不等于关闭梯度。

```
model.eval()
with torch.no_grad():
    output = model(x)
```

推理时通常两者都需要：

1. `model.eval()`：改变 dropout、batch norm 等模块行为。
2. `torch.no_grad()` 或 `torch.inference_mode()`：关闭 autograd，节省显存和计算开销。

面试回答：

`model.train()` 和 `model.eval()` 改的是 Module 的 `training` 状态，会递归影响 Dropout、BatchNorm 这类依赖训练模式的层。它们不

3.14 Module.to、device 和 dtype 管理

`model.to(device)` 会递归移动模型参数和 buffer。

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

输入数据也必须在同一个 device：

```
x = x.to(device)
output = model(x)
```

常见错误：

Expected all tensors to be on the same device

通常是模型在 GPU、输入在 CPU，或者某个未注册 tensor 没有被 `model.to(device)` 移动。
这也是为什么固定 mask、位置编码这类 tensor 应该注册为 buffer，而不是普通属性：

```
class BadMask(nn.Module):
    def __init__(self):
        super().__init__()
        self.mask = torch.ones(1, 1, 1024, 1024)

    def forward(self, x):
        return x.masked_fill(self.mask == 0, 0)
```

`self.mask` 不会随 `model.to("cuda")` 移动，容易 device mismatch。

正确写法：

```
class GoodMask(nn.Module):
    def __init__(self):
        super().__init__()
        mask = torch.ones(1, 1, 1024, 1024)
        self.register_buffer("mask", mask, persistent=False)

    def forward(self, x):
        return x.masked_fill(self.mask == 0, 0)
```

`model.to(dtype=torch.bfloat16)` 可以转换浮点参数和 buffer 的 dtype：

```
model = model.to(dtype=torch.bfloat16)
```

注意：整数类型 tensor 不会被转换成浮点；token id 通常仍然是 `torch.long`。

3.15 apply、children、modules 和递归遍历

`nn.Module` 提供了多种遍历方式。

直接子模块：

```
for child in model.children():
    print(type(child).__name__)
```

所有递归子模块，包括自身：

```
for module in model.modules():
    print(type(module).__name__)
```

带名称遍历：

```
for name, module in model.named_modules():
    print(name, type(module).__name__)
```

这些接口常用于：

1. 自定义初始化。
2. 替换某些层。
3. 给特定模块加 hook。
4. 收集特定模块的参数。
5. 打印模型结构。

自定义初始化示例：

```
def init_weights(module):
    if isinstance(module, nn.Linear):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
```

```
nn.init.zeros_(module.bias)
```

```
model.apply(init_weights)
```

apply 会递归访问所有子模块，并对每个模块执行函数。

替换层时要小心，不要一边遍历一边直接修改复杂层级导致遗漏。简单场景可以这样：

```
for name, module in model.named_children():
    if isinstance(module, nn.ReLU):
        setattr(model, name, nn.GELU())
```

复杂模型替换通常需要写递归函数，或者在模型构建阶段就通过配置决定层类型。

3.16 自定义初始化

初始化对训练稳定性很重要。PyTorch 标准层自带默认初始化，但大模型项目经常会按论文或架构要求自定义初始化。

一个常见模式：

```
class TinyTransformer(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)
        self.layers = nn.ModuleList([
            Block(config.hidden_size) for _ in range(config.num_layers)
        ])
        self.norm = nn.LayerNorm(config.hidden_size)
        self.lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)

        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, input_ids):
        x = self.embed_tokens(input_ids)
        for layer in self.layers:
            x = layer(x)
        x = self.norm(x)
        return self.lm_head(x)
```

初始化要注意：

1. 不要在每次 forward 时初始化。
2. 初始化通常放在模型构造结束后。
3. 加载预训练权重后不要再次随机初始化覆盖权重。
4. 特定结构可能需要特殊初始化，例如 residual 分支缩放。

如果要做权重绑定，例如语言模型里 embedding 和 LM head 共享权重：

```
self.lm_head.weight = self.embed_tokens.weight
```

这表示两个模块引用同一个 Parameter。保存、加载、优化器更新时要理解它们是共享权重，而不是两份独立参数。

3.17 大模型中的 Module 组织方式

一个简化版 decoder-only Transformer 通常可以按下面层级组织：

```
LanguageModel
embed_tokens
layers
  0 DecoderLayer
    self_attn
    q_proj
    k_proj
    v_proj
    o_proj
    mlp
    gate_proj
    up_proj
    down_proj
    input_layernorm
    post_attention_layernorm
  1 DecoderLayer
  ...
norm
lm_head
```

代码骨架：

```
class DecoderLayer(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.self_attn = SelfAttention(config)
        self.mlp = MLP(config)
        self.input_layernorm = nn.LayerNorm(config.hidden_size)
        self.post_attention_layernorm = nn.LayerNorm(config.hidden_size)

    def forward(self, x, attention_mask=None):
        residual = x
        x = self.input_layernorm(x)
        x = self.self_attn(x, attention_mask=attention_mask)
        x = residual + x

        residual = x
        x = self.post_attention_layernorm(x)
        x = self.mlp(x)
        x = residual + x
        return x

class LanguageModel(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)
        self.layers = nn.ModuleList([
            DecoderLayer(config) for _ in range(config.num_layers)
        ])
        self.norm = nn.LayerNorm(config.hidden_size)
        self.lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)
```

```
def forward(self, input_ids, attention_mask=None):
    x = self.embed_tokens(input_ids)
    for layer in self.layers:
        x = layer(x, attention_mask=attention_mask)
    x = self.norm(x)
    logits = self.lm_head(x)
    return logits
```

这种组织方式的优点：

1. 层级清楚，便于读 `state_dict`。
2. 参数命名稳定，便于加载和迁移。
3. 每个子模块职责明确，便于单元测试。
4. 可以局部替换 attention、MLP、norm 或 adapter。
5. 便于 FSDP、DDP、checkpointing 这类工程工具按模块粒度处理。

3.18 forward 返回值怎么设计

小模型可以直接返回 tensor：

```
return logits
```

但大模型常常需要返回更多信息：

1. loss。
2. logits。
3. hidden_states。
4. attentions。
5. past_key_values。

简单项目可以返回 dict：

```
return {
    "loss": loss,
    "logits": logits,
    "hidden_states": hidden_states,
}
```

也可以使用 dataclass：

```
from dataclasses import dataclass
```

```
@dataclass
class ModelOutput:
    loss: torch.Tensor | None
    logits: torch.Tensor
    hidden_states: tuple[torch.Tensor, ...] | None = None
```

设计原则：

1. 训练脚本需要什么就返回什么，不要过度复杂化。
2. 返回结构要稳定，避免不同分支返回完全不同类型。
3. 如果支持 `output_hidden_states`，默认可以关闭，避免额外显存。
4. loss 可以在 model 内部算，也可以在 trainer 外部算，但项目内要统一。

例如语言模型 forward：

```
def forward(self, input_ids, labels=None):
    logits = self.compute_logits(input_ids)
    loss = None
```

```

if labels is not None:
    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = labels[:, 1:].contiguous()
    loss = nn.functional.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
    )

return {"loss": loss, "logits": logits}

```

这里要注意：如果使用 `view`，前面常常需要 `contiguous()`；如果不想关心连续性，可以用 `reshape`。

3.19 hooks：观察和修改中间过程

`nn.Module` 支持 hook，用于观察或修改 forward、backward 过程。常见用途：

1. 抓取中间激活。
2. 调试某一层输出是否有 NaN。
3. 统计激活范围。
4. 做可视化。
5. 在研究代码中临时修改输出。

forward hook 示例：

```
activations = {}
```

```

def save_activation(name):
    def hook(module, inputs, output):
        activations[name] = output.detach()
    return hook

```

```
handle = model.layers[0].register_forward_hook(save_activation("layer0"))
```

```

with torch.no_grad():
    _ = model(input_ids)

```

```
print(activations["layer0"].shape)
```

```
handle.remove()
```

hook 使用注意：

1. 用完要 `remove()`，否则可能影响后续实验。
2. 保存激活时通常要 `detach()`，否则可能保留计算图导致显存增长。
3. hook 会增加调试复杂度，不适合作为核心业务逻辑长期依赖。
4. 分布式和编译场景下 hook 行为可能更复杂。

面试回答：

Module hook 可以在 forward 或 backward 过程中插入回调，常用于抓中间激活、排查 NaN 或统计层输出。使用时要注意 `remove handle`。

3.20 register_module、setattr 和动态添加层

有时模型结构需要根据配置动态构建。只要用 `setattr` 把 module 赋给 `nn.Module` 属性，也会注册。

```

class DynamicModel(nn.Module):
    def __init__(self, hidden_size, num_layers):
        super().__init__()

```



```

    for i in range(num_layers):
        setattr(self, f"layer_{i}", nn.Linear(hidden_size, hidden_size))

    def forward(self, x):
        for i in range(3):
            layer = getattr(self, f"layer_{i}")
            x = layer(x)
        return x

```

这能工作，但不如 ModuleList 清晰，因为 forward 中还要知道层数。更好的写法：

```

self.layers = nn.ModuleList([
    nn.Linear(hidden_size, hidden_size) for _ in range(num_layers)
])

```

动态添加层要注意：如果 optimizer 已经创建，再向模型里添加新参数，optimizer 默认不知道这些新参数。

```
model.new_head = nn.Linear(768, 2)
```

如果这发生在 optimizer 创建之后，需要重新创建 optimizer，或者显式 add_param_group。大多数情况下，应在构建 optimizer 前完成模型结构定义。

3.21 常见 bug：参数没有注册

症状：loss 在变，但某些层参数不更新；或者 state_dict 里没有预期权重。

常见原因：

1. 子模块放在普通 list 或 dict 中。
2. 可训练 tensor 没有包装成 nn.Parameter。
3. 在 forward 中临时创建层。
4. 创建 optimizer 后才添加新层。
5. 手动覆盖了已经注册的参数或模块。

排查方法：

```

for name, param in model.named_parameters():
    print(name, param.shape, param.requires_grad)

```

检查目标层是否出现。如果没有出现，说明没有注册。

再检查 optimizer：

```

num_model_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
num_optim_params = sum(
    p.numel()
    for group in optimizer.param_groups
    for p in group["params"]
)

```

```
print(num_model_params, num_optim_params)
```

如果两者不一致，要确认是否有意冻结、是否参数分组遗漏、是否创建 optimizer 的时机不对。

3.22 常见 bug：eval 后结果仍然变化

如果 model.eval() 后同一个输入输出仍然变化，可能原因包括：

1. forward 中手动调用了随机函数，例如 torch.randn、随机采样。
2. 推理逻辑里使用了采样解码，例如 multinomial sampling。
3. 模型内部有自定义 dropout 没有遵守 self.training。
4. 输入本身不同，例如 padding 或 mask 不一致。

5. 某些 CUDA 算子存在非确定性。

自定义 dropout 类逻辑应该检查 `self.training`:

```
class CustomDropout(nn.Module):
    def __init__(self, p):
        super().__init__()
        self.p = p

    def forward(self, x):
        if not self.training or self.p == 0:
            return x
        mask = torch.rand_like(x) > self.p
        return x * mask / (1 - self.p)
```

如果只是推理忘了关梯度，输出通常不会因此随机变化，但会浪费显存：

```
model.eval()
with torch.inference_mode():
    output = model(x)
```

3.23 常见 bug：加载 checkpoint 报错

典型报错包括：

```
Missing key(s) in state_dict
Unexpected key(s) in state_dict
size mismatch for ...
```

排查顺序：

1. 打印当前模型 `state_dict().keys()`。
2. 打印 checkpoint 的 keys。
3. 比较是否有 module. 前缀差异。
4. 检查模型配置是否一致，例如 hidden size、层数、词表大小。
5. 检查是否只加载部分模块。

size mismatch 通常比 missing key 更严重，因为名称对上了但 shape 不一致。例如词表大小变化：

```
size mismatch for embed_tokens.weight
```

如果只是扩展词表，需要特殊处理：

```
old_weight = state["embed_tokens.weight"]
new_weight = model.embed_tokens.weight.data
```

```
num = min(old_weight.size(0), new_weight.size(0))
new_weight[:num].copy_(old_weight[:num])
```

这类操作要明确知道语义，不能随便忽略。

3.24 LoRA、Adapter 与可训练参数筛选

在参数高效微调中，经常冻结 base model，只训练 LoRA 或 adapter 参数。

典型逻辑：

```
for param in model.parameters():
    param.requires_grad = False

for name, param in model.named_parameters():
    if "lora_" in name or "adapter" in name:
```

```
param.requires_grad = True
```

```
trainable_params = [p for p in model.parameters() if p.requires_grad]  
optimizer = torch.optim.AdamW(trainable_params, lr=1e-4)
```

检查可训练参数:

```
for name, param in model.named_parameters():  
    if param.requires_grad:  
        print(name, param.numel())
```

面试中常见追问是: LoRA 参数为什么能被 optimizer 看到? 答案仍然回到 nn.Module 注册机制: LoRA 层里的 A、B 矩阵必须是注册的 nn.Parameter, LoRA 子模块必须挂在模型模块树上, 否则 optimizer、state_dict、DDP 都不会自动处理它们。

保存 LoRA 时也常常只保存可训练参数:

```
lora_state = {  
    name: param.detach().cpu()  
    for name, param in model.named_parameters()  
    if param.requires_grad  
}
```

```
torch.save(lora_state, "lora.pt")
```

真实项目中还要保存 LoRA 配置, 例如 rank、alpha、target modules, 否则只保存权重不够恢复结构。

3.25 DDP、FSDP 与 Module 边界

分布式训练工具依赖 nn.Module 的参数树。

DDP 常见写法:

```
model = model.to(local_rank)  
model = torch.nn.parallel.DistributedDataParallel(  
    model,  
    device_ids=[local_rank],  
)
```

DDP 会遍历模型参数, 注册梯度同步 hook。没有注册到 module 树上的参数不会被同步。

保存时常用:

```
if rank == 0:  
    torch.save(model.module.state_dict(), "model.pt")
```

因为 DDP 包装后的外层对象有 .module 指向原始模型。

FSDP 更依赖模块边界, 因为它会按模块切分、扁平化、分片参数。模型组织得越清晰, 越容易指定 wrap policy, 例如按 Transformer block 包装。

工程建议:

1. 不要把整个大模型写在一个巨大 forward 函数里。
2. Transformer block、attention、MLP、norm 分成清晰子模块。
3. 参数创建在 __init__ 中完成。
4. 避免未注册参数和临时层。
5. 保存时明确是 DDP 包装前还是包装后的 state_dict。

3.26 torch.compile 对 Module 的影响

torch.compile 可以编译模型以获得性能提升:

```
model = torch.compile(model)
```

它通常包装 module 的调用过程，而不是改变你写 forward 的基本规则。为了更容易被编译和优化，模型代码应尽量：

1. 避免在 forward 中创建新的可训练层。
2. 避免过多依赖 Python 副作用。
3. 避免每次 forward 动态改变模块结构。
4. 对 shape 变化和控制流保持可预期。

不是所有模型都适合直接 compile。遇到 graph break、编译时间过长、显存变化或数值不一致时，需要逐步缩小范围，例如只 compile 某个 block 或推理路径。

对面试来说，关键不是背 compile 参数，而是知道：nn.Module 仍然是模型结构入口，forward 的 Python 写法会影响编译器是否能稳定捕获计算图。

3.27 最小可运行 Module 审计 demo

下面的 demo 把本章最容易踩坑的机制放到一个小例子里：普通 list 不注册子模块、ModuleList 会注册子模块、Parameter 会进入参数表、buffer 会进入 state_dict、persistent=False buffer 不保存、train/eval 会递归切换、冻结参数后 optimizer 需要重新按可训练参数创建、保存加载后 eval 输出应一致。

```
import tempfile
from pathlib import Path
```

```
import torch
from torch import nn
```

```
class ResidualBlock(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(hidden_size, hidden_size),
            nn.LayerNorm(hidden_size),
            nn.GELU(),
        )

    def forward(self, x):
        return x + self.net(x)

class BadStack(nn.Module):
    def __init__(self, hidden_size, num_layers):
        super().__init__()
        self.layers = [nn.Linear(hidden_size, hidden_size) for _ in range(num_layers)]

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

class GoodStack(nn.Module):
    def __init__(self, hidden_size, num_layers):
        super().__init__()
        self.layers = nn.ModuleList([
            nn.Linear(hidden_size, hidden_size) for _ in range(num_layers)
        ])
```

```
])
```

```
def forward(self, x):  
    for layer in self.layers:  
        x = layer(x)  
    return x
```

```
class TinyClassifier(nn.Module):  
    def __init__(self, input_size, hidden_size, num_classes, num_blocks):  
        super().__init__()  
        self.input_scale = nn.Parameter(torch.ones(input_size))  
        self.register_buffer("feature_mean", torch.zeros(input_size))  
        self.register_buffer("scratch_mask", torch.ones(1, input_size), persistent=False)  
  
        self.encoder = nn.Sequential(  
            nn.Linear(input_size, hidden_size),  
            nn.LayerNorm(hidden_size),  
            nn.GELU(),  
        )  
        self.blocks = nn.ModuleList([  
            ResidualBlock(hidden_size) for _ in range(num_blocks)  
        ])  
        self.dropout = nn.Dropout(p=0.2)  
        self.head = nn.Linear(hidden_size, num_classes)  
  
    def forward(self, x, labels=None):  
        x = (x - self.feature_mean) * self.input_scale  
        hidden = self.encoder(x)  
        for block in self.blocks:  
            hidden = block(hidden)  
        hidden = self.dropout(hidden)  
        logits = self.head(hidden)  
        loss = None  
        if labels is not None:  
            loss = nn.functional.cross_entropy(logits, labels)  
        return {"loss": loss, "logits": logits, "hidden": hidden}
```

```
torch.manual_seed(7)
```

```
input_size = 6  
hidden_size = 12  
num_classes = 2
```

```
bad_stack = BadStack(hidden_size, num_layers=2)  
good_stack = GoodStack(hidden_size, num_layers=2)  
model = TinyClassifier(input_size, hidden_size, num_classes, num_blocks=2)
```

```
bad_stack_param_names = [name for name, _ in bad_stack.named_parameters()]  
good_stack_param_names = [name for name, _ in good_stack.named_parameters()]  
param_names = [name for name, _ in model.named_parameters()]  
buffer_names = [name for name, _ in model.named_buffers()]  
state_keys = list(model.state_dict().keys())  
module_names = [name for name, _ in model.named_modules() if name]
```

```

model.train()
recursive_train = all(module.training for module in model.modules())
model.eval()
recursive_eval = not any(module.training for module in model.modules())

x = torch.randn(64, input_size)
labels = (x[:, 0] + 0.5 * x[:, 1] > 0).long()

model.train()
model.dropout.p = 0.0
optimizer = torch.optim.AdamW(model.parameters(), lr=0.03)
losses = []

for _ in range(40):
    optimizer.zero_grad(set_to_none=True)
    outputs = model(x, labels=labels)
    loss = outputs["loss"]
    loss.backward()
    optimizer.step()
    losses.append(round(float(loss.detach()), 4))

for name, param in model.named_parameters():
    if name.startswith("encoder") or name.startswith("blocks"):
        param.requires_grad = False

trainable_params = [param for param in model.parameters() if param.requires_grad]
frozen_optimizer = torch.optim.AdamW(trainable_params, lr=0.01)
num_model_trainable = sum(param.numel() for param in trainable_params)
num_optimizer_params = sum(
    param.numel()
    for group in frozen_optimizer.param_groups
    for param in group["params"]
)

with tempfile.TemporaryDirectory() as tmpdir:
    path = Path(tmpdir) / "tiny_classifier.pt"
    torch.save(model.state_dict(), path)

    clone = TinyClassifier(input_size, hidden_size, num_classes, num_blocks=2)
    clone.load_state_dict(torch.load(path, map_location="cpu"))

    model.eval()
    clone.eval()
    with torch.inference_mode():
        same_after_load = torch.allclose(
            model(x)["logits"],
            clone(x)["logits"],
            atol=1e-6,
        )

partial_state = dict(model.state_dict())
partial_state.pop("head.weight")
partial_state.pop("head.bias")
fresh = TinyClassifier(input_size, hidden_size, num_classes, num_blocks=2)

```

```
missing, unexpected = fresh.load_state_dict(partial_state, strict=False)
```

```
report = {
    "bad_stack_param_names": bad_stack_param_names,
    "good_stack_param_count": len(good_stack_param_names),
    "module_count": len(module_names),
    "has_input_scale": "input_scale" in param_names,
    "buffer_names": buffer_names,
    "state_has_feature_mean": "feature_mean" in state_keys,
    "state_has_scratch_mask": "scratch_mask" in state_keys,
    "loss_first_last": (losses[0], losses[-1]),
    "trainable_after_freeze": num_model_trainable,
    "optimizer_param_count": num_optimizer_params,
    "missing_keys": sorted(missing),
    "unexpected_keys": unexpected,
}

checks = {
    "bad_stack_unregistered": len(bad_stack_param_names) == 0,
    "modulelist_registered": len(good_stack_param_names) == 4,
    "parameter_registered": "input_scale" in param_names,
    "buffer_registered": "feature_mean" in buffer_names,
    "buffer_saved": "feature_mean" in state_keys,
    "nonpersistent_buffer_skipped": "scratch_mask" not in state_keys,
    "train_eval_recursive": recursive_train and recursive_eval,
    "loss_decreased": losses[-1] < losses[0],
    "optimizer_matches_trainable": num_model_trainable == num_optimizer_params,
    "strict_false_reports_missing": (
        sorted(missing) == ["head.bias", "head.weight"] and unexpected == []
    ),
    "load_roundtrip_same_logits": same_after_load,
}

print("module audit report:")
for key, value in report.items():
    print(f"{key}: {value}")

print("checks:")
for key, value in checks.items():
    print(f"{key}: {value}")
```

```
assert all(checks.values())
```

如果这个 demo 在有 PyTorch 的环境里运行，应该看到：

1. BadStack 的参数名为空，说明普通 list 没有注册子模块。
2. GoodStack 有 4 个参数 tensor，来自两个 Linear 的 weight 和 bias。
3. input_scale 出现在 named_parameters() 中。
4. feature_mean 出现在 buffer 和 state_dict 中。
5. scratch_mask 是非持久化 buffer，不出现在 state_dict 中。
6. train() / eval() 递归切换所有子模块。
7. 冻结后 optimizer 参数量和可训练参数量一致。
8. 保存加载后 eval 模式下 logits 一致。
9. strict=False 只报告有意删除的 head.weight 和 head.bias。

这个例子的重点不是分类任务本身，而是建立 Module 工程审计习惯：每次遇到“参数没更新、checkpoint 加载不对、eval 行为异常、LoRA 参数没保存”这类问题，先从注册树、state_dict、requires_grad 和 optimizer

参数组查起。

3.28 面试高频问题

问题 1: `nn.Module` 和 `nn.Parameter` 的关系是什么？

参考回答：

`nn.Module` 是模型容器，负责注册参数、子模块和 `buffer`。`nn.Parameter` 是特殊 `Tensor`，被赋值为 `Module` 属性时会自动注册为参数。

问题 2: 为什么子模块不能放普通 `list`？

参考回答：

普通 Python `list` 里的 `Module` 不会被 `nn.Module` 自动注册，因此 `parameters()`、`state_dict()`、`to(device)`、`train/eval`、`DDP` 都

问题 3: `model.eval()` 会关闭梯度吗？

参考回答：

不会。`model.eval()` 只切换模块的 `training` 标志，影响 `Dropout`、`BatchNorm` 等层的行为。关闭梯度需要使用 `torch.no_grad()` 或

问题 4: `state_dict` 里保存哪些内容？

参考回答：

`state_dict` 保存模型注册的 `Parameter` 和持久化 `buffer`，不保存 Python 类定义、`forward` 代码和 `optimizer` 状态。`optimizer` 状态

问题 5: 加载 `checkpoint` 时 `missing key` 和 `unexpected key` 怎么处理？

参考回答：

`missing key` 表示当前模型需要的参数 `checkpoint` 没有，`unexpected key` 表示 `checkpoint` 有但当前模型不用。要先比较模型和 `che`

问题 6: `Parameter` 和 `buffer` 的区别是什么？

参考回答：

`Parameter` 是可训练权重，通常会被 `optimizer` 更新；`buffer` 是不训练但属于模型状态的 `Tensor`，比如 `BatchNorm running statist`

问题 7: 为什么创建 `optimizer` 后再添加新层可能有问题？

参考回答：

`optimizer` 在创建时拿到的是当时的参数列表。之后如果给 `model` 新增一层，新层参数虽然可能注册到 `Module` 上，但 `optimizer` 的

3.29 本章小练习

练习 1: 写一个 `ResidualMLP`，包含两层 `Linear`、`GELU`、`Dropout` 和 `residual connection`，要求所有层都在 `__init__` 中定义。

练习 2: 写一个 `Stack`，用 `ModuleList` 堆叠 4 个 `ResidualMLP`，并打印所有参数名，确认每层参数都被注册。

练习 3: 给模型注册一个固定 `buffer scale`，在 `forward` 中使用它，然后调用 `model.to(device)`，确认 `buffer` 跟着移动。

练习 4: 保存模型 `state_dict`，重新构造同结构模型并加载，检查同一输入下输出是否一致。注意推理时使用 `eval()` 和 `inference_mode()`。

练习 5: 故意把子模块放进普通 `list`，观察 `named_parameters()` 和 `state_dict()` 的变化，再改成 `ModuleList`。

3.30 本章总结

`nn.Module` 是 PyTorch 工程化的中心。它不是简单的“模型基类”，而是参数、子模块、`buffer`、状态切换、保存加载和分布式工具之间的连接点。

本章需要记住几条主线：

1. 可训练权重必须是注册的 `Parameter`，或者属于已注册子模块。
2. 子模块要用属性赋值、`ModuleList`、`ModuleDict` 等方式注册，不能藏在普通 `list` 或 `dict` 中。
3. 不训练但属于模型状态的 `tensor` 应该注册为 `buffer`。
4. `state_dict` 保存参数和持久化 `buffer`，是推荐的模型保存加载方式。
5. `train()` 和 `eval()` 改变模块行为，但不负责关闭梯度。
6. `model.to(device)` 只会递归处理注册过的参数和 `buffer`。
7. `optimizer` 只会更新它拿到的参数，参数注册和 `optimizer` 创建时机都很重要。
8. 大模型代码应按 `embedding`、`block`、`attention`、`MLP`、`norm`、`head` 等模块清晰组织。

如果你能解释清楚 `nn.Module` 的注册机制、`Parameter` 和 `buffer` 的区别、`state_dict` 的内容、`train/eval` 的作用，以及常见加载报错的排查路径，就已经具备了阅读和维护大多数 PyTorch 模型代码的基础。

第四章：Dataset 与 DataLoader

`Dataset` 和 `DataLoader` 是 PyTorch 里把“原始数据”变成“可训练 batch”的核心组件。前面三章解决的是 `tensor`、`autograd` 和模型组织，到了真实训练里，最容易出问题的往往不是模型本身，而是数据管线：样本格式不统一、长度不一致、padding 方向错了、shuffle 没生效、分布式下数据重复、`collate_fn` 写错、`num_workers` 太大导致卡死、`pin_memory` 和 `device` 搬运时机不合理。

本章目标不是背 API，而是把一个可维护的数据管线讲清楚：`Dataset` 负责描述“单个样本怎么取”，`DataLoader` 负责描述“样本怎么组成 batch 并高效送到训练循环”，`collate_fn` 负责处理变长样本和复杂样本结构，`sampler` 负责控制采样顺序，padding 和 packing 负责把不同长度样本整理成模型可吃的张量。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前，已核对 PyTorch 官方 `torch.utils.data`、`Dataset`、`IterableDataset`、`DataLoader`、`default_collate`、`sampler`、`DistributedSampler`、`worker seed`、`worker_init_fn`、`pin_memory`、`persistent_workers` 和 `prefetch_factor` 文档口径。

本章聚焦大模型训练最常见的数据管线问题：`map-style dataset`、`iterable-style dataset`、`__len__` / `__getitem__`、`collate_fn`、padding、labels ignore index、packing、shuffle、`sampler`、`batch sampler`、`length bucket`、`DataLoader worker`、`pinned memory`、分布式数据切分，以及最小可运行的数据管线审计 demo。

本章不展开生产级数据湖、远程对象存储流式读取、`WebDataset` / `Datasets` / `DataPipes`、复杂多模态数据解码、GPU 数据预取、分布式 checkpoint 数据状态恢复或大规模训练数据治理。这些内容分别放在数据工程、训练系统、AI Infra 和多模态章节中展开。

4.1 为什么数据管线很重要

很多训练问题表面看像模型问题，根因其实在数据管线。

例如：

1. loss 不下降，最后发现 labels 和 `input_ids` 错位了。
2. 模型输出乱码，最后发现 padding token 被算进 loss。
3. 训练很慢，最后发现每个 batch 都在 Python 里做重活。
4. 分布式训练各卡数据一样，最后发现 `sampler` 没设对。
5. 显存突然爆了，最后发现 `collate_fn` 把超长样本直接堆进 batch。

数据管线的职责是把“外部数据格式”稳定变成“模型训练格式”。一个好的管线要做到：

1. 样本读取逻辑清楚。
2. batch 组装逻辑统一。
3. 变长输入可处理。
4. 顺序控制可复现。
5. 分布式场景不重复、不漏样。
6. 尽量减少 CPU 成为瓶颈。

面试回答：

Dataset 和 DataLoader 负责把原始数据组织成训练可用的 batch。Dataset 定义单个样本如何读取，DataLoader 负责并行取样、打乱

4.1.1 关键 shape 公式与数据管线调试速查

第一，map-style Dataset 可以理解成从索引到样本的映射：

$$\mathcal{D}:\{0,\ldots,N-1\}\rightarrow\mathcal{S},\quad s_i=\mathcal{D}(i)$$

其中 N 是样本数， s_i 是第 i 个样本。__len__() 给出 N ，__getitem__(i) 给出 s_i 。

第二，collate_fn 是从样本列表到 batch 的函数：

$$C(\{s_1,\ldots,s_B\})=(X,Y,M)$$

在 causal LM 训练中，常见 batch shape 是：

$$\begin{aligned} X &\in \mathbb{Z}^{B \times T_b}, \\ Y &\in \mathbb{Z}^{B \times T_b}, \\ M &\in \{0,1\}^{B \times T_b} \end{aligned}$$

其中 B 是 batch size， T_b 是当前 batch 内 padding 后长度， X 是 input_ids， Y 是 labels， M 是 attention_mask。

第三，变长序列 padding 后的长度通常是当前 batch 内最长样本：

$$T_b = \max_{1 \leq i \leq B} L_i$$

其中 L_i 是第 i 个样本的真实 token 长度。padding token 利用率可以粗略写成：

$$\begin{aligned} R_{\mathrm{valid}} &= \frac{\sum_{i=1}^B L_i}{B T_b}, \\ R_{\mathrm{pad}} &= 1 - R_{\mathrm{valid}} \end{aligned}$$

R_{valid} 越低，说明 batch 里浪费在 padding 上的计算越多。length bucket 的目标就是让同一个 batch 内的 L_i 更接近，从而降低 R_{pad} 。

第四，causal LM 的 loss 通常使用右移后的 logits 和 labels：

$$\begin{aligned} \mathrm{Logits}_{\mathrm{shift}} &\in \mathbb{R}^{B \times (T_b-1) \times V}, \\ Y_{\mathrm{shift}} &\in \mathbb{Z}^{B \times (T_b-1)} \end{aligned}$$

其中 V 是词表大小。padding 位置的 label 通常设为 -100，让 CrossEntropyLoss(ignore_index=-100) 忽略它们。

第五，分布式采样器要把样本索引分给不同 rank。理想情况下：

$$S_r \cap S_q = \varnothing, \quad r \neq q$$

其中 S_r 是第 r 个 rank 在一个 epoch 中看到的索引集合。实际 PyTorch DistributedSampler 为了让每个 rank 样本数一致，可能在数据量不能整除时补样本；所以要理解 drop_last、样本数和重复样本之间的取舍。

4.2 Dataset 的两种基本范式

PyTorch 里最常见的是两类 Dataset：map-style dataset 和 iterable-style dataset。

4.2.1 map-style Dataset

map-style Dataset 需要实现两个核心方法：

1. __len__(): 返回样本数。
2. __getitem__(idx): 按索引返回第 idx 个样本。

一个最小例子：

```
from torch.utils.data import Dataset
```

```
class NumberDataset(Dataset):
    def __init__(self, values):
```

```

        self.values = values

    def __len__(self):
        return len(self.values)

    def __getitem__(self, idx):
        x = self.values[idx]
        return {"x": x, "y": x * 2}

```

使用时:

```

dataset = NumberDataset([1, 2, 3])
print(len(dataset))
print(dataset[0])

```

map-style Dataset 的特点是:

1. 能随机访问。
2. 适合监督学习、固定语料、离线数据集。
3. 容易配合 shuffle、sampler 和 batch sampler。

4.2.2 iterable-style Dataset

iterable-style Dataset 适合流式数据或无法预先知道总长度的数据。它更像一个迭代器，而不是索引表。

示意:

```

from torch.utils.data import IterableDataset

```

```

class StreamDataset(IterableDataset):
    def __iter__(self):
        for i in range(5):
            yield {"x": i, "y": i * 2}

```

适用场景:

1. 超大规模语料流式读取。
2. 在线日志数据流。
3. 远程对象存储逐条读取。
4. 无法随机访问的数据源。

区别要点:

1. map-style 更适合有索引的数据。
2. iterable-style 更适合流式和超大规模数据。
3. iterable-style 通常不能直接依赖 shuffle=True，而要自己设计打乱逻辑。

4.3 一个最小可训练文本 Dataset

大模型训练里最常见的数据结构不是单个数值，而是 token 序列。一个简单的文本 Dataset 通常会把文本样本转成 input_ids、labels 或 attention_mask。

示例:

```

from torch.utils.data import Dataset

```

```

class TextDataset(Dataset):
    def __init__(self, texts, tokenizer, max_length=16):
        self.texts = texts

```

```

        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        ids = self.tokenizer(text)[: self.max_length]
        return {
            "input_ids": ids,
            "length": len(ids),
        }

```

这里先不急着 padding，因为不同样本长度往往不同，更合理的做法是在 `collate_fn` 里统一处理。

如果是监督训练，还可能直接返回：

```

{
    "input_ids": [...],
    "labels": [...],
    "attention_mask": [...],
}

```

设计 Dataset 时常见原则：

1. `__getitem__` 只负责单样本读取和轻量预处理。
2. 不要把大批量 padding 放在 `__getitem__` 里。
3. 不要把昂贵的 batch 级逻辑塞进 Dataset。
4. 复杂的 batch 拼接放到 `collate_fn`。

4.4 DataLoader 解决什么问题

DataLoader 负责把 Dataset 取出的样本变成训练循环可直接消费的 batch。

典型用法：

```
from torch.utils.data import DataLoader
```

```

loader = DataLoader(
    dataset,
    batch_size=8,
    shuffle=True,
    num_workers=4,
)

```

```

for batch in loader:
    ...

```

DataLoader 常见职责：

1. 批量取样。
2. 可选 shuffle。
3. 多进程并行读取数据。
4. 调用 `collate_fn` 组 batch。
5. 可选 pinned memory 加速 GPU 搬运。
6. 配合 sampler 控制采样顺序。

可以把它理解成 “Dataset 之上的 batch 组装层”。

4.5 batch 是怎么拼起来的

如果样本结构简单，DataLoader 默认会尝试把样本堆成张量。

例如 Dataset 返回：

```
{ "x": tensor([1, 2]), "y": tensor(3) }
```

DataLoader 可能自动把一个 batch 变成：

```
{  
    "x": tensor([[1, 2], [4, 5]]),  
    "y": tensor([3, 6]),  
}
```

但默认 collate 只适合 “结构一致、shape 一致” 的样本。如果样本长度不一样，就会报错。

例如变长 token 序列：

```
[1, 2, 3]  
[4, 5]  
[6, 7, 8, 9]
```

不能直接 stack。这时就需要自定义 collate_fn。

4.6 collate_fn: batch 组装的关键

collate_fn 接收一个样本列表，输出一个 batch。

典型签名：

```
def collate_fn(samples):  
    ...  
    return batch
```

4.6.1 变长序列 padding

最常见的做法是把一个 batch 里所有序列 pad 到同样长度。

示例：

```
import torch
```

```
def pad_sequences(seqs, pad_value=0):  
    max_len = max(len(seq) for seq in seqs)  
    batch = []  
    mask = []  
    for seq in seqs:  
        pad_len = max_len - len(seq)  
        batch.append(seq + [pad_value] * pad_len)  
        mask.append([1] * len(seq) + [0] * pad_len)  
    return (  
        torch.tensor(batch, dtype=torch.long),  
        torch.tensor(mask, dtype=torch.bool),  
    )
```

对应 collate_fn：

```
def collate_fn(samples):  
    input_ids, attention_mask = pad_sequences(  
        [s["input_ids"] for s in samples],  
        pad_value=0,
```

```
)
return {
    "input_ids": input_ids,
    "attention_mask": attention_mask,
}
```

这里注意两点：

1. pad_value 要和 tokenizer 的 pad token 一致。
2. attention_mask 要正确标记有效 token 和 padding token。

4.6.2 labels 的 padding

监督学习时 labels 也可能需要 padding。但很多训练任务里，padding 部分不应该参与 loss。

常见做法：

```
def pad_labels(seqs, pad_value=-100):
    max_len = max(len(seq) for seq in seqs)
    out = []
    for seq in seqs:
        pad_len = max_len - len(seq)
        out.append(seq + [pad_value] * pad_len)
    return torch.tensor(out, dtype=torch.long)
```

-100 是 PyTorch 交叉熵里常见的 ignore index。这样 padding 位置不会计入 loss。

4.6.3 一个更完整的 collate_fn

```
def collate_fn(samples):
    max_len = max(len(s["input_ids"]) for s in samples)

    input_ids = []
    labels = []
    attention_mask = []

    for s in samples:
        ids = s["input_ids"]
        lab = s.get("labels", ids)
        pad_len = max_len - len(ids)

        input_ids.append(ids + [0] * pad_len)
        labels.append(lab + [-100] * pad_len)
        attention_mask.append([1] * len(ids) + [0] * pad_len)

    return {
        "input_ids": torch.tensor(input_ids, dtype=torch.long),
        "labels": torch.tensor(labels, dtype=torch.long),
        "attention_mask": torch.tensor(attention_mask, dtype=torch.bool),
    }
```

这就是很多语言模型训练脚本里最核心的数据拼 batch 逻辑。

4.7 padding 的方向和坑

padding 不只是“补零”，还要注意方向。

4.7.1 right padding

右侧 padding 最常见：

[1, 2, 3, 0, 0]

优点：

1. 最直观。
2. 和大多数训练代码兼容。
3. attention mask 容易构造。

4.7.2 left padding

左侧 padding 常见于某些生成推理场景：

[0, 0, 1, 2, 3]

它有时更方便把不同长度 prompt 对齐到最右侧。但在训练里，如果模板、mask 或位置编码没有处理好，容易引入错位。

4.7.3 常见坑

1. padding token 被算进 loss。
2. attention mask 方向反了。
3. labels 和 input_ids 没有对齐。
4. 左 padding 与位置编码或模板不一致。
5. 只 pad 了 input 没 pad labels。

一个简单检查方法是：打印一个 batch 的 input_ids、labels、attention_mask，确认 padding 的位置完全符合预期。

4.8 packing：把多个短样本拼进一个长序列

除了 padding，还有一种提高 token 利用率的方法叫 packing。

4.8.1 为什么需要 packing

如果 batch 中很多样本都很短，单纯 padding 会浪费大量计算。

例如：

样本 A: 20 tokens

样本 B: 18 tokens

样本 C: 22 tokens

如果都 pad 到 22，浪费不大；但如果一个 batch 中长度分布很散，浪费会很明显。

packing 的思路是把多个短样本拼接到同一个固定长度序列里，减少 padding 浪费。

4.8.2 简化 packing 示例

```
def pack_sequences(seqs, max_length, eos_id=2):
    packed = []
    cur = []

    for seq in seqs:
        if len(cur) + len(seq) + 1 > max_length:
            packed.append(cur)
            cur = []
        cur.extend(seq + [eos_id])
```

```
if cur:
    packed.append(cur)
```

```
return packed
```

这里每个样本之间加 eos_id, 表示样本边界。

4.8.3 packing 的风险

1. 样本边界要处理清楚。
2. labels 和 mask 要正确切分。
3. 不适合所有任务。
4. 对对话数据或有严格轮次结构的数据, packing 要更谨慎。

简单说: padding 更稳, packing 更省, 但实现复杂度更高。

4.9 shuffle、sampler 和 batch_sampler

DataLoader 里控制 “怎么取样” 的几个概念容易混淆。

4.9.1 shuffle

shuffle=True 表示每个 epoch 随机打乱样本顺序。

```
loader = DataLoader(dataset, batch_size=8, shuffle=True)
```

它适合单机单卡的简单场景。

4.9.2 sampler

sampler 决定 “按什么顺序取样本索引”。

例如:

```
from torch.utils.data import SequentialSampler, RandomSampler
```

在复杂场景里你可能需要:

1. 按长度排序后再分桶。
2. 按类别均衡采样。
3. 按权重采样。
4. 分布式训练下只取某个 rank 的样本子集。

4.9.3 batch_sampler

batch_sampler 直接产出一批索引, 而不是单个索引。

适合:

1. 按长度 bucket 组 batch。
2. 自定义 batch 大小策略。
3. 复杂采样逻辑。

一般优先级是:

1. 简单场景用 shuffle=True。
2. 需要控制采样顺序用 sampler。
3. 需要控制 “每个 batch 里有哪些样本” 用 batch_sampler。

4.10 按长度分桶：减少 padding 浪费

语言模型训练里，长度差异很大时常用 bucket sampler。

直觉是：把长度相近的样本放进同一个 batch，这样 pad 更少。

简化思路：

```
def bucket_by_length(samples, bucket_size=4):
    samples = sorted(samples, key=lambda x: len(x["input_ids"]))
    buckets = []
    for i in range(0, len(samples), bucket_size):
        buckets.append(samples[i : i + bucket_size])
    return buckets
```

好处：

1. 减少 padding。
2. 提高 token 利用率。
3. 常常能提升训练吞吐。

代价：

1. batch 不再完全随机。
2. 可能引入轻微分布偏差。
3. 实现更复杂。

4.11 DataLoader 的性能参数

几个最常见的参数：

```
DataLoader(
    dataset,
    batch_size=8,
    shuffle=True,
    num_workers=4,
    pin_memory=True,
    persistent_workers=True,
)
```

4.11.1 num_workers

num_workers 决定用多少个子进程并行读数据。

直觉：

1. 数据读取和预处理放到多个 worker。
2. 主进程专注训练。
3. 适合 CPU 预处理较重的数据集。

但不是越大越好：

1. worker 太多会争抢 CPU。
2. 某些数据读取库不适合多进程。
3. Windows、notebook、共享环境里可能更脆弱。

4.11.2 pin_memory

pin_memory=True 可以让 CPU 内存页锁定，通常有助于加快数据拷贝到 GPU。

训练时常见写法：

```
for batch in loader:
    batch = {k: v.to(device, non_blocking=True) for k, v in batch.items()}
```

这和 pin_memory 配合时更有效。

4.11.3 persistent_workers

persistent_workers=True 可以让 worker 在多个 epoch 间保持存活，减少反复启动开销。

适合：

1. 多 epoch 训练。
2. worker 启动成本高。

4.11.4 prefetch_factor

prefetch_factor 控制每个 worker 预先加载多少个 batch。

如果数据集读取慢，可以通过预取隐藏一部分 IO 延迟；但太大也会增加内存占用。

4.12 分布式训练中的数据切分

多卡训练时，不能让每张卡都看到完全一样的数据顺序，否则等于重复训练。

这时常用 DistributedSampler。

示例：

```
from torch.utils.data import DataLoader, DistributedSampler
```

```
sampler = DistributedSampler(dataset, shuffle=True)
loader = DataLoader(dataset, batch_size=8, sampler=sampler)
```

要点：

1. 每个 rank 拿到不同子集。
2. 每个 epoch 需要调用 sampler.set_epoch(epoch)，否则 shuffle 可能不变。
3. sampler 和 shuffle=True 通常不能同时乱配。

示例：

```
for epoch in range(num_epochs):
    sampler.set_epoch(epoch)
    for batch in loader:
        ...
```

如果忘记 set_epoch，每个 epoch 的样本顺序可能完全一样，削弱随机性。

4.13 一个完整的数据管线例子

下面把 Dataset、collate_fn 和 DataLoader 串起来。

```
import torch
from torch.utils.data import Dataset, DataLoader
```

```
class ToyTextDataset(Dataset):
    def __init__(self, texts, tokenizer):
        self.texts = texts
        self.tokenizer = tokenizer

    def __len__(self):
```

```

        return len(self.texts)

    def __getitem__(self, idx):
        ids = self.tokenizer(self.texts[idx])
        return {"input_ids": ids, "labels": ids.copy()}

def collate_fn(samples):
    max_len = max(len(s["input_ids"]) for s in samples)
    input_ids, labels, attention_mask = [], [], []

    for s in samples:
        ids = s["input_ids"]
        lab = s["labels"]
        pad_len = max_len - len(ids)

        input_ids.append(ids + [0] * pad_len)
        labels.append(lab + [-100] * pad_len)
        attention_mask.append([1] * len(ids) + [0] * pad_len)

    return {
        "input_ids": torch.tensor(input_ids, dtype=torch.long),
        "labels": torch.tensor(labels, dtype=torch.long),
        "attention_mask": torch.tensor(attention_mask, dtype=torch.bool),
    }

```

```

texts = ["hello world", "tiny dataset", "pytorch"]
tokenizer = lambda s: [ord(c) % 100 for c in s][:8]

```

```

dataset = ToyTextDataset(texts, tokenizer)
loader = DataLoader(
    dataset,
    batch_size=2,
    shuffle=True,
    collate_fn=collate_fn,
)

```

```

for batch in loader:
    print(batch["input_ids"].shape)
    print(batch["attention_mask"])

```

这个例子虽然简单，但结构已经接近真实训练脚本：

1. Dataset 负责单样本读取。
2. collate_fn 负责 batch 组装和 padding。
3. DataLoader 负责批量加载和打乱。

4.14 常见 bug 与排查顺序

4.14.1 collate_fn 报错

常见原因：

1. 样本字段不一致。
2. 某个样本长度为 0。
3. 直接尝试 stack 变长序列。

4. 返回了 Python 对象而不是 tensor。

排查方法：先打印前几个样本的原始结构，再打印 batch 结构。

4.14.2 数据加载特别慢

常见原因：

1. `__getitem__` 里做了太重的预处理。
2. `num_workers` 太少。
3. 磁盘 IO 慢。
4. 远程读取或压缩解压成本太高。
5. `collate_fn` 里有大量 Python 循环。

4.14.3 每个 epoch 顺序都一样

常见原因：

1. 没开 shuffle。
2. 分布式 sampler 没设 epoch。
3. 自己写的 sampler 没有随机化。

4.14.4 loss 异常偏小

常见原因：

1. labels 被错误地 padding 成有效 token。
2. padding token 参与了 loss。
3. shift 操作对齐错了。
4. mask 把大部分 token 忽略了。

4.14.5 多卡训练样本重复

常见原因：

1. 没有使用 `DistributedSampler`。
2. sampler 配置不对。
3. 每个 rank 都读了完整数据集。

4.15 最小可运行 DataLoader 审计 demo

下面这个 demo 把本章核心点串起来：Dataset 只返回单样本，`collate_fn` 负责 padding、`labels=-100` 和 `attention_mask`，DataLoader 用固定 generator 做可复现 shuffle，最后用 `DistributedSampler` 模拟两个 rank 的数据切分。

```
import torch
from torch.utils.data import Dataset, DataLoader, DistributedSampler
```

```
PAD_ID = 0
IGNORE_INDEX = -100
```

```
class ToyCausalDataset(Dataset):
    def __init__(self, sequences):
        self.sequences = [list(seq) for seq in sequences]

    def __len__(self):
        return len(self.sequences)
```

```

def __getitem__(self, idx):
    ids = self.sequences[idx]
    if len(ids) < 2:
        raise ValueError("causal LM sample must contain at least 2 tokens")
    return {"input_ids": ids, "labels": ids.copy(), "length": len(ids)}

def collate_causal_lm(samples):
    max_len = max(s["length"] for s in samples)
    input_ids, labels, attention_mask = [], [], []

    for s in samples:
        ids = s["input_ids"]
        lab = s["labels"]
        pad_len = max_len - len(ids)
        input_ids.append(ids + [PAD_ID] * pad_len)
        labels.append(lab + [IGNORE_INDEX] * pad_len)
        attention_mask.append([1] * len(ids) + [0] * pad_len)

    return {
        "input_ids": torch.tensor(input_ids, dtype=torch.long),
        "labels": torch.tensor(labels, dtype=torch.long),
        "attention_mask": torch.tensor(attention_mask, dtype=torch.bool),
        "lengths": torch.tensor([s["length"] for s in samples], dtype=torch.long),
    }

def padding_waste(lengths, batch_size):
    total_slots = 0
    real_tokens = 0
    for i in range(0, len(lengths), batch_size):
        batch = lengths[i : i + batch_size]
        total_slots += len(batch) * max(batch)
        real_tokens += sum(batch)
    return round((total_slots - real_tokens) / total_slots, 3)

sequences = [
    [11, 12, 13, 14],
    [21, 22],
    [31, 32, 33, 34, 35, 36],
    [41, 42, 43],
    [51, 52, 53, 54, 55],
    [61, 62, 63],
]

dataset = ToyCausalDataset(sequences)
generator = torch.Generator().manual_seed(7)
loader = DataLoader(
    dataset,
    batch_size=3,
    shuffle=True,
    generator=generator,
    collate_fn=collate_causal_lm,

```

```

    num_workers=0,
)

batch = next(iter(loader))
valid_tokens = int(batch["attention_mask"].sum().item())
all_slots = batch["attention_mask"].numel()
ignore_pad_ok = bool((batch["labels"][~batch["attention_mask"]] == IGNORE_INDEX).all())
shift_logits_shape = (batch["input_ids"].size(0), batch["input_ids"].size(1) - 1, 128)
shift_labels_shape = tuple(batch["labels"][:, 1:].shape)

lengths = [len(x) for x in sequences]
plain_waste = padding_waste(lengths, batch_size=3)
bucketed_waste = padding_waste(sorted(lengths), batch_size=3)

sampler_rank0 = DistributedSampler(dataset, num_replicas=2, rank=0, shuffle=True, seed=13)
sampler_rank1 = DistributedSampler(dataset, num_replicas=2, rank=1, shuffle=True, seed=13)
sampler_rank0.set_epoch(0)
sampler_rank1.set_epoch(0)
rank0_indices = list(iter(sampler_rank0))
rank1_indices = list(iter(sampler_rank1))

print("batch_input_shape=", tuple(batch["input_ids"].shape))
print("batch_lengths=", batch["lengths"].tolist())
print("attention_mask=", batch["attention_mask"].int().tolist())
print("valid_token_ratio=", round(valid_tokens / all_slots, 3))
print("ignore_pad_ok=", ignore_pad_ok)
print("shift_logits_shape=", shift_logits_shape)
print("shift_labels_shape=", shift_labels_shape)
print("padding_waste_plain=", plain_waste)
print("padding_waste_bucketed=", bucketed_waste)
print("rank0_indices=", rank0_indices)
print("rank1_indices=", rank1_indices)
print("distributed_overlap=", sorted(set(rank0_indices) & set(rank1_indices)))

```

期望输出类似：

```

batch_input_shape= (3, 6)
batch_lengths= [2, 6, 3]
attention_mask= [[1, 1, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1], [1, 1, 1, 0, 0, 0]]
valid_token_ratio= 0.611
ignore_pad_ok= True
shift_logits_shape= (3, 5, 128)
shift_labels_shape= (3, 5)
padding_waste_plain= 0.303
padding_waste_bucketed= 0.148
rank0_indices= [4, 0, 2]
rank1_indices= [3, 5, 1]
distributed_overlap= []

```

这段输出要检查几件事：

1. input_ids 是 [B,T_b]，本例为 [3,6]。
2. attention_mask=0 的位置，labels 必须是 -100。
3. next-token loss 使用 T_b-1 个位置，因此 shift 后 shape 是 [B,T_b-1,V] 和 [B,T_b-1]。
4. length bucket 把 padding waste 从 0.303 降到 0.148，说明长度相近的样本放在一起能减少浪费。
5. 两个 rank 的索引无交集，说明这个 toy 场景下没有重复样本。

4.16 面试官会怎么问

问题一：Dataset 和 DataLoader 有什么区别？

回答模板：

Dataset 负责定义单个样本如何读取和返回，DataLoader 负责把样本按 batch 组织起来，并支持 shuffle、并行加载、collate_fn 和

问题二：为什么变长序列不能直接用默认 DataLoader？

回答模板：

默认 DataLoader 会尝试把样本 stack 成规则张量，但变长序列长度不一致，无法直接 stack。需要自定义 collate_fn，在 batch 级

问题三：collate_fn 的作用是什么？

回答模板：

collate_fn 接收一个样本列表，负责把它们组装成 batch。对于大模型训练，它通常要做 padding、构造 attention_mask、处理 lab

问题四：num_workers、pin_memory、persistent_workers 分别有什么作用？

回答模板：

num_workers 控制并行加载数据的 worker 数量；pin_memory 可以提升 CPU 到 GPU 的拷贝效率；persistent_workers 可以让 worker

问题五：为什么分布式训练要用 DistributedSampler？

回答模板：

因为多卡训练时每张卡应该看到不同的数据子集，否则会重复训练。DistributedSampler 会按 rank 切分数据，保证各卡数据不重复，

问题六：为什么 collate_fn 通常比在 Dataset 里 padding 更合适？

回答模板：

Dataset 更适合处理单样本逻辑，而 padding 是 batch 级决策，因为 pad 到多长取决于当前 batch 的最长样本。如果在 Dataset 里提

问题七：DataLoader 里如何保证随机性可复现？

回答模板：

单进程场景可以给 DataLoader 传入固定 seed 的 torch.Generator；多 worker 场景还要理解 worker seed，并在需要时使用 worker

4.17 常见误区

1. 以为 Dataset 要负责 batch 组装。通常单样本逻辑放 Dataset，batch 逻辑放 collate_fn。
2. 以为 DataLoader 只是迭代器。它还负责并行加载、采样和 batch 组装。
3. 以为变长序列一定先在 Dataset 里 pad。更常见是交给 collate_fn。
4. 以为 shuffle=True 就能解决分布式数据切分。多卡时通常还要配合 sampler。
5. 以为 padding token 可以随便算进 loss。多数任务里 padding 应该被 mask 掉。
6. 以为 num_workers 越大越好。它需要和 CPU、IO 和预处理成本一起权衡。
7. 以为 packing 只是把序列拼起来。其实还要处理样本边界、labels 和 mask。

4.18 小练习

1. 写一个 Dataset，返回文本和长度两个字段。
2. 写一个 collate_fn，把变长 token 序列 pad 成 batch。
3. 给 padding 部分构造 attention_mask 和 labels=-100。
4. 用 DataLoader 读取你的数据集，打印一个 batch 的 shape。
5. 把 shuffle=True 改成 DistributedSampler 形式，并在伪代码里写出 set_epoch。
6. 写一个简单的 length bucket 函数，比较 bucket 前后 batch 的平均 padding 比例。

4.19 本章总结

Dataset 负责单样本逻辑，DataLoader 负责 batch 组装和并行加载，collate_fn 负责把变长或复杂样本整理成训练可用的张量结构。对于大模型训练来说，最常见的数据处理任务是 padding、attention mask、labels 对齐和分布式采样。

要记住的主线是：

1. map-style Dataset 适合随机访问，IterableDataset 适合流式数据。
2. 变长样本通常在 collate_fn 里处理，而不是在 Dataset 里硬 pad。
3. padding、labels 和 attention_mask 必须一起设计。
4. shuffle、sampler 和 batch_sampler 是不同层次的采样控制。
5. num_workers、pin_memory、persistent_workers 影响吞吐和延迟。
6. 分布式训练必须保证样本切分正确，避免重复训练。
7. packing 能提升 token 利用率，但实现和调试复杂度更高。

下一章会进入训练循环工程，重点讲一个完整训练 step 应该怎么写，怎么组织 optimizer、scheduler、梯度裁剪、日志、验证、checkpoint 和异常恢复。

第五章：训练循环工程

训练循环是把模型、数据、损失函数和优化器连成一个可持续实验系统的地方。前面几章分别讲了 tensor、autograd、nn.Module 和 DataLoader，但真实训练不是把它们简单拼起来就结束。一个可靠的训练循环还要处理 device 搬运、loss 计算、反向传播、梯度裁剪、梯度累积、学习率调度、日志、验证、checkpoint、恢复训练、异常检测和可复现性。

很多新手训练脚本能跑，但不稳定：loss 记录不准、验证时忘记 no_grad、scheduler step 时机错、梯度累积没有缩放 loss、checkpoint 只保存模型没有保存 optimizer、恢复训练后学习率不对、训练中断无法续跑、NaN 出现后不知道从哪里查。本章目标是建立一套从最小训练 step 到工程化训练循环的完整框架。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前，已核对 PyTorch 官方 Optimizer.zero_grad、AdamW、LRScheduler / StepLR、clip_grad_norm、no_grad、inference_mode、Module.train() / eval()、模型保存加载教程和 checkpoint 相关文档口径。

本章聚焦大模型训练最常见的训练循环工程问题：forward、loss、backward、optimizer step、scheduler step、zero_grad(set_to_none=True)、梯度累积、梯度裁剪、语言模型 shift loss、batch 到 device、日志、验证循环、完整 checkpoint、恢复训练、随机状态和非有限 loss 检查。

本章不展开 AMP / GradScaler、FSDP / DeepSpeed / ZeRO、DDP 梯度同步、activation checkpointing、编译优化、生产级实验追踪平台或分布式 checkpoint shard。这些内容会分别放到混合精度、分布式训练、AI Infra 和训练平台章节中展开。

5.1 训练循环解决什么问题

一个训练循环的本质是反复执行：

1. 从 DataLoader 取一个 batch。
2. 把 batch 移到正确 device。

3. 执行 forward。
4. 计算 loss。
5. 执行 backward。
6. 更新参数。
7. 清理梯度。
8. 记录指标。
9. 定期验证和保存 checkpoint。

最小形式看起来很简单：

```
for batch in train_loader:
    optimizer.zero_grad()
    outputs = model(batch["input_ids"])
    loss = criterion(outputs, batch["labels"])
    loss.backward()
    optimizer.step()
```

但真实工程会多出很多约束：

1. batch 里可能有多个字段。
2. 不同 tensor 要移动到 GPU。
3. loss 可能来自模型内部，也可能来自外部 criterion。
4. 需要梯度裁剪防止 loss spike。
5. 大 batch 可能要用梯度累积模拟。
6. 学习率 scheduler 要和 optimizer step 对齐。
7. 验证必须关闭梯度并切换 eval 模式。
8. checkpoint 要能恢复完整训练状态。

面试回答：

训练循环不是只有 forward、backward 和 optimizer.step。工程上还要处理 batch 到 device、loss mask、梯度累积、梯度裁剪、sc

5.1.1 关键公式与训练循环调试速查

第一，普通单 step 的参数更新可以抽象为：

$$g_t = \nabla_{\theta_t} L_t(\theta_t), \quad \theta_{t+1} = U(\theta_t, g_t, \eta_t, s_t)$$

其中 θ_t 是第 t 次 optimizer step 前的参数， g_t 是当前 step 使用的梯度， η_t 是学习率， s_t 是 optimizer 内部状态，例如 AdamW 的一阶矩和二阶矩。训练循环的目标是保证 g_t 来自正确 batch、正确 loss、正确 scale，并且 s_t 和 η_t 能随 checkpoint 恢复。

第二，梯度累积时，真实 optimizer step 的梯度通常是多个 micro-batch 梯度的平均：

$$g_t = \frac{1}{K} \sum_{k=1}^K \nabla_{\theta_t} L_{t,k}(\theta_t)$$

其中 K 是 accum_steps。代码里常见做法是每个 micro-step 使用 loss / K 再 backward()。如果不除以 K ，梯度尺度会接近放大 K 倍。

第三，梯度裁剪常见口径是 global norm clipping：

$$\hat{g} = \begin{cases} g, & \text{if } \|g\|_2 \leq c \\ g \cdot \frac{c}{\|g\|_2}, & \text{if } \|g\|_2 > c \end{cases}$$

其中 c 是 max_norm。PyTorch 的 clip_grad_norm_ 会在已有梯度上原地裁剪，并返回裁剪前的总范数。

第四，按 optimizer step 更新的 scheduler 应跟随真实更新次数，而不是 micro-step 次数：

$$\eta_t = S(t_{\text{opt}})$$

其中 t_{opt} 是已经执行的 optimizer step 数。如果梯度累积 $K=4$ ，每个 micro-step 都调用 scheduler，学习率会比预期快 4 倍变化。

第五，语言模型训练常见 loss 是 shift 后的 token-level 平均：

$$L = \frac{1}{N_{\{\text{valid}\}} \sum_{i=1}^B \sum_{j=1}^{T-1} \|\mathbf{y}_{\{i,j+1\}} - \mathbf{y}_{\{i,j\}}\| \cdot \text{CE}(\mathbf{p}_{\{i,j\}}, \mathbf{y}_{\{i,j+1\}})}$$

其中 $\mathbf{p}_{\{i,j\}}$ 是第 j 个位置预测下一个 token 的词表分布， $\mathbf{y}_{\{i,j+1\}}$ 是下一个位置的 label， N_{valid} 是未被 ignore 的有效 label 数。

第六，完整训练 checkpoint 可以抽象为：

$$C_t = (\theta_t, s_t, h_t, e_t, r_t, c)$$

其中 θ_t 是模型权重， s_t 是 optimizer state， h_t 是 scheduler state， e_t 是 epoch / global step， r_t 是随机状态， c 是训练 config。只保存 θ_t 不能严格恢复训练轨迹。

5.2 最小训练 step

先看一个最小分类训练 step。

```
import torch
from torch import nn

model = nn.Linear(10, 3)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

x = torch.randn(32, 10)
labels = torch.randint(0, 3, (32,))

optimizer.zero_grad(set_to_none=True)
logits = model(x)
loss = criterion(logits, labels)
loss.backward()
optimizer.step()
```

关键顺序：

1. `zero_grad` 清掉旧梯度。
2. `forward` 得到 logits。
3. `loss` 连接 logits 和 labels。
4. `backward` 计算梯度。
5. `optimizer.step` 更新参数。

为什么通常先 `zero_grad` 再 `forward`？因为 PyTorch 梯度默认累积。只要确保每个 step 的 `backward` 前旧梯度被清掉，放在 step 末尾也能工作，但工程里更常见的是放在 `forward` 前，语义更清楚。

推荐：

```
optimizer.zero_grad(set_to_none=True)
set_to_none=True 会把梯度设为 None，通常更省内存，性能也更好。
```

5.3 batch 到 device

模型和数据必须在同一个 device 上。

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

训练循环中:

```
def move_to_device(batch, device):
    out = {}
    for key, value in batch.items():
        if torch.is_tensor(value):
            out[key] = value.to(device, non_blocking=True)
        else:
            out[key] = value
    return out
```

使用:

```
for batch in train_loader:
    batch = move_to_device(batch, device)
```

如果 DataLoader 使用 `pin_memory=True`, 配合 `non_blocking=True` 通常可以让 CPU 到 GPU 的搬运更高效。

常见错误:

Expected all tensors to be on the same device

排查时先打印:

```
print(next(model.parameters()).device)
print(batch["input_ids"].device)
```

5.4 loss 在哪里计算

loss 可以在模型内部计算, 也可以在训练循环外部计算。

5.4.1 模型外部计算 loss

```
logits = model(input_ids)
loss = nn.functional.cross_entropy(
    logits.view(-1, logits.size(-1)),
    labels.view(-1),
)
```

优点:

1. 模型只负责 forward, 职责清楚。
2. 便于复用同一个模型做不同任务。
3. loss 逻辑在 trainer 里更明显。

缺点:

1. 每个训练脚本都要写 loss 细节。
2. 对语言模型 shift logits/labels 容易重复出错。

5.4.2 模型内部计算 loss

```
outputs = model(input_ids=batch["input_ids"], labels=batch["labels"])
loss = outputs["loss"]
```

优点:

1. 训练脚本更简洁。
2. loss shift、mask 等逻辑封装在模型里。
3. 接近 Hugging Face 风格。

缺点:

1. 模型和训练目标绑定更紧。
2. 多任务或研究实验中可能不够灵活。

工程建议: 一个项目内保持一致。教学代码可以外部算 loss, 便于理解; 较完整的语言模型项目可以让 `forward(labels=...)` 返回 loss, 降低训练循环复杂度。

5.5 语言模型 loss 的标准形态

因果语言模型通常用当前位置预测下一个 token。假设:

logits: [B, T, V]

labels: [B, T]

需要错位:

```
shift_logits = logits[:, :-1, :]
```

```
shift_labels = labels[:, 1:]
```

再展平:

```
loss = nn.functional.cross_entropy(
    shift_logits.reshape(-1, shift_logits.size(-1)),
    shift_labels.reshape(-1),
    ignore_index=-100,
)
```

这里的关键点:

1. `logits[:, :-1, :]` 用第 0 到第 T-2 个位置预测后一个 token。
2. `labels[:, 1:]` 是目标 token。
3. padding 或不需要训练的位置应设为 -100。
4. `ignore_index=-100` 会跳过这些位置。

常见 bug:

1. 忘记 shift, 导致模型预测当前位置 token。
2. labels padding 用了 pad token id 而不是 -100。
3. logits 和 labels 展平维度不一致。
4. attention_mask 和 loss mask 混淆。

5.6 梯度裁剪

大模型训练中, loss spike 和梯度爆炸并不少见。梯度裁剪是常见稳定手段。

```
loss.backward()
grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

顺序很重要:

1. 先 `backward()` 得到梯度。
2. 再 clip grad。
3. 最后 optimizer step。

如果使用混合精度和 GradScaler, 需要先 unscale 再裁剪, 这会在下一章详细展开。

记录梯度范数很有用:

```
grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
logger["grad_norm"] = float(grad_norm)
```

如果训练突然发散, 优先查看:

1. loss 是否突然变大。
2. grad norm 是否突然变大。
3. 学习率是否在异常区间。
4. batch 是否包含异常样本。

5.7 梯度累积

显存有限时，常用梯度累积模拟更大的 batch。

假设每次只能放 micro_batch_size=4，但希望等效 batch 是 16，就可以累积 4 次再 step。

```
accum_steps = 4
optimizer.zero_grad(set_to_none=True)

for step, batch in enumerate(train_loader):
    outputs = model(**batch)
    loss = outputs["loss"]
    loss = loss / accum_steps
    loss.backward()

    if (step + 1) % accum_steps == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

为什么要除以 accum_steps？如果不除，每次 backward 的梯度会直接相加，累积 4 次后梯度尺度约等于平均梯度的 4 倍，相当于放大学习率。

还要注意最后不足一个 accum window 的 batch。如果训练循环允许最后一步也更新，可以写：

```
is_accum_step = (step + 1) % accum_steps == 0
is_last_step = (step + 1) == len(train_loader)

if is_accum_step or is_last_step:
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

5.8 optimizer 和 scheduler 的 step 时机

优化器更新参数，scheduler 更新学习率。

常见结构：

```
loss.backward()
optimizer.step()
scheduler.step()
optimizer.zero_grad(set_to_none=True)
```

大多数按 step 更新的 scheduler，scheduler.step() 放在 optimizer.step() 后。某些特殊 scheduler 或旧版本示例可能不同，要看文档。

梯度累积时，scheduler 通常应该只在真实 optimizer step 后更新，而不是每个 micro-step 都更新。

```
if should_step:
    optimizer.step()
    scheduler.step()
    optimizer.zero_grad(set_to_none=True)
```

常见错误：

1. 每个 micro-step 都 scheduler step，导致学习率衰减过快。

2. 恢复训练时没加载 scheduler state, 学习率从头开始。
3. warmup steps 按 batch 数算错, 没有考虑梯度累积。

5.9 日志记录: 不要只看 loss

训练日志至少应包含:

1. step 或 epoch。
2. train loss。
3. learning rate。
4. grad norm。
5. tokens/s 或 samples/s。
6. GPU 显存。
7. validation loss 或任务指标。

最小日志:

```
if global_step % log_every == 0:
    lr = optimizer.param_groups[0]["lr"]
    print({
        "step": global_step,
        "loss": loss.item(),
        "lr": lr,
    })
```

注意, 梯度累积时 `loss.item()` 可能是除以 `accum_steps` 后的值。如果想记录原始 loss, 可以在缩放前保存:

```
raw_loss = outputs["loss"]
loss = raw_loss / accum_steps
```

不要把带计算图的 loss tensor 长期保存到 list 里:

```
losses.append(loss) # 风险: 保留计算图
```

应保存数值:

```
losses.append(loss.item())
```

5.10 validation 循环

验证循环和训练循环不同。验证时应该:

1. `model.eval()`。
2. 使用 `torch.no_grad()` 或 `torch.inference_mode()`。
3. 不调用 `backward`。
4. 不调用 `optimizer step`。
5. 验证结束后切回 `model.train()`。

示例:

```
def evaluate(model, val_loader, device):
    model.eval()
    total_loss = 0.0
    total_count = 0

    with torch.inference_mode():
        for batch in val_loader:
            batch = move_to_device(batch, device)
            outputs = model(**batch)
            loss = outputs["loss"]
```

```

batch_size = batch["input_ids"].size(0)
total_loss += loss.item() * batch_size
total_count += batch_size

```

```

model.train()
return total_loss / max(total_count, 1)

```

为什么用 batch size 加权？因为最后一个 batch 可能更小，直接平均每个 batch 的 loss 会给小 batch 过高权重。

如果是语言模型，更严谨的做法是按有效 token 数加权，而不是按 batch size 加权：

```
valid_tokens = (batch["labels"] != -100).sum().item()
```

5.11 checkpoint 保存什么

一个完整训练 checkpoint 不应只保存模型权重。

推荐保存：

```

checkpoint = {
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "scheduler": scheduler.state_dict() if scheduler is not None else None,
    "epoch": epoch,
    "global_step": global_step,
    "config": config,
    "rng_state": torch.get_rng_state(),
}

```

```
torch.save(checkpoint, "checkpoint.pt")
```

如果使用 CUDA，还可以保存：

```
checkpoint["cuda_rng_state"] = torch.cuda.get_rng_state_all()
```

完整保存的好处：

1. 参数能恢复。
2. optimizer 动量和二阶矩能恢复。
3. scheduler 学习率位置能恢复。
4. step 和 epoch 能恢复。
5. 随机状态尽量一致。

如果只保存模型权重，恢复后虽然能继续训练，但 optimizer 和 scheduler 状态丢失，训练轨迹会改变，严重时会导致 loss spike。

5.12 checkpoint 加载和恢复训练

恢复训练：

```
checkpoint = torch.load("checkpoint.pt", map_location="cpu")
```

```

model.load_state_dict(checkpoint["model"])
optimizer.load_state_dict(checkpoint["optimizer"])

```

```

if scheduler is not None and checkpoint["scheduler"] is not None:
    scheduler.load_state_dict(checkpoint["scheduler"])

```

```

start_epoch = checkpoint["epoch"]
global_step = checkpoint["global_step"]

```

```
torch.set_rng_state(checkpoint["rng_state"])
```

如果 optimizer state 加载后在 CPU，而模型在 GPU，通常 optimizer 会在下一次 step 时处理，但更稳妥的做法是确保模型先移动到 device，再加载或迁移 optimizer state。真实项目中要根据框架行为检查。

推理只需要模型：

```
state = torch.load("model.pt", map_location="cpu")
model.load_state_dict(state)
model.eval()
```

训练恢复和推理加载是两个不同目标，不要混在一起。

5.13 保存 best checkpoint 和 last checkpoint

工程里常保存两类 checkpoint：

1. last.pt：最近一次状态，便于中断恢复。
2. best.pt：验证指标最好的模型，便于最终评估或上线。

示例：

```
best_val_loss = float("inf")
```

```
val_loss = evaluate(model, val_loader, device)
save_checkpoint("last.pt")
```

```
if val_loss < best_val_loss:
    best_val_loss = val_loss
    save_checkpoint("best.pt")
```

注意：

1. best.pt 如果用于继续训练，也要保存完整训练状态。
2. 如果只用于推理，可以只保存模型权重。
3. 文件名里可以带 step，便于回滚。

5.14 可复现性

可复现性不是绝对的，但可以尽量提高。

常见设置：

```
import random
import numpy as np
import torch
```

```
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

还可以设置：

```
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True
```

但要知道：

1. 完全确定性可能降低性能。

2. 某些 CUDA 算子本身可能非确定。
3. 分布式训练、DataLoader 多 worker、混合精度都会增加复现难度。
4. 面试中更重要的是知道哪些因素会影响复现。

5.15 训练中的异常检测

训练循环要主动检查异常。

5.15.1 NaN 和 Inf

```
if not torch.isfinite(loss):
    print("non-finite loss", loss.item())
    break
```

检查梯度：

```
for name, param in model.named_parameters():
    if param.grad is not None and not torch.isfinite(param.grad).all():
        print("bad grad", name)
```

5.15.2 loss spike

如果 loss 突然变大，排查顺序：

1. 当前 batch 是否异常。
2. 学习率是否过大或 warmup 配置错误。
3. grad norm 是否突然变大。
4. 混合精度是否溢出。
5. mask 是否出现全忽略或全 -inf。

5.15.3 OOM

如果出现 CUDA OOM：

1. 减小 batch size。
2. 减小 sequence length。
3. 使用梯度累积。
4. 使用混合精度。
5. 使用 gradient checkpointing。
6. 检查是否保存了带图 tensor。

5.16 一个工程化训练循环骨架

下面给出一个较完整但仍然可读的训练循环。

```
def train(
    model,
    train_loader,
    val_loader,
    optimizer,
    scheduler,
    device,
    num_epochs,
    accum_steps=1,
    log_every=10,
    eval_every=100,
):
    model.to(device)
```

```

model.train()

global_step = 0
optimizer.zero_grad(set_to_none=True)

for epoch in range(num_epochs):
    for micro_step, batch in enumerate(train_loader):
        batch = move_to_device(batch, device)

        outputs = model(**batch)
        raw_loss = outputs["loss"]
        loss = raw_loss / accum_steps

        if not torch.isfinite(raw_loss):
            raise RuntimeError(f"non-finite loss: {raw_loss.item()}")

        loss.backward()

        should_step = (micro_step + 1) % accum_steps == 0
        is_last = (micro_step + 1) == len(train_loader)

        if should_step or is_last:
            grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            if scheduler is not None:
                scheduler.step()
            optimizer.zero_grad(set_to_none=True)

            global_step += 1

            if global_step % log_every == 0:
                lr = optimizer.param_groups[0]["lr"]
                print({
                    "epoch": epoch,
                    "step": global_step,
                    "loss": raw_loss.item(),
                    "lr": lr,
                    "grad_norm": float(grad_norm),
                })

            if val_loader is not None and global_step % eval_every == 0:
                val_loss = evaluate(model, val_loader, device)
                print({"step": global_step, "val_loss": val_loss})
            model.train()

```

这个骨架覆盖了：

1. device 搬运。
2. loss 缩放。
3. 梯度累积。
4. 非有限 loss 检查。
5. 梯度裁剪。
6. optimizer 和 scheduler step。
7. 日志和验证。

真实项目还会继续加入 checkpoint 保存、分布式同步、混合精度、进度条、实验追踪、异常恢复和配置管理。

5.17 最小可运行训练循环审计 demo

下面这个 demo 用一个极小 causal LM 串起训练循环的关键动作：shift loss、ignore_index=-100、梯度累积、梯度裁剪、scheduler 跟随 optimizer step、token 加权验证、完整 checkpoint 和恢复验证。

```
import math
import tempfile
import warnings
from pathlib import Path

import torch
import torch.nn.functional as F
from torch import nn

warnings.filterwarnings("ignore", message="CUDA initialization.*")
IGNORE_INDEX = -100

def set_seed(seed=0):
    torch.manual_seed(seed)

class TinyCausalLM(nn.Module):
    def __init__(self, vocab_size=16, hidden_size=12):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.lm_head = nn.Linear(hidden_size, vocab_size)

    def forward(self, input_ids, labels=None):
        hidden = self.embed(input_ids)
        logits = self.lm_head(hidden)
        out = {"logits": logits}
        if labels is not None:
            shift_logits = logits[:, :-1, :].reshape(-1, logits.size(-1))
            shift_labels = labels[:, 1:].reshape(-1)
            out["loss"] = F.cross_entropy(
                shift_logits,
                shift_labels,
                ignore_index=IGNORE_INDEX,
            )
        return out

def make_batches():
    batches = []
    raw = [
        [[1, 2, 3, 4, 5, 6], [1, 2, 3, 4, 5, 0]],
        [[2, 3, 4, 5, 6, 7], [2, 3, 4, 5, 0, 0]],
        [[3, 4, 5, 6, 7, 8], [3, 4, 5, 0, 0, 0]],
        [[4, 5, 6, 7, 8, 9], [4, 5, 0, 0, 0, 0]],
    ]
    for pair in raw:
        input_ids = torch.tensor(pair, dtype=torch.long)
        labels = input_ids.clone()
        labels[input_ids == 0] = IGNORE_INDEX
        attention_mask = input_ids.ne(0)
```

```

        batches.append({
            "input_ids": input_ids,
            "labels": labels,
            "attention_mask": attention_mask,
        })
    return batches

def move_to_device(batch, device):
    return {k: v.to(device) if torch.is_tensor(v) else v for k, v in batch.items()}

def evaluate(model, batches, device):
    was_training = model.training
    model.eval()
    token_loss_sum = 0.0
    token_count = 0
    with torch.inference_mode():
        for batch in batches:
            batch = move_to_device(batch, device)
            out = model(batch["input_ids"], labels=batch["labels"])
            valid = int(batch["labels"][ :, 1:].ne(IGNORE_INDEX).sum().item())
            token_loss_sum += out["loss"].item() * valid
            token_count += valid
    if was_training:
        model.train()
    return token_loss_sum / max(token_count, 1)

def train_one_pass(model, batches, optimizer, scheduler, device, accum_steps=2):
    model.train()
    optimizer.zero_grad(set_to_none=True)
    optimizer_steps = 0
    raw_losses = []
    grad_norms = []

    for micro_step, batch in enumerate(batches):
        batch = move_to_device(batch, device)
        out = model(batch["input_ids"], labels=batch["labels"])
        raw_loss = out["loss"]
        if not torch.isfinite(raw_loss):
            raise RuntimeError("non-finite loss")

        (raw_loss / accum_steps).backward()
        raw_losses.append(round(raw_loss.item(), 4))

        should_step = (micro_step + 1) % accum_steps == 0
        is_last = (micro_step + 1) == len(batches)
        if should_step or is_last:
            grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad(set_to_none=True)
            optimizer_steps += 1
            grad_norms.append(round(float(grad_norm), 4))

```

```

    return raw_losses, grad_norms, optimizer_steps

set_seed(123)
device = torch.device("cpu")
model = TinyCausalLM().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=0.08, weight_decay=0.01)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=1, gamma=0.8)
batches = make_batches()

initial_val = evaluate(model, batches, device)
losses, grad_norms, optimizer_steps = train_one_pass(
    model, batches, optimizer, scheduler, device, accum_steps=2
)
final_val = evaluate(model, batches, device)

with tempfile.TemporaryDirectory() as tmpdir:
    ckpt_path = Path(tmpdir) / "last.pt"
    torch.save({
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(),
        "global_step": optimizer_steps,
        "rng_state": torch.get_rng_state(),
    }, ckpt_path)

    restored = TinyCausalLM().to(device)
    restored_optimizer = torch.optim.AdamW(restored.parameters(), lr=0.08, weight_decay=0.01)
    restored_scheduler = torch.optim.lr_scheduler.StepLR(restored_optimizer, step_size=1, gamma=0.8)
    checkpoint = torch.load(ckpt_path, map_location="cpu", weights_only=False)
    restored.load_state_dict(checkpoint["model"])
    restored_optimizer.load_state_dict(checkpoint["optimizer"])
    restored_scheduler.load_state_dict(checkpoint["scheduler"])
    torch.set_rng_state(checkpoint["rng_state"])
    restored_val = evaluate(restored, batches, device)

print("initial_val_loss=", round(initial_val, 4))
print("micro_losses=", losses)
print("grad_norms=", grad_norms)
print("optimizer_steps=", optimizer_steps)
print("scheduler_last_epoch=", scheduler.last_epoch)
print("final_lr=", round(optimizer.param_groups[0]["lr"], 5))
print("final_val_loss=", round(final_val, 4))
print("loss_decreased=", final_val < initial_val)
print("restored_val_matches=", math.isclose(final_val, restored_val, rel_tol=0, abs_tol=1e-7))
print("restored_global_step=", checkpoint["global_step"])
print("model_training_after_eval=", model.training)

```

期望输出类似：

```

initial_val_loss= 2.7991
micro_losses= [2.7366, 2.5374, 1.8882, 2.2651]
grad_norms= [1.7233, 1.478]
optimizer_steps= 2
scheduler_last_epoch= 2

```

```
final_lr= 0.0512
final_val_loss= 1.1386
loss_decreased= True
restored_val_matches= True
restored_global_step= 2
model_training_after_eval= True
```

这段输出要检查几件事：

1. `micro_losses` 有 4 个值，但 `optimizer_steps=2`，说明 `accum_steps=2` 生效。
2. `scheduler_last_epoch=2`，说明 `scheduler` 只在真实 `optimizer step` 后更新。
3. `final_lr=0.0512`，对应 $0.08 * 0.8^2$ 。
4. `loss_decreased=True`，说明训练循环确实更新了参数。
5. `restored_val_matches=True`，说明模型、`optimizer`、`scheduler` 和随机状态可以从 `checkpoint` 恢复到一致验证结果。
6. `model_training_after_eval=True`，说明验证函数结束后恢复了原本的训练模式。

5.18 训练循环常见 bug

5.18.1 loss 不下降

排查顺序：

1. 数据和 `labels` 是否正确。
2. `loss` 是否真的依赖模型参数。
3. 参数是否进入 `optimizer`。
4. 学习率是否过大或过小。
5. 梯度是否为 `None`、`0`、`NaN` 或 `Inf`。
6. 是否忘记 `optimizer.step()`。
7. 是否在 `forward` 中误用了 `no_grad()` 或 `detach()`。

5.18.2 训练速度很慢

可能原因：

1. `DataLoader` 太慢。
2. `batch` 到 `device` 阻塞。
3. 频繁打印日志。
4. 验证太频繁。
5. 保存 `checkpoint` 太频繁。
6. Python 逻辑太重。

5.18.3 恢复训练后 loss spike

可能原因：

1. 没加载 `optimizer state`。
2. 没加载 `scheduler state`。
3. 学习率从初始值重新开始。
4. 混合精度 `scaler` 状态丢失。
5. 数据顺序或随机状态变化。

5.18.4 验证结果不稳定

可能原因：

1. 忘记 `model.eval()`。
2. 忘记 `torch.no_grad()` 或 `inference_mode()`。
3. 验证集 `shuffle` 或采样不稳定。

4. 生成评估使用了随机采样。
5. 指标聚合方式错误。

5.19 面试官会怎么问

问题一：一个标准训练 step 包含哪些步骤？

回答模板：

通常包括 zero_grad、forward、loss 计算、backward、可选梯度裁剪、optimizer.step、scheduler.step 和日志记录。顺序上要先

问题二：梯度累积为什么要把 loss 除以 accum_steps？

回答模板：

因为 PyTorch 梯度默认累积。如果累积 N 个 micro-batch 但不缩放 loss，最终梯度尺度会接近平均梯度的 N 倍，相当于放大学习率。

问题三：验证循环和训练循环有什么区别？

回答模板：

验证时要 model.eval()，并使用 no_grad 或 inference_mode 关闭梯度，不执行 backward 和 optimizer.step。验证结束后要切回 mo

问题四：checkpoint 应该保存什么？

回答模板：

如果只是推理，保存模型 state_dict 就够了；如果要恢复训练，应该保存 model、optimizer、scheduler、epoch、global_step、co

问题五：训练 loss 出现 NaN 怎么排查？

回答模板：

先确认是 loss 本身 NaN 还是梯度 NaN，再看当前 batch、学习率、grad norm、混合精度 loss scaling、mask 是否出现全 - inf、输入是否有异常值。工程上我会记录 loss、lr、grad norm，并在非有限 loss 出现时保存问题 batch 或中断排查。

5.20 常见误区

1. 以为训练循环只要能跑就行。工程训练循环必须可观测、可恢复、可排查。
2. 忘记 optimizer.zero_grad()，导致梯度跨 step 累积。
3. 梯度累积时没有缩放 loss。
4. 每个 micro-step 都 scheduler step，导致学习率变化过快。
5. 验证时只写 model.eval()，忘记关闭梯度。
6. checkpoint 只保存模型，不保存 optimizer 和 scheduler。
7. 日志里保存带图 tensor，导致显存增长。
8. 只看平均 loss，不看 grad norm、lr、tokens/s 和验证指标。
9. 恢复训练后不检查学习率，导致训练轨迹异常。

5.21 小练习

1. 写一个最小分类训练循环，包含 zero_grad、forward、loss、backward 和 step。
2. 给训练循环加 move_to_device，支持 batch dict。
3. 给语言模型 logits 和 labels 写 shift loss。
4. 给训练循环加入梯度裁剪，并打印 grad norm。
5. 实现梯度累积，确认 optimizer step 次数减少。
6. 写一个验证函数，正确使用 eval() 和 inference_mode()。
7. 保存完整 checkpoint，并写恢复训练代码。
8. 故意把 scheduler step 放到 micro-step，观察学习率变化是否符合预期。

9. 在日志中记录 loss、lr、grad norm 和 samples/s。
10. 构造一个 NaN loss，并写出排查打印代码。

5.22 本章总结

训练循环是深度学习工程的主干。一个合格的训练循环不仅要完成 forward、loss、backward 和 optimizer step，还要处理 device、梯度累积、梯度裁剪、scheduler、日志、验证、checkpoint、恢复训练和异常检测。

需要记住的主线：

1. 每个 step 都要明确清梯度、前向、反向和更新参数的顺序。
2. 语言模型 loss 要正确 shift，并用 ignore_index 忽略不该训练的位置。
3. 梯度裁剪放在 backward 后、optimizer step 前。
4. 梯度累积要缩放 loss，并且 scheduler 通常只跟随真实 optimizer step。
5. 验证需要 eval() 和 no_grad 或 inference_mode()。
6. 完整 checkpoint 应保存模型、optimizer、scheduler、step、config 和随机状态。
7. 稳定训练依赖可观测性：loss、lr、grad norm、吞吐、显存和验证指标都要看。

下一章会进入混合精度与显存优化，讲清 fp16、bf16、autocast、GradScaler、activation checkpointing、显存组成和常见 OOM 排查。

第六章：混合精度与显存优化

大模型训练最常见的两个资源瓶颈是显存和算力。本章讨论如何在尽量不损失模型质量的前提下，降低显存占用、提高吞吐并避免数值不稳定。核心手段包括混合精度训练、梯度缩放、activation checkpointing、梯度累积、参数和激活的显存规划，以及对 OOM 的系统性排查。

很多训练脚本不是模型不对，而是显存吃不下：batch size 太大、序列太长、激活太多、优化器状态太重、没有用 AMP、梯度没有及时释放、保存了多余中间 tensor、DataLoader 把 batch 预取得过多、验证时忘了关梯度。本章的目标是建立一个对显存有感知的训练思维。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前，已核对 PyTorch 官方 AMP / torch.amp.autocast / torch.amp.GradScaler、Automatic Mixed Precision examples、torch.utils.checkpoint.checkpoint、CUDA memory management 和 torch.cuda.memory_allocated / memory_reserved / max_memory_allocated 文档口径。

本章聚焦大模型训练中最常见的混合精度与显存优化问题：FP32 / FP16 / BF16 的存储和数值边界、autocast、GradScaler、loss scaling / unscale、activation checkpointing、梯度累积、参数 / 梯度 / optimizer state / activation 显存估算、显存泄漏和 OOM 排查。

本章不展开 FP8 训练、ZeRO / FSDP 参数分片、Tensor Parallel、Sequence Parallel、FlashAttention kernel 内部、CUDA allocator 细节、NVIDIA Nsight profiling 或生产级显存调度器。这些内容会在分布式训练、推理框架、AI Infra 和性能 profiling 章节中展开。

6.1 显存到底花在哪里

训练时显存通常分成几类：

1. 模型参数。
2. 梯度。
3. 优化器状态。
4. 前向激活。
5. 临时 workspace 和 kernel buffer。
6. 输入 batch 和中间缓存。

粗略理解：

1. 参数是模型本体。

2. 梯度是 backward 的结果。
3. 优化器状态通常比参数还大，尤其是 Adam 系列。
4. 激活和序列长度、batch size、层数强相关。
5. 临时 buffer 取决于算子实现和 kernel。

在大模型训练里，很多时候真正的瓶颈不是参数，而是激活和优化器状态。

面试回答：

训练显存主要花在参数、梯度、优化器状态、激活和临时 buffer 上。参数是模型本体，梯度是反向传播结果，Adam 类优化器还要保

6.1.1 关键公式与显存调试速查

第一，单个 tensor 的存储量可以粗略写成：

$$M(X) = n(X) \cdot b(X)$$

其中 $n(X)$ 是元素个数， $b(X)$ 是每个元素的字节数。常见情况下，FP32 是 4 bytes，FP16 / BF16 是 2 bytes。

第二，训练显存可以粗略分解为：

$$M_{\{\mathrm{train}\}} \approx M_{\{\mathrm{param}\}} + M_{\{\mathrm{grad}\}} + M_{\{\mathrm{opt}\}} + M_{\{\mathrm{act}\}} + M_{\{\mathrm{tmp}\}}$$

其中 M_{param} 是参数， M_{grad} 是梯度， M_{opt} 是 optimizer state， M_{act} 是前向激活， M_{tmp} 是临时 workspace、kernel buffer 和缓存。

第三，AdamW 常见状态显存大致是：

$$M_{\{\mathrm{AdamW}\}} \approx 2P \cdot b_{\{\mathrm{state}\}}$$

其中 P 是参数个数， b_{state} 是一阶矩和二阶矩的存储字节数。如果状态使用 FP32，则每个参数额外需要约 8 bytes 的 optimizer state。

第四，Transformer 激活显存可以粗略理解为随 batch、序列长度、hidden size 和层数线性增长：

$$M_{\{\mathrm{act}\}} \propto B T d L b_{\{\mathrm{act}\}}$$

其中 B 是 batch size， T 是 sequence length， d 是 hidden size， L 是层数， b_{act} 是激活 dtype 字节数。实际 attention、中间 MLP、checkpointing 和 kernel 实现会改变常数项。

第五，activation checkpointing 的核心取舍是：

$$M_{\{\mathrm{act}\}}^{\{\mathrm{ckpt}\}} < M_{\{\mathrm{act}\}}, \quad C_{\{\mathrm{backward}\}}^{\{\mathrm{ckpt}\}} > C_{\{\mathrm{backward}\}}$$

也就是用反向时额外重算一部分 forward，换取更低的前向激活保存量。

第六，loss scaling 的数值直觉是：

$$\begin{aligned} \tilde{L} &= sL, \\ \nabla_{\theta} \tilde{L} &= s \nabla_{\theta} L, \\ g &= \frac{1}{s} \nabla_{\theta} \tilde{L} \end{aligned}$$

其中 s 是 scale factor。FP16 中先放大 loss 可以减少小梯度下溢风险；optimizer step 前要 unscale 回真实梯度，再做 gradient clipping。

6.2 为什么需要混合精度

传统上很多训练用 fp32。fp32 精度高，但显存和带宽开销大。

混合精度训练的思路是：

1. 用较低精度存储和计算大部分张量。
2. 保留少数关键位置使用较高精度。
3. 在数值敏感环节保持稳定。

常见组合：

1. fp16。
2. bf16。
3. fp32 + amp。

它的目标不是 “全部变低精度”，而是找到质量、性能和显存之间的平衡点。

优势：

1. 显存更省。
2. 带宽压力更小。
3. 通常训练更快。

代价：

1. 数值更脆弱。
2. 某些算子容易溢出或下溢。
3. 需要更多工程控制。

6.3 FP16 和 BF16 的区别

fp16 和 bf16 都是 16 位浮点，但表示方式不同。

6.3.1 FP16

fp16 的优点是：

1. 计算和存储都省。
2. 在很多 GPU 上支持成熟。

缺点是：

1. 指数范围较小。
2. 更容易溢出、下溢。
3. 训练时通常要配合 GradScaler。

6.3.2 BF16

bf16 的指数范围接近 fp32，动态范围更大。

优点是：

1. 更不容易数值溢出。
2. 大模型训练通常更稳。
3. 在很多现代硬件上支持很好。

缺点是：

1. 尾数精度比 fp16 低。
2. 某些硬件或旧环境支持不如 fp16 普遍。

工程上常见判断：

1. 如果硬件和框架支持良好，bf16 往往更省心。
2. 如果只能用 fp16，通常要配合 GradScaler。

6.4 autocast：自动混合精度

PyTorch 的 AMP 通过 autocast 自动把合适的算子放到低精度执行。

示例：

```
import torch

with torch.amp.autocast("cuda", dtype=torch.float16):
    logits = model(input_ids)
    loss = criterion(logits, labels)
```

直觉：

1. 某些算子适合低精度，比如大部分矩阵乘法。
2. 某些算子更适合保留高精度，比如 softmax、归一化、loss 计算中的敏感部分。
3. autocast 负责自动选择。

工程意义：

1. 减少手动 dtype 管理。
2. 更容易获得性能收益。
3. 降低写错精度策略的概率。

但要记住：autocast 不是万能的。它只是帮助管理前向中的算子精度，仍然需要结合模型结构和具体异常去判断。

6.5 GradScaler：解决 fp16 的数值问题

fp16 训练容易遇到梯度太小导致下溢。GradScaler 的做法是先放大 loss，再反向传播，再把梯度缩回去。

典型结构：

```
import torch

scaler = torch.amp.GradScaler("cuda")

for batch in train_loader:
    optimizer.zero_grad(set_to_none=True)

    with torch.amp.autocast("cuda", dtype=torch.float16):
        outputs = model(**batch)
        loss = outputs["loss"]

    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
```

关键步骤：

1. scale(loss) 放大 loss。
2. backward() 在放大后的尺度上计算梯度。
3. unscale_(optimizer) 把梯度缩回来。
4. 必要时再做梯度裁剪。
5. step 和 update 完成参数更新和缩放因子调整。

为什么要先 unscale_ 再裁剪？因为裁剪的是实际梯度，不是被放大的临时梯度。

bf16 通常不那么依赖 GradScaler，但代码结构上仍可以保持一致。

6.6 mixed precision 训练的推荐模板

一个常见训练模板：

```
scaler = torch.amp.GradScaler("cuda", enabled=use_fp16)
```

```

for batch in train_loader:
    batch = move_to_device(batch, device)
    optimizer.zero_grad(set_to_none=True)

    with torch.amp.autocast("cuda", dtype=torch.float16, enabled=use_amp):
        outputs = model(**batch)
        loss = outputs["loss"]

    if use_fp16:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
    else:
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

```

这里要注意：

1. autocast 控制前向精度。
2. GradScaler 主要服务 fp16。
3. bf16 训练通常不需要缩放，但也可以保留统一结构。

6.7 activation checkpointing

activation checkpointing 的核心思想是：前向时少存一部分激活，反向时再重新计算这部分前向，从而用计算换显存。

直觉：

1. 普通训练保留很多中间激活，方便 backward。
2. checkpointing 只保留关键节点。
3. backward 时重新跑部分 forward 来恢复中间值。

PyTorch 里可以这样用：

```
from torch.utils.checkpoint import checkpoint
```

```

def forward_block(x, block):
    return checkpoint(block, x)

```

或者在模型里包住某些子模块。

优点：

1. 明显省显存。
2. 可以支持更长序列或更大 batch。

缺点：

1. backward 更慢，因为要重算前向。
2. 代码结构更复杂。

适合：

1. Transformer block 很深的模型。
2. 长上下文训练。
3. 显存非常紧张实验。

6.8 梯度累积和显存

梯度累积不仅是优化 batch size 的办法，也是显存优化手段。

如果一次只能放很小的 batch，就把多个 micro-batch 的梯度累积起来再更新。

示意：

```
accum_steps = 8
optimizer.zero_grad(set_to_none=True)

for step, batch in enumerate(train_loader):
    with torch.amp.autocast("cuda", dtype=torch.float16, enabled=use_amp):
        outputs = model(**batch)
        loss = outputs["loss"] / accum_steps

    loss.backward()

    if (step + 1) % accum_steps == 0:
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

优点：

1. 降低单次 forward/backward 的激活峰值。
2. 模拟更大 batch。

缺点：

1. 更新频率变低。
2. 训练吞吐不一定线性提升。

6.9 影响显存的关键因素

显存不是只由 batch size 决定，还包括：

1. 序列长度。
2. 模型层数。
3. hidden size。
4. 注意力头数。
5. 精度。
6. optimizer 类型。
7. 是否保留中间激活。
8. 是否开启 checkpointing。
9. 是否存了额外的日志或缓存。

尤其对 Transformer 来说，序列长度增加往往会显著推高激活和 attention 相关开销。

一个实用判断：如果显存爆了，先问自己是参数太大、batch 太大、序列太长、还是优化器状态太重。

6.10 optimizer 状态为什么也占很多显存

以 Adam 为例，除了参数本身，还要保存一阶矩和二阶矩。

直觉上，每个参数可能对应：

1. 参数值。
2. 梯度。
3. 一阶矩。
4. 二阶矩。

所以 Adam 类优化器的总显存压力明显高于 SGD。

这也是为什么在超大模型训练里，经常会考虑：

1. 参数分片。
2. 优化器状态分片。
3. 更低精度状态。
4. 更省显存的训练策略。

本章重点不是展开分布式分片，而是先建立 “optimizer 状态本身也很贵” 的意识。

6.11 训练中常见的显存优化顺序

当你遇到 OOM，可以按下面顺序试：

1. 减小 batch size。
2. 减小 sequence length。
3. 开启 mixed precision。
4. 开启 gradient accumulation。
5. 开启 activation checkpointing。
6. 检查是否保留了多余 tensor。
7. 检查 validation 是否忘了关梯度。
8. 检查 DataLoader 预取和 batch 缓存。

这个顺序的逻辑是：

1. 先改最直接影响显存的变量。
2. 再上混合精度和 checkpointing。
3. 最后排查代码层面的泄漏。

6.12 显存泄漏的常见来源

不是所有显存增长都是 “模型太大”。很多是代码把图保留下来了。

常见来源：

1. 把带计算图的 loss 存进 list。
2. 把中间 activation 长期缓存。
3. 验证时没用 no_grad 或 inference_mode。
4. hook 里保存了未 detach 的 tensor。
5. 每步都把大 batch 的输出堆到内存里。
6. 重复创建没有释放的大 tensor。

如果怀疑泄漏，可以每隔几步打印：

```
print(torch.cuda.memory_allocated() / 1024**2)
print(torch.cuda.memory_reserved() / 1024**2)
```

还可以更系统地用 PyTorch 的显存统计接口排查。

6.13 OOM 排查方法

当出现 OOM 时，不要只盯着最后一行报错，要分层排查。

6.13.1 先看是训练还是验证

1. 如果训练 OOM，关注 batch、seq len、激活、梯度和 optimizer state。
2. 如果验证 OOM，重点看是否忘记关梯度、是否缓存了输出、是否 batch 过大。

6.13.2 先缩小 batch 和 sequence length

这是最直接的验证手段。若缩小后能跑，说明问题和激活/序列长度强相关。

6.13.3 检查是否在累积图

例如：

```
losses.append(loss)
```

如果长期保存带图对象，显存会持续增长。

应该改成：

```
losses.append(loss.item())
```

6.13.4 检查验证流程

验证时应：

1. `model.eval()`。
2. `torch.inference_mode()`。
3. 不做 `backward`。
4. 不把大输出长期保存在 GPU 上。

6.14 一个完整 AMP + checkpointing 的训练骨架

```
import torch
from torch.utils.checkpoint import checkpoint
```

```
def train_step(model, batch, optimizer, scaler, use_amp=True):
    optimizer.zero_grad(set_to_none=True)

    with torch.amp.autocast("cuda", dtype=torch.float16, enabled=use_amp):
        outputs = model(**batch)
        loss = outputs["loss"]

    if use_amp:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
    else:
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

    return loss.item()
```

如果模型内部某些 block 需要 checkpointing，可以把对应子模块包装起来。重点是要理解：

1. AMP 解决的是精度和带宽问题。
2. checkpointing 解决的是激活显存问题。
3. 梯度累积解决的是单步 batch 太大的问题。

这三者经常一起用。

6.15 最小可运行 AMP 与显存审计 demo

下面这个 demo 不依赖 GPU，也可以在 CPU 上运行。它演示四件事：显存组成估算、autocast 会让适合的算子输出 BF16、GradScaler 的 scale / unscale / step 结构，以及 activation checkpointing 会在 backward 阶段重算部分 forward。

```
import warnings

import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.checkpoint import checkpoint

warnings.filterwarnings("ignore", message="CUDA initialization.*")

DTYPE_BYTES = {
    "fp32": 4,
    "fp16": 2,
    "bf16": 2,
}

def mib(num_bytes):
    return round(num_bytes / 1024**2, 3)

def estimate_memory(param_count, activation_elements, param_dtype="bf16", activation_dtype="bf16"):
    param_bytes = param_count * DTYPE_BYTES[param_dtype]
    grad_bytes = param_count * DTYPE_BYTES[param_dtype]
    adamw_state_bytes = 2 * param_count * DTYPE_BYTES["fp32"]
    activation_bytes = activation_elements * DTYPE_BYTES[activation_dtype]
    return {
        "params_mib": mib(param_bytes),
        "grads_mib": mib(grad_bytes),
        "adamw_states_mib": mib(adamw_state_bytes),
        "activations_mib": mib(activation_bytes),
        "total_mib": mib(param_bytes + grad_bytes + adamw_state_bytes + activation_bytes),
    }

class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(8, 16), nn.ReLU(), nn.Linear(16, 4))

    def forward(self, x):
        return self.net(x)

torch.manual_seed(0)
model = TinyMLP()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
scaler = torch.amp.GradScaler("cpu")

x = torch.randn(6, 8)
y = torch.tensor([0, 1, 2, 3, 1, 0])
```



```

optimizer.zero_grad(set_to_none=True)
with torch.amp.autocast("cpu", dtype=torch.bfloat16):
    logits = model(x)
    loss = F.cross_entropy(logits, y)

autocast_dtype = str(logits.dtype).replace("torch.", "")
scale_before = scaler.get_scale()
scaler.scale(loss).backward()
scaler.unscale_(optimizer)
grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
scaler.step(optimizer)
scaler.update()

```

```

class CountBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(8, 8)
        self.calls = 0

    def forward(self, z):
        self.calls += 1
        return torch.relu(self.linear(z))

```

```

plain_block = CountBlock()
z = torch.randn(4, 8, requires_grad=True)
plain_block(z).sum().backward()
plain_calls = plain_block.calls

```

```

ckpt_block = CountBlock()
ckpt_block.load_state_dict(plain_block.state_dict())
z2 = torch.randn(4, 8, requires_grad=True)
checkpoint(ckpt_block, z2, use_reentrant=False).sum().backward()
ckpt_calls = ckpt_block.calls

```

```

estimate = estimate_memory(param_count=1_000_000, activation_elements=2_000_000)
print("memory_estimate=", estimate)
print("autocast_dtype=", autocast_dtype)
print("scaler_enabled=", scaler.is_enabled())
print("scale_before=", float(scale_before))
print("grad_norm_before_clip=", round(float(grad_norm), 4))
print("scale_after=", float(scaler.get_scale()))
print("checkpoint_calls=", {"plain": plain_calls, "checkpoint": ckpt_calls})
print("checkpoint_recomputed=", ckpt_calls > plain_calls)
print("cuda_available=", torch.cuda.is_available())

```

期望输出类似:

```

memory_estimate= {'params_mib': 1.907, 'grads_mib': 1.907, 'adamw_states_mib': 7.629, 'activations_mib': 3.815, 'total_mib': 15.25}
autocast_dtype= bfloat16
scaler_enabled= True
scale_before= 65536.0
grad_norm_before_clip= 0.746
scale_after= 65536.0
checkpoint_calls= {'plain': 1, 'checkpoint': 2}

```

```
checkpoint_recomputed= True
cuda_available= False
```

这段输出要检查几件事：

1. AdamW 的两个 FP32 state 在估算中比 BF16 参数和梯度更大，说明 optimizer state 是训练显存大头之一。
2. autocast_dtype=bfloat16，说明 CPU autocast 已让线性层输出进入 BF16。
3. scale_before=65536.0 展示 GradScaler 的初始 loss scale；本例无溢出，所以 scale_after 不变。
4. checkpoint_calls 中 checkpoint 版本调用两次 forward，说明 backward 阶段发生了重算。
5. cuda_available=False 只是当前机器状态，不影响本 demo 解释 AMP 和 checkpointing 的核心机制。

6.16 工程中怎么选 fp16 和 bf16

简单经验：

1. 如果硬件支持良好且你希望更稳，优先 bf16。
2. 如果环境主要支持 fp16，使用 fp16 + GradScaler。
3. 如果发现训练频繁 NaN，先看学习率、初始化、loss、mask，再看换 bf16 是否更稳。

不要把“混合精度”理解成固定 recipe。它仍然要和模型结构、数据分布和硬件一起权衡。

6.17 面试官会怎么问

问题一：FP16 和 BF16 有什么区别？

回答模板：

两者都是 16 位浮点，但 BF16 的指数位更多，动态范围更接近 FP32，因此更不容易溢出，通常训练更稳；FP16 的精度和效率也很好，

问题二：autocast 和 GradScaler 分别做什么？

回答模板：

autocast 负责自动把适合的算子放到低精度执行，减少手动 dtype 管理；GradScaler 主要用于 FP16，通过放大 loss、反向后再缩回

问题三：activation checkpointing 为什么能省显存？

回答模板：

因为它不保存所有前向激活，而是在 backward 时重新计算一部分前向，从而用额外计算换取更少的激活显存。它适合深层模型和长序

问题四：为什么 Adam 比 SGD 更占显存？

回答模板：

因为 Adam 不只保存参数和梯度，还要为每个参数保存一阶矩和二阶矩状态，所以 optimizer state 很大。对超大模型来说，optimizer

问题五：训练 OOM 时你怎么排查？

回答模板：

我会先区分是训练还是验证阶段，再从 batch size、sequence length、精度、梯度累积、activation checkpointing、optimizer state

6.18 常见误区

1. 以为 fp16 一定比 bf16 更好。实际上 bf16 往往更稳。
2. 以为 AMP 等于自动省一切显存。它主要解决精度和部分带宽问题，不是万能。
3. 以为 GradScaler 对所有混合精度都必须。bf16 通常不一定需要。
4. 以为 checkpointing 没有代价。它会增加 backward 计算量。

5. 以为 OOM 都是 batch 太大。也可能是 optimizer state、保存图或验证没关梯度。
6. 以为 loss 降低说明显存问题消失。数值稳定和显存问题是两个不同维度。

6.19 小练习

1. 写一个带 autocast 的最小训练 step。
2. 把训练循环改成 fp16 + GradScaler 版本。
3. 给 Transformer block 加 checkpointing 包装。
4. 统计不同 batch size 下的 memory_allocated 和 memory_reserved。
5. 故意把验证循环写错，忘记 inference_mode，观察显存变化。
6. 写一个 OOM 排查 checklist。
7. 比较 fp32、fp16、bf16 下同一模型的训练速度和稳定性。

6.20 本章总结

混合精度与显存优化的核心不是背几个 API，而是建立资源感知的训练思维。

要记住的主线：

1. 显存主要花在参数、梯度、优化器状态、激活和临时 buffer。
2. fp16 和 bf16 都是混合精度方案，但 bf16 通常更稳。
3. autocast 管前向精度，GradScaler 主要解决 fp16 的数值问题。
4. activation checkpointing 用计算换显存。
5. 梯度累积可以模拟更大 batch 并缓解单步显存压力。
6. OOM 排查要区分训练、验证、显存泄漏和配置错误。
7. 一个好的训练脚本要同时考虑数值稳定、吞吐和显存峰值。

下一章会进入分布式训练入门，讲清数据并行、梯度同步、DDP、通信开销和多 GPU 训练的基本组织方式。

第七章：分布式训练入门

单卡训练能帮助你理解模型和训练循环，但大模型工程很快会遇到单卡装不下、训练太慢、数据太多的问题。分布式训练的目标是把计算、数据、参数、梯度或优化器状态拆到多张 GPU、多台机器上，让训练规模和速度突破单卡限制。

本章是入门章节，不追求覆盖所有 Megatron、DeepSpeed、ZeRO、FSDP 细节，而是先建立分布式训练的核心概念：rank、world size、process group、DDP、all-reduce、DistributedSampler、torchrun、FSDP 的基本直觉，以及常见 debug 方法。只要这些概念清楚，后续学习训练系统和大模型训练框架会容易很多。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前，已核对 PyTorch 官方 torch.distributed、DistributedDataParallel、torchrun / elastic launch、DistributedSampler、collective communication、all_reduce 和 Fully Sharded Data Parallel 文档口径。

本章聚焦 PyTorch 分布式训练入门中最常用、最容易在面试中被追问的内容：rank / local rank / world size、process group、DDP 梯度同步、all-reduce、torchrun 环境变量、DistributedSampler 数据切分、global batch size、DDP 与 DataParallel / FSDP 的边界、no_sync()、rank 0 checkpoint 和分布式 debug。

本章不展开 ZeRO / DeepSpeed、Megatron-LM、tensor parallel、pipeline parallel、sequence parallel、MoE expert parallel、NCCL 拓扑调优、RDMA / InfiniBand、分布式 checkpoint shard 格式或生产级集群调度。这些内容会在大模型训练系统、AI Infra 和性能优化章节中继续展开。

7.1 为什么需要分布式训练

单卡训练有两个主要限制：

1. 显存不够。
2. 速度不够。

显存不够时，一个 GPU 放不下模型参数、梯度、优化器状态和激活。速度不够时，即使能放下模型，训练一个 epoch 或一轮预训练也可能太慢。

分布式训练试图解决这些问题：

1. 数据并行：每张卡放一份模型，处理不同 batch，最后同步梯度。
2. 参数分片：把参数、梯度或优化器状态拆到不同卡上。
3. 张量并行：把单层大矩阵计算拆到多卡。
4. 流水线并行：把模型层切到多卡，像流水线一样执行。

本章重点先讲数据并行和 DDP，因为它是最常用、最容易入门的分布式训练方式。

面试回答：

分布式训练主要解决单卡显存和速度限制。最基础的是数据并行，每张 GPU 持有一份模型，处理不同 mini-batch，反向传播后通过 all-reduce 同步梯度。更大规模时会进一步使用参数分片、张量并行和流水线并行来降低单卡显存和提升训练速度。

7.1.1 关键公式与分布式调试速查

第一，rank、local_rank 和机器编号的关系可以粗略写成：

$$r = n_{\{\mathrm{node}\}} G + r_{\{\mathrm{local}\}}$$

其中 r 是全局 rank, n_{node} 是机器编号, G 是每台机器的 GPU 数, r_{local} 是本机内 GPU 或进程编号。world_size=W 表示总进程数。

第二，DDP 数据并行的梯度同步可以写成：

$$\bar{g}_t = \frac{1}{W} \sum_{r=0}^{W-1} g_{t,r}$$

其中 $g_{t,r}$ 是第 r 个 rank 在第 t 次更新前得到的本地梯度， $\bar{g}_t$ 是 all-reduce 后每个 rank 都拿到的平均梯度。

第三，每个 rank 用同一个平均梯度更新参数：

$$\theta_{t+1} = \theta_t - \eta \bar{g}_t$$

因此只要初始参数一致、梯度同步一致、optimizer step 顺序一致，各 rank 的参数会保持一致。

第四，global batch size 是：

$$B_{\{\mathrm{global}\}} = B_{\{\mathrm{device}\}} \cdot W \cdot A$$

其中 B_{device} 是每个 rank 的 micro-batch size, W 是 world size, A 是 gradient accumulation steps。从单卡切到多卡时，如果每卡 batch 不变，global batch 会变大，学习率、warmup、训练步数和日志统计都要重新核对。

第五，DistributedSampler 的目标是让每个 rank 看到不同索引子集：

$$I_r \cap I_s = \varnothing, \quad r \neq s$$

在不丢样本的普通 map-style dataset 场景中，希望所有 rank 的索引集合覆盖当前 epoch 的训练样本。真实实现还要处理 shuffle、补齐或 drop_last。

第六，DDP 单卡状态显存不因 world size 自动下降：

$$M_{\{\mathrm{DDP}, \mathrm{rank}\}} \approx M_{\{\mathrm{param}\}} + M_{\{\mathrm{grad}\}} + M_{\{\mathrm{opt}\}} + M_{\{\mathrm{act}\}}$$

DDP 主要提升吞吐；如果单卡装不下完整参数、梯度和 optimizer state，需要进一步考虑 FSDP / ZeRO 等分片方法。

第七，梯度累积配合 no_sync() 的通信次数可以近似理解为：

$$C_{\{\mathrm{sync}, \mathrm{no_sync}\}} \approx \frac{C_{\{\mathrm{sync}, \mathrm{naive}\}}}{A}$$

也就是 A 个 micro-step 只在最后一次同步，而不是每个 micro-step 都 all-reduce。

7.2 rank、local rank 和 world size

分布式训练里最先要理解几个概念。

7.2.1 world size

world_size 表示总进程数。通常一个 GPU 对应一个训练进程。

如果一台机器有 8 张 GPU，并且每张 GPU 一个进程：

```
world_size = 8
```

如果 2 台机器，每台 8 张 GPU：

```
world_size = 16
```

7.2.2 rank

rank 是全局进程编号，范围是：

```
0, 1, ..., world_size - 1
```

通常 rank=0 负责打印日志、保存 checkpoint、汇总指标等，避免所有进程重复写文件。

7.2.3 local rank

local_rank 是当前机器内的 GPU 编号。

单机 8 卡时，local rank 通常是：

```
0, 1, ..., 7
```

训练时通常用它设置当前进程使用哪张 GPU：

```
torch.cuda.set_device(local_rank)
device = torch.device("cuda", local_rank)
```

核心区别：

1. rank 是全局编号。
2. local_rank 是本机编号。
3. world_size 是总进程数。

7.3 process group 是什么

process group 可以理解为一组参与通信的进程。

初始化分布式环境：

```
import torch.distributed as dist
```

```
dist.init_process_group(backend="nccl")
```

常见 backend：

1. nccl：GPU 训练最常用。
2. gloo：CPU 或某些调试场景常用。

初始化后，进程之间就可以执行通信操作，例如 all-reduce、broadcast、barrier。

训练结束时通常销毁：

```
dist.destroy_process_group()
```

7.4 数据并行的核心直觉

数据并行的流程：

1. 每张 GPU 上放一份相同模型。
2. 每张 GPU 处理不同 mini-batch。
3. 每张 GPU 独立 forward 和 backward。
4. backward 后每张 GPU 得到一份梯度。
5. 通过 all-reduce 对梯度求平均。
6. 每张 GPU 用相同平均梯度更新参数。

这样更新后，每张 GPU 的模型参数仍然保持一致。

如果没有梯度同步，每张 GPU 会基于不同数据更新出不同模型，训练就不再是一个统一模型。

7.5 all-reduce 是什么

all-reduce 是分布式训练中最核心的通信操作之一。

假设有 4 张 GPU，每张 GPU 上某个参数的梯度分别是：

g_0, g_1, g_2, g_3

all-reduce sum 后，每张 GPU 都得到：

$g_0 + g_1 + g_2 + g_3$

如果要平均梯度，就再除以 `world_size`。

DDP 内部会自动处理这个过程。用户通常不需要手写 all-reduce 梯度，但要理解它的代价：通信开销会随着参数量、网络带宽、GPU 拓扑和同步频率变化。

面试回答：

all-reduce 会把每个进程上的张量做归约，并把归约结果返回给所有进程。DDP 中每张卡 backward 后得到本地梯度，通过 all-reduce 求和或平均后，每张卡拿到一致的梯度，再执行 optimizer step，所以参数保持同步。

7.6 DDP 的基本用法

PyTorch 推荐使用 `DistributedDataParallel`，也就是 DDP。

基本结构：

```
import os
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
```

```
def setup_distributed():
    dist.init_process_group(backend="nccl")
    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)
    return local_rank
```

```
local_rank = setup_distributed()
device = torch.device("cuda", local_rank)
```

```
model = MyModel().to(device)
model = DDP(model, device_ids=[local_rank])
```

训练循环中:

```
for batch in train_loader:
    batch = move_to_device(batch, device)
    optimizer.zero_grad(set_to_none=True)
    outputs = model(*batch)
    loss = outputs["loss"]
    loss.backward()
    optimizer.step()
```

DDP 会在 backward 过程中自动同步梯度。

保存 checkpoint 时常用:

```
if dist.get_rank() == 0:
    torch.save(model.module.state_dict(), "model.pt")
```

因为 DDP 包装后的原始模型在 `model.module` 里。

7.7 torchrun 启动训练

现代 PyTorch 常用 torchrun 启动分布式训练。

单机 4 卡:

```
torchrun --nproc_per_node=4 train.py
```

torchrun 会为每个进程设置环境变量, 例如:

1. RANK
2. LOCAL_RANK
3. WORLD_SIZE

训练脚本通过这些变量初始化分布式环境。

读取示例:

```
rank = int(os.environ["RANK"])
local_rank = int(os.environ["LOCAL_RANK"])
world_size = int(os.environ["WORLD_SIZE"])
```

不要手动让多个进程抢同一张 GPU。最常见的正确方式是: 每个进程绑定一个 `local_rank` 对应的 GPU。

7.8 DistributedSampler

DDP 里每个进程应该看到不同数据子集。否则每张卡都训练同样样本, 就浪费多了卡。

使用 DistributedSampler:

```
from torch.utils.data import DataLoader, DistributedSampler
```

```
sampler = DistributedSampler(dataset, shuffle=True)
loader = DataLoader(
    dataset,
    batch_size=batch_size,
    sampler=sampler,
    num_workers=4,
)
```

注意: 使用 sampler 时通常不要再设置 `shuffle=True` 给 DataLoader, 因为 shuffle 由 sampler 管理。

每个 epoch 要调用:

```
sampler.set_epoch(epoch)
```

否则每个 epoch 的 shuffle 顺序可能一样。

完整结构：

```
for epoch in range(num_epochs):
    sampler.set_epoch(epoch)
    for batch in loader:
        ...
```

7.9 global batch size 怎么算

分布式训练里要区分：

1. 每卡 batch size。
2. world size。
3. gradient accumulation steps。
4. global batch size。

公式：

$$\text{global_batch_size} = \text{per_device_batch_size} * \text{world_size} * \text{accumulation_steps}$$

例如：

```
per_device_batch_size = 4
world_size = 8
accumulation_steps = 2
global_batch_size = 4 * 8 * 2 = 64
```

这会影响学习率、warmup steps、训练步数和日志统计。

常见错误：从单卡切到 8 卡后，没有意识到 global batch size 变大，导致优化行为变化。

7.10 DDP 和 DataParallel 的区别

PyTorch 早期有 DataParallel，但现在训练更推荐 DDP。

区别：

1. DataParallel 通常单进程多线程，主卡负担重。
2. DDP 是多进程，每个 GPU 一个进程，性能和扩展性更好。
3. DDP 梯度同步更高效。
4. DDP 更适合多机多卡。

面试中可以简单说：真实训练优先用 DDP，不建议用 DataParallel 做严肃多卡训练。

7.11 DDP 的通信时机

DDP 不是等所有 backward 完成后才一次性同步全部梯度。它会把参数分成 bucket，并在某些梯度 ready 后触发通信。

这样可以让通信和反向计算重叠：

1. 后面层的梯度先算出来。
2. DDP 开始同步这些梯度 bucket。
3. 同时 backward 继续计算前面层梯度。

这叫 communication overlap。

理解这个机制有助于解释：

1. 为什么 DDP 比朴素手写同步更高效。
2. 为什么参数使用路径不一致会导致卡住。

3. 为什么梯度同步与 autograd hook 有关。

7.12 梯度累积下的 no_sync

梯度累积时，如果每个 micro-step 都同步梯度，会浪费通信。

DDP 提供 no_sync()：

```
for step, batch in enumerate(loader):
    should_sync = (step + 1) % accum_steps == 0

    if should_sync:
        context = nullcontext()
    else:
        context = model.no_sync()

    with context:
        outputs = model(**batch)
        loss = outputs["loss"] / accum_steps
        loss.backward()

    if should_sync:
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

这里 no_sync() 会跳过当前 micro-step 的梯度同步，只在真正更新前同步一次。

需要导入：

```
from contextlib import nullcontext
```

7.13 FSDP 的基本直觉

DDP 每张卡都有一份完整模型参数、梯度和优化器状态。模型变大后，单卡显存仍然可能不够。

FSDP 的直觉是把参数、梯度和优化器状态分片到不同 GPU 上。

可以这样理解：

1. DDP：每张卡都有完整模型副本。
2. FSDP：每张卡只长期持有一部分参数状态，需要计算某层时再临时 gather。

优点：

1. 降低单卡显存。
2. 支持更大模型。

代价：

1. 通信更复杂。
2. 对模块边界更敏感。
3. checkpoint、调试和性能优化更复杂。

本章只要求建立直觉：DDP 主要解决速度扩展，FSDP 进一步解决大模型显存分片问题。

7.14 分布式 checkpoint

DDP 场景通常只让 rank=0 保存：

```
if dist.get_rank() == 0:
    torch.save({
        "model": model.module.state_dict(),
```

```
    "optimizer": optimizer.state_dict(),
    "step": global_step,
}, "checkpoint.pt")
```

原因：所有 rank 的模型参数同步后相同，没必要每个 rank 都写一份。

但 FSDP 或分片训练场景下，checkpoint 可能是 sharded checkpoint，保存和加载要复杂很多。入门阶段先掌握 DDP 的保存方式。

注意：只让 rank 0 写日志和 checkpoint，避免多个进程同时写同一个文件。

7.15 分布式训练常见 bug

7.15.1 程序卡住

常见原因：

1. 某些 rank 没走到同一个 collective 操作。
2. 条件分支导致不同 rank 执行路径不同。
3. 某些参数在部分 rank 没参与 forward。
4. 数据集长度或 batch 数在不同 rank 不一致。
5. 网络或 NCCL 通信异常。

7.15.2 某些参数没有梯度

常见原因：

1. 模型分支没被当前 batch 使用。
2. 参数 `requires_grad=False`。
3. loss 不依赖该参数。
4. 误用了 `detach()`。

DDP 中如果存在未使用参数，可以尝试设置：

```
DDP(model, device_ids=[local_rank], find_unused_parameters=True)
```

但这通常有额外开销，不应作为长期默认修复。更好的方式是理解为什么参数未使用。

7.15.3 每张卡数据重复

常见原因：

1. 没用 `DistributedSampler`。
2. sampler 配置错。
3. iterable dataset 没按 rank 切分。

7.15.4 只有 rank 0 正常，其他 rank 报错

可能是：

1. 只有 rank 0 创建了某些文件或目录。
2. 数据路径在不同机器不可见。
3. 某些初始化逻辑只在 rank 0 执行，但其他 rank 也依赖结果。

7.16 分布式 debug 清单

排查分布式问题时，先打印基础信息：

```
print(
    "rank", dist.get_rank(),
    "world_size", dist.get_world_size(),
```

```
    "device", torch.cuda.current_device()),  
)
```

检查:

1. 每个 rank 是否绑定不同 GPU。
2. 每个 rank 的 batch shape 是否合理。
3. 每个 rank 是否都进入训练循环。
4. 每个 rank 的 loss 是否有限。
5. 是否所有 rank 都参与同样 collective。
6. sampler 是否 set_epoch。
7. 只有 rank 0 是否写 checkpoint。

有时可以加 barrier 定位卡住位置:

```
dist.barrier()
```

但不要滥用 barrier。barrier 可以帮助定位同步点, 也可能掩盖真实性能问题。

7.17 一个最小 DDP 训练骨架

```
import os  
import torch  
import torch.distributed as dist  
from torch.nn.parallel import DistributedDataParallel as DDP  
from torch.utils.data import DataLoader, DistributedSampler  
  
def setup():  
    dist.init_process_group(backend="nccl")  
    local_rank = int(os.environ["LOCAL_RANK"])  
    torch.cuda.set_device(local_rank)  
    return local_rank  
  
def cleanup():  
    dist.destroy_process_group()  
  
def main():  
    local_rank = setup()  
    device = torch.device("cuda", local_rank)  
  
    dataset = MyDataset()  
    sampler = DistributedSampler(dataset, shuffle=True)  
    loader = DataLoader(dataset, batch_size=8, sampler=sampler, num_workers=4)  
  
    model = MyModel().to(device)  
    model = DDP(model, device_ids=[local_rank])  
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)  
  
    for epoch in range(3):  
        sampler.set_epoch(epoch)  
        model.train()  
  
        for batch in loader:  
            batch = move_to_device(batch, device)  
            optimizer.zero_grad(set_to_none=True)
```

```

        outputs = model(**batch)
        loss = outputs["loss"]
        loss.backward()
        optimizer.step()

    if dist.get_rank() == 0:
        torch.save(model.module.state_dict(), "model.pt")

cleanup()

if __name__ == "__main__":
    main()

```

启动:

```
torchrun --nproc_per_node=4 train.py
```

这个骨架包含了 DDP 的最小关键点: 初始化、local rank、DistributedSampler、DDP 包装、rank 0 保存和 cleanup。

7.18 最小可运行分布式训练审计 demo

真实 DDP 需要多进程和 GPU 环境, 下面这个 demo 用纯 Python 模拟分布式训练中最重要的检查点: rank 数据切分、global batch size、all-reduce 平均梯度、同步前后参数是否一致、no_sync() 减少通信次数、collective 是否所有 rank 都参与, 以及是否只有 rank 0 写 checkpoint。

```
from itertools import combinations
```

```

def distributed_indices(num_samples, world_size, rank, epoch=0):
    indices = list(range(num_samples))
    offset = (epoch * 3) % num_samples
    indices = indices[offset:] + indices[:offset]
    return indices[rank::world_size]

def find_overlaps(parts):
    overlaps = []
    for left, right in combinations(parts, 2):
        common = sorted(set(parts[left]) & set(parts[right]))
        if common:
            overlaps.append((left, right, common))
    return overlaps

def audit_collective(traces, collective_name):
    missing = [rank for rank, events in traces.items() if collective_name not in events]
    return {
        "ok": not missing,
        "missing": missing,
    }

world_size = 4
per_device_batch_size = 2
accum_steps = 3

```

```

dataset_size = 16

epoch0_parts = {
    rank: distributed_indices(dataset_size, world_size, rank, epoch=0)
    for rank in range(world_size)
}
epoch1_parts = {
    rank: distributed_indices(dataset_size, world_size, rank, epoch=1)
    for rank in range(world_size)
}

all_epoch0_indices = sorted(index for part in epoch0_parts.values() for index in part)
sampler_overlap = find_overlaps(epoch0_parts)
global_batch_size = per_device_batch_size * world_size * accum_steps

rank_grads = [0.2, 0.4, -0.1, 0.3]
avg_grad = sum(rank_grads) / world_size
theta0 = 1.0
lr = 0.5
synced_params = [round(theta0 - lr * avg_grad, 3) for _ in rank_grads]
unsynced_params = [theta0 - lr * grad for grad in rank_grads]
unsynced_param_drift = round(max(unsynced_params) - min(unsynced_params), 3)

micro_steps = 6
sync_without_no_sync = micro_steps
sync_with_no_sync = sum(
    1 for step in range(micro_steps)
    if (step + 1) % accum_steps == 0
)

ok_trace = {
    rank: ["forward", "backward", "all_reduce", "optimizer_step"]
    for rank in range(world_size)
}
bad_trace = {
    0: ["forward", "backward", "all_reduce", "optimizer_step"],
    1: ["forward", "backward", "all_reduce", "optimizer_step"],
    2: ["forward", "backward", "optimizer_step"],
    3: ["forward", "backward", "all_reduce", "optimizer_step"],
}
ok_collective = audit_collective(ok_trace, "all_reduce")
bad_collective = audit_collective(bad_trace, "all_reduce")

rank0_writers = [rank for rank in range(world_size) if rank == 0]
checks = {
    "sampler_no_overlap": sampler_overlap == [],
    "sampler_full_coverage": all_epoch0_indices == list(range(dataset_size)),
    "sampler_epoch_changed": epoch0_parts[0] != epoch1_parts[0],
    "synced_params_equal": len(set(synced_params)) == 1,
    "unsynced_would_drift": unsynced_param_drift > 0,
    "no_sync_reduces_calls": sync_with_no_sync < sync_without_no_sync,
    "detects_missing_collective": not bad_collective["ok"],
    "only_rank0_writes": rank0_writers == [0],
}

```

```

print("global_batch_size=", global_batch_size)
print("sampler_epoch0=", epoch0_parts)
print("sampler_overlap=", sampler_overlap)
print("sampler_coverage=", len(all_epoch0_indices))
print("sampler_epoch_changed=", checks["sampler_epoch_changed"])
print("all_reduce_avg_grad=", round(avg_grad, 3))
print("synced_params=", synced_params)
print("unsynced_param_drift=", unsynced_param_drift)
print("no_sync_calls=", {
    "without_no_sync": sync_without_no_sync,
    "with_no_sync": sync_with_no_sync,
})
print("collective_audit=", {
    "ok": ok_collective["ok"],
    "bad": bad_collective["ok"],
    "bad_missing": bad_collective["missing"],
})
print("rank0_writers=", rank0_writers)
print("checks=", checks)

```

期望输出：

```

global_batch_size= 24
sampler_epoch0= {0: [0, 4, 8, 12], 1: [1, 5, 9, 13], 2: [2, 6, 10, 14], 3: [3, 7, 11, 15]}
sampler_overlap= []
sampler_coverage= 16
sampler_epoch_changed= True
all_reduce_avg_grad= 0.2
synced_params= [0.9, 0.9, 0.9, 0.9]
unsynced_param_drift= 0.25
no_sync_calls= {'without_no_sync': 6, 'with_no_sync': 2}
collective_audit= {'ok': True, 'bad': False, 'bad_missing': [2]}
rank0_writers= [0]
checks= {'sampler_no_overlap': True, 'sampler_full_coverage': True, 'sampler_epoch_changed': True, 'synced_params_equal': True}

```

这段 demo 要抓住几个点：

1. `sampler_overlap=[]` 说明同一个 epoch 内不同 rank 没有重复读样本。
2. `global_batch_size=24` 说明多卡和梯度累积会改变真实 batch。
3. all-reduce 平均后所有 rank 的参数更新一致；如果各自用本地梯度更新，`unsynced_param_drift` 会大于 0。
4. `no_sync()` 的直觉是多个 micro-step 只同步一次，demo 中通信次数从 6 次降到 2 次。
5. collective audit 能发现 rank 2 没有进入 all_reduce，这类不一致是分布式训练卡住的常见原因。
6. checkpoint 写入者只有 rank 0，避免多进程同时写同一个文件。

7.19 面试官会怎么问

问题一：DDP 的工作原理是什么？

回答模板：

DDP 是数据并行。每个进程通常绑定一张 GPU，并持有一份模型副本。每个进程处理不同 mini-batch，backward 后通过 all-reduce 同步梯度，保证每张卡拿到一致的平均梯度，然后各自执行 optimizer step，所以参数保持一致。

问题二：rank、local rank、world size 分别是什么？

回答模板：

world size 是总训练进程数，rank 是全局进程编号，local rank 是当前机器内的进程或 GPU 编号。通常用 local rank 绑定当前 GPU。

问题三：为什么 DDP 要用 DistributedSampler?

回答模板：

因为每个 rank 应该处理不同数据子集，否则多张卡会重复训练同样数据。DistributedSampler 会按 rank 和 world size 切分数据，

问题四：DDP 和 FSDP 有什么区别?

回答模板：

DDP 每张卡通常持有完整模型副本，主要通过数据并行提升吞吐；FSDP 会把参数、梯度和优化器状态分片到多张卡上，降低单卡显存。

问题五：分布式训练卡住怎么排查?

回答模板：

我会先确认每个 rank 是否启动、绑定正确 GPU、进入同一训练阶段，再检查是否所有 rank 都参与同样的 collective 操作，数据 ba

7.20 常见误区

1. 以为多卡就是自动变快。通信开销、数据加载和 batch 配置都会影响加速比。
2. 以为 DDP 会自动处理数据切分。数据切分要用 DistributedSampler 或自己实现。
3. 以为每张卡 batch size 不变时训练完全等价。global batch size 变大后优化行为会变化。
4. 以为所有 rank 都可以写同一个 checkpoint。通常只让 rank 0 写。
5. 以为 find_unused_parameters=True 是万能修复。它有开销，应该理解参数为什么没被使用。
6. 以为 FSDP 只是更快的 DDP。FSDP 主要解决显存分片问题，复杂度也更高。

7.21 小练习

1. 写出 rank、local_rank、world_size 的区别。
2. 画出 4 卡 DDP 的 forward、backward、all-reduce 和 step 流程。
3. 用公式计算 per_device_batch_size=2、world_size=8、accum_steps=4 时的 global batch size。
4. 写一个最小 DDP 初始化函数，读取 LOCAL_RANK 并设置 GPU。
5. 给 DataLoader 加上 DistributedSampler，并在 epoch 开头调用 set_epoch。
6. 解释为什么梯度累积时可以用 no_sync() 减少通信。
7. 设计一个分布式训练卡住时的排查 checklist。

7.22 本章总结

分布式训练的入门主线是：多进程、多 GPU、每个进程一个 rank，DDP 负责梯度同步，DistributedSampler 负责数据切分，torchrun 负责启动和环境变量设置。

需要记住：

1. rank 是全局进程编号，local_rank 是本地 GPU 编号，world_size 是总进程数。
2. DDP 是最常见的数据并行方式，每张卡处理不同数据，backward 后 all-reduce 梯度。
3. DistributedSampler 防止多卡重复读同样数据。
4. global batch size 等于每卡 batch、卡数和梯度累积步数的乘积。
5. 只有 rank 0 通常负责日志和 checkpoint。
6. FSDP 进一步把参数、梯度和优化器状态分片，用于更大模型。
7. 分布式 debug 要关注 rank 是否一致进入同步点、数据是否切分正确、通信是否卡住。

下一章会进入 debug 与 profiling，系统讲如何排查 shape、dtype、device、NaN、OOM、性能瓶颈和训练异常。

第八章：Debug 与 Profiling

深度学习工程能力的一大分水岭，不是能不能把模型跑起来，而是模型出问题能不能快速定位。真实训练中常见问题包括 shape 不对、dtype 不对、device 不一致、loss 不下降、梯度为 None、NaN、OOM、DataLoader 太慢、GPU 利用率低、吞吐不稳定、验证和训练结果不一致、分布式卡住。Debug 和 profiling 的目标，就是把“感觉模型有问题”变成“定位到具体层、具体张量、具体 batch 或具体系统瓶颈”。

本章不追求列出所有 PyTorch 调试工具，而是建立一套优先级清楚的排查方法：先确认数据和 shape，再确认 loss 和梯度，再确认数值稳定，再看显存，再看性能瓶颈。很多时候，系统化排查比背 API 更重要。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前，已核对 PyTorch 官方 `torch.autograd.detect_anomaly / set_detect_anomaly`、CUDA semantics、CUDA memory management、`torch.cuda.memory_allocated / memory_reserved`、`torch.cuda.synchronize`、`torch.profiler`、PyTorch profiler recipe、Performance Tuning Guide 和 DataLoader 相关文档口径。

本章聚焦 PyTorch 训练调试和性能定位中最常见的问题：shape / dtype / device、loss 与 label 对齐、NaN / Inf、梯度为 None 或异常、forward hook、OOM、DataLoader 瓶颈、CPU-GPU 拷贝、CUDA 异步计时、`torch.profiler` 基础用法、分布式日志和最小可复现 bug。

本章不展开 Nsight Systems / Nsight Compute、CUDA kernel 级优化、`torch.compile` / Inductor profiler、NCCL 深度 profiling、生产级 observability 平台、分布式 trace 聚合或复杂性能建模。这些内容会在 AI Infra、推理框架和性能优化章节中继续展开。

8.1 Debug 的基本原则

遇到训练问题时，不要一上来改模型结构或调学习率。更稳的顺序是：

1. 先复现。
2. 再缩小问题范围。
3. 打印关键张量元信息。
4. 验证数据和 label。
5. 验证 loss 是否合理。
6. 验证梯度是否存在且有限。
7. 再考虑优化器、学习率、混合精度和分布式。

一个好的 debug 习惯是：每次只改一个变量，并记录修改前后的现象。否则你可能让问题暂时消失，但不知道真正原因。

面试回答：

我 debug 训练问题时会先缩小范围：确认数据样本和 label 是否正确，再检查 tensor 的 shape、dtype、device，然后看 loss 是否

8.1.1 关键公式与 debug/profiling 速查

第一，调试一个 tensor 时不要只看数值，还要看元信息：

$$m(X) = (\mathrm{shape}(X), \mathrm{dtype}(X), \mathrm{device}(X), \mathrm{requires_grad}(X), \mathrm{stride}(X))$$

shape / dtype / device 错误往往比模型结构错误更常见。

第二，causal LM loss 展平前后必须满足：

$$N_{\{\mathrm{logit}\}} = B(T-1), \quad N_{\{\mathrm{label}\}} = B(T-1)$$

也就是 `shift_logits.reshape(-1, V)` 的第 0 维必须和 `shift_labels.reshape(-1)` 的长度一致。

第三，非有限值检查可以抽象为：

$$I_{\{\mathrm{finite}\}}(X) = \prod_i I(|x_i| < \infty)$$

如果 logits、loss、grad 或参数中任一关键张量 $I_{\text{finite}}=0$ ，应先定位非有限值第一次出现的位置，再调学习率或改模型。

第四，梯度检查常看每个参数的梯度范数：

$G_l = \|\nabla_{\theta_l} L\|_2$

$G_l = \text{None}$ 、 $G_l = 0$ 、 G_l 非有限或异常大，分别对应断图、无信号、数值错误或梯度爆炸等不同问题。

第五，OOM 要区分峰值过高和持续增长：

$\Delta M_t = M_t - M_{t-1}$

如果每步结束后 M_t 持续上升，常见原因是保存了带计算图的 tensor、验证没关梯度或评估缓存过多输出；如果只有峰值超限，常见方向是 batch、sequence length、activation、optimizer state 和临时 workspace。

第六，吞吐可以粗略写成：

$R_{\{\text{tok/s}\}} = \frac{B}{T} \{t_{\{\text{step}\}}\}$

其中 B 是 batch size， T 是序列长度， t_{step} 是 step 耗时。优化前要先区分数据加载、拷贝、forward、backward、optimizer 和通信分别占多少。

第七，CUDA 计时需要同步：

$t_{\{\text{cuda}\}} = t_{\{\text{end after sync}\}} - t_{\{\text{start after sync}\}}$

因为 CUDA kernel 默认异步执行，不同步时 Python 侧计时可能只测到 launch 开销。

第八，profiler 的核心输出是按算子聚合的时间和内存：

$T_{\{\text{op}\}} = \sum_k t_{\{\text{op}\},k}, \quad \text{quad}$

$M_{\{\text{op}\}} = \sum_k m_{\{\text{op}\},k}$

真正要看的是瓶颈归因：DataLoader、CPU-GPU copy、forward、backward、optimizer、通信，还是少数高频小算子。

8.2 打印 tensor 元信息

很多 bug 一打印 tensor 元信息就能定位。

```
import torch
```

```
def debug_tensor(name, x):
    if not torch.is_tensor(x):
        print(name, type(x))
        return
    print(
        name,
        "shape=", tuple(x.shape),
        "dtype=", x.dtype,
        "device=", x.device,
        "requires_grad=", x.requires_grad,
        "is_contiguous=", x.is_contiguous(),
    )
```

使用：

```
debug_tensor("input_ids", batch["input_ids"])
debug_tensor("labels", batch["labels"])
debug_tensor("logits", logits)
debug_tensor("loss", loss)
```

最先检查：

1. shape 是否符合预期。
2. dtype 是否符合模块要求。
3. device 是否一致。
4. loss 是否是标量。
5. logits 和 labels 是否能对齐。

8.3 Shape debug

Shape 错误是最常见的 PyTorch bug。

语言模型里常见 shape:

```
input_ids: [B, T]
labels: [B, T]
hidden_states: [B, T, d]
logits: [B, T, V]
```

计算 loss 前:

```
shift_logits = logits[:, :-1, :]
shift_labels = labels[:, 1:]
```

```
debug_tensor("shift_logits", shift_logits)
debug_tensor("shift_labels", shift_labels)
```

然后展平:

```
loss = torch.nn.functional.cross_entropy(
    shift_logits.reshape(-1, shift_logits.size(-1)),
    shift_labels.reshape(-1),
    ignore_index=-100,
)
```

常见错误:

1. logits 是 [B, T, V], labels 是 [B, T], 但忘记展平。
2. 展平后 logits 的第 0 维和 labels 的第 0 维不一致。
3. attention scores 期望 [B, H, T, T], mask 写成了 [B, T, 1]。
4. view 用在非 contiguous tensor 上报错。

调试建议: 把每个模块的输入输出 shape 都打印一次, 直到发现第一个不符合预期的位置。

8.4 Dtype debug

Dtype 错误常见于 embedding、loss、mask 和混合精度。

常见规则:

1. input_ids 通常是 torch.long。
2. labels 通常是 torch.long。
3. logits 通常是浮点类型。
4. attention mask 可以是 bool, 也可以转换成加性浮点 mask。
5. 模型参数可能是 fp32、fp16 或 bf16。

典型报错:

```
expected scalar type Long but found Float
```

通常表示你把 float tensor 传给了需要索引的模块, 例如 nn.Embedding。

检查代码:

```
for key, value in batch.items():
    debug_tensor(key, value)
```

混合精度下，某些操作可能因为 dtype 不一致报错。优先确认：

1. 模型参数 dtype。
2. 输入 dtype。
3. autocast 作用域是否正确。
4. loss 是否在合理 dtype 下计算。

8.5 Device debug

Device mismatch 是另一个高频错误。

典型报错：

Expected all tensors to be on the same device

排查：

```
print("model device:", next(model.parameters()).device)
for key, value in batch.items():
    if torch.is_tensor(value):
        print(key, value.device)
```

常见原因：

1. 模型在 GPU，batch 还在 CPU。
2. 某个 buffer 没有注册，model.to(device) 没移动它。
3. 在 forward 中新建 tensor 时没指定 device。
4. loss 里用到的 class weight 还在 CPU。

错误示例：

```
mask = torch.ones(seq_len, seq_len) # 默认在 CPU
scores = scores.masked_fill(mask == 0, -1e9)
```

更安全：

```
mask = torch.ones(seq_len, seq_len, device=scores.device)
```

或者注册为 buffer。

8.6 NaN 和 Inf debug

NaN 是训练中最危险的问题之一。先判断 NaN 出现在：

1. 输入数据。
2. logits。
3. loss。
4. 梯度。
5. 参数。

检查函数：

```
def check_finite(name, x):
    if torch.is_tensor(x) and not torch.isfinite(x).all():
        print("non-finite:", name)
        print("nan:", torch.isnan(x).any().item())
        print("inf:", torch.isinf(x).any().item())
```

训练中使用：

```
check_finite("logits", logits)
check_finite("loss", loss)
```

检查梯度:

```
for name, param in model.named_parameters():
    if param.grad is not None:
        check_finite("grad " + name, param.grad)
```

常见原因:

1. 学习率太大。
2. fp16 溢出。
3. softmax 前 logits 过大。
4. mask 产生整行 -inf。
5. loss 分母为 0。
6. 数据中有异常值。
7. 梯度爆炸。

8.7 anomaly detection

PyTorch 提供 anomaly detection, 能帮助定位 backward 报错来源。

```
torch.autograd.set_detect_anomaly(True)
```

或者:

```
with torch.autograd.detect_anomaly():
    loss.backward()
```

它会让 PyTorch 尝试报告导致反向异常的前向位置。

注意:

1. 它会明显降低速度。
2. 只适合 debug, 不适合长期训练。
3. 它不一定能定位所有 NaN 根因, 但对 in-place 和 backward 异常很有帮助。

8.8 梯度 debug

loss 不下降时, 先确认梯度是否存在。

```
for name, param in model.named_parameters():
    if param.requires_grad:
        if param.grad is None:
            print("grad is None:", name)
        else:
            print(name, param.grad.norm().item())
```

常见问题:

1. 参数没有进入 optimizer。
2. 参数 requires_grad=False。
3. loss 不依赖该参数。
4. 中间用了 detach()。
5. forward 分支没有用到该模块。

梯度全 0 可能说明:

1. 激活饱和。
2. mask 把 loss 全忽略了。
3. loss 写错。
4. 学习率不是原因, 梯度本身就没有信号。

梯度非常大可能说明：

1. 学习率过大。
2. 初始化不稳定。
3. 数据异常。
4. loss scale 或混合精度异常。
5. 需要梯度裁剪。

8.9 hook 调试中间激活

如果怀疑某一层输出异常，可以用 forward hook。

```
activations = {}
```

```
def save_activation(name):
    def hook(module, inputs, output):
        if torch.is_tensor(output):
            activations[name] = output.detach().cpu()
    return hook
```

```
handle = model.layers[0].register_forward_hook(save_activation("layer0"))
```

运行一次 forward 后：

```
print(activations["layer0"].shape)
handle.remove()
```

注意：

1. 保存激活时要 detach()。
2. 用完 hook 要 remove()。
3. 不要长期保存大激活。
4. 分布式和 compile 场景下 hook 可能增加复杂度。

8.10 OOM debug

OOM 需要区分是“瞬间峰值太高”还是“显存持续增长”。

打印显存：

```
if torch.cuda.is_available():
    print("allocated", torch.cuda.memory_allocated() / 1024**2)
    print("reserved", torch.cuda.memory_reserved() / 1024**2)
```

常见原因：

1. batch size 太大。
2. sequence length 太长。
3. 没开混合精度。
4. 没开 activation checkpointing。
5. 保存了带图 tensor。
6. 验证时没关梯度。
7. DataLoader 或评估代码缓存过多输出。

排查顺序：

1. 减小 batch size。
2. 减小 sequence length。
3. 关掉额外输出，例如 hidden states 和 attentions。

4. 检查是否保存了 loss、logits、activation tensor。
5. 开启 AMP 和 checkpointing。

8.11 DataLoader 性能 debug

GPU 利用率低，不一定是模型慢，可能是 DataLoader 喂不饱 GPU。

现象：

1. GPU 利用率周期性掉到很低。
2. 每步训练时间波动很大。
3. CPU 占用很高。
4. 数据读取线程成为瓶颈。

排查：

1. 增加 num_workers。
2. 使用 pin_memory=True。
3. 减少 __getitem__ 中的重预处理。
4. 把可离线处理的 tokenization 提前做掉。
5. 检查远程存储或磁盘 IO。

简单计时：

```
import time

start = time.time()
for step, batch in enumerate(loader):
    if step == 100:
        break
print("data time", time.time() - start)
```

如果纯 DataLoader 迭代都很慢，瓶颈不在模型。

8.12 性能 profiling 的基本思路

性能优化前要先 profiling。不要凭感觉优化。

先区分瓶颈：

1. DataLoader 慢。
2. CPU 到 GPU 拷贝慢。
3. forward 慢。
4. backward 慢。
5. optimizer step 慢。
6. 分布式通信慢。

最简单的计时方式：

```
torch.cuda.synchronize()
start = time.time()

outputs = model(**batch)
loss = outputs["loss"]
loss.backward()
optimizer.step()

torch.cuda.synchronize()
elapsed = time.time() - start
```

为什么要 torch.cuda.synchronize()？因为 CUDA 操作默认异步，如果不同步，Python 计时可能不准确。

8.13 torch.profiler 入门

PyTorch 提供 torch.profiler 用于分析 CPU/GPU 时间。

简化示例：

```
import torch.profiler

with torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
    ],
    record_shapes=True,
    profile_memory=True,
) as prof:
    for step, batch in enumerate(train_loader):
        if step >= 5:
            break
        train_step(batch)
        prof.step()

print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=20))
```

你可以看到：

1. 哪些算子耗时最多。
2. CPU 和 CUDA 时间分布。
3. shape 信息。
4. 显存分配情况。

profiler 本身有开销，不要在完整训练中一直开。

8.14 常见性能瓶颈

8.14.1 小 batch 导致 GPU 利用率低

小 batch 下 GPU 可能吃不满。解决思路：

1. 增大 batch。
2. 梯度累积。
3. 合并小 kernel。
4. 使用更高效算子。

8.14.2 DataLoader 慢

解决思路：

1. 预处理离线化。
2. 调整 num_workers。
3. 使用 pinned memory。
4. 减少 Python 层复杂逻辑。

8.14.3 CPU-GPU 拷贝慢

解决思路：

1. pin_memory=True。
2. .to(device, non_blocking=True)。
3. 避免每步创建大量小 tensor。

8.14.4 通信慢

分布式中常见。解决思路：

1. 增大计算通信比。
2. 梯度累积配合 `no_sync()`。
3. 检查网络和 NCCL。
4. 减少不必要同步。

8.15 训练结果异常的系统排查

8.15.1 loss 不下降

排查：

1. 数据和 labels 是否正确。
2. loss 是否对齐任务。
3. 参数是否进入 optimizer。
4. 梯度是否存在。
5. 学习率是否合理。
6. mask 是否把有效 token 忽略。
7. 模型是否处于 train 模式。

8.15.2 train loss 降, val loss 不降

可能原因：

1. 过拟合。
2. 训练集和验证集分布不同。
3. 验证代码有 bug。
4. train/eval 模式切换错误。
5. 数据泄漏或评估指标错误。

8.15.3 训练很快但效果很差

可能原因：

1. loss mask 错误。
2. labels 全是 -100 或大部分被忽略。
3. 数据被截断得太严重。
4. tokenizer 或 chat template 不一致。
5. 学习率或 batch 配置不合理。

8.16 分布式 debug

分布式问题最重要的是确认每个 rank 的状态。

打印：

```
print(
    "rank", dist.get_rank(),
    "world_size", dist.get_world_size(),
    "device", torch.cuda.current_device(),
)
```

常见问题：

1. 某些 rank 没进入同一个 collective。
2. 某个 rank 数据加载失败。
3. 只有 rank 0 创建了文件, 其他 rank 读不到。

4. 每个 rank batch 数不一致。
5. 日志混在一起看不清。

建议：

1. 日志带 rank。
2. 只让 rank 0 保存 checkpoint。
3. 用小数据、小 batch 先复现。
4. 必要时用 `dist.barrier()` 定位卡点。

8.17 一个实用 debug checklist

遇到训练异常，可以按这个顺序查：

1. 打印一个原始样本。
2. 打印 collate 后 batch。
3. 检查 input、labels、mask 的 shape 和 dtype。
4. 跑一个 batch 的 forward。
5. 检查 logits shape。
6. 检查 loss 是否有限。
7. backward 后检查梯度是否存在。
8. 检查 optimizer 是否包含目标参数。
9. 检查学习率和 scheduler。
10. 检查 train/eval/no_grad 使用是否正确。
11. 检查显存是否持续增长。
12. 如果慢，再做 profiler。

这个顺序的核心是从数据到模型、从正确性到性能，不要先优化性能再修正正确性。

8.18 一个最小可复现 bug 的写法

当你遇到复杂 bug，最有效的方法是构造最小复现。

步骤：

1. 固定随机种子。
2. 使用极小模型。
3. 使用 1-2 个 batch 的假数据。
4. 去掉分布式和混合精度。
5. 保留能复现 bug 的最少代码。

示例：

```
torch.manual_seed(0)
```

```
model = TinyModel()
batch = {
    "input_ids": torch.randint(0, 100, (2, 8)),
    "labels": torch.randint(0, 100, (2, 8)),
}
```

```
outputs = model(**batch)
loss = outputs["loss"]
loss.backward()
```

如果最小例子不复现，说明问题可能来自数据、分布式、混合精度、DataLoader 或训练循环外围逻辑。

8.19 最小可运行 debug/profiling 审计 demo

下面这个 demo 在 CPU 上即可运行。它把本章的核心检查点串起来：tensor 元信息、causal LM shift loss 对齐、finite 检查、forward hook、梯度范数、optimizer 参数覆盖、简单 step 计时，以及可选的 torch.profiler 入口。

当前机器如果 CUDA driver 不匹配，torch.profiler 可能产生底层驱动探测日志。为了让章节 demo 输出稳定，下面默认不启动 profiler；需要真实 profiler 时，把环境变量 RUN_TORCH_PROFILER=1 打开即可。

```
import os
import time
import warnings

os.environ.setdefault("CUDA_VISIBLE_DEVICES", "")
warnings.filterwarnings("ignore", message="CUDA initialization.*")

import torch
import torch.nn.functional as F
from torch import nn

def debug_tensor(name, x):
    return {
        "name": name,
        "shape": tuple(x.shape),
        "dtype": str(x.dtype).replace("torch.", ""),
        "device": x.device.type,
        "requires_grad": bool(x.requires_grad),
        "contiguous": bool(x.is_contiguous()),
    }

def finite_status(name, x):
    finite = torch.isfinite(x)
    return {
        "name": name,
        "all_finite": bool(finite.all()),
        "nan": bool(torch.isnan(x).any()),
        "inf": bool(torch.isinf(x).any()),
    }

class TinyLM(nn.Module):
    def __init__(self, vocab_size=16, hidden_size=8):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.norm = nn.LayerNorm(hidden_size)
        self.head = nn.Linear(hidden_size, vocab_size)

    def forward(self, input_ids):
        hidden = self.norm(self.embed(input_ids))
        return self.head(hidden)

torch.manual_seed(0)
model = TinyLM()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```

```

batch = {
    "input_ids": torch.tensor([[1, 2, 3, 4], [2, 3, 4, 0]], dtype=torch.long),
    "labels": torch.tensor([[1, 2, 3, 4], [2, 3, 4, -100]], dtype=torch.long),
}
activation_shapes = {}

def hook(module, inputs, output):
    activation_shapes["norm"] = tuple(output.shape)

handle = model.norm.register_forward_hook(hook)
optimizer.zero_grad(set_to_none=True)
logits = model(batch["input_ids"])
shift_logits = logits[:, :-1, :]
shift_labels = batch["labels"][:, 1:]
loss = F.cross_entropy(
    shift_logits.reshape(-1, shift_logits.size(-1)),
    shift_labels.reshape(-1),
    ignore_index=-100,
)
loss.backward()
grad_norms = {
    name: round(float(param.grad.norm()), 4)
    for name, param in model.named_parameters()
    if param.grad is not None
}
optimizer_param_ids = {id(param) for group in optimizer.param_groups for param in group["params"]}
all_trainable_in_optimizer = all(
    id(param) in optimizer_param_ids
    for param in model.parameters()
    if param.requires_grad
)
handle.remove()

bad_logits = torch.tensor([[0.0, float("inf")], [float("nan"), 1.0]])
nonfinite_report = finite_status("bad_logits", bad_logits)

timing_start = time.perf_counter()
for _ in range(5):
    timed_logits = model(batch["input_ids"])
    timed_loss = timed_logits.sum()
    timed_loss.backward()
    model.zero_grad(set_to_none=True)
elapsed_ms = round((time.perf_counter() - timing_start) * 1000, 3)

profiler_enabled = os.environ.get("RUN_TORCH_PROFILER") == "1"
profiler_summary = {"enabled": profiler_enabled, "event_count_positive": False, "has_addmm": False}
if profiler_enabled:
    with torch.profiler.profile(
        activities=[torch.profiler.ProfilerActivity.CPU],
        record_shapes=True,
        profile_memory=True,
    ) as prof:
        prof_logits = model(batch["input_ids"])

```

```

        prof_loss = prof_logits.sum()
        prof_loss.backward()
    events = prof.key_averages()
    profiler_summary = {
        "enabled": True,
        "event_count_positive": len(events) > 0,
        "has_addmm": any("addmm" in event.key for event in events),
    }

valid_label_count = int((shift_labels != -100).sum())
metadata = [
    debug_tensor("input_ids", batch["input_ids"]),
    debug_tensor("logits", logits),
    debug_tensor("shift_logits", shift_logits),
    debug_tensor("shift_labels", shift_labels),
]
checks = {
    "loss_is_scalar": loss.ndim == 0,
    "shift_flat_match": shift_logits.reshape(-1, shift_logits.size(-1)).shape[0]
    == shift_labels.reshape(-1).shape[0],
    "valid_labels_exist": valid_label_count > 0,
    "loss_finite": bool(torch.isfinite(loss)),
    "grads_present": set(grad_norms)
    == {"embed.weight", "norm.weight", "norm.bias", "head.weight", "head.bias"},
    "optimizer_covers_trainable": all_trainable_in_optimizer,
    "hook_removed": len(model.norm._forward_hooks) == 0,
    "detects_nonfinite": not nonfinite_report["all_finite"],
    "timing_positive": elapsed_ms > 0,
}

print("metadata=", metadata)
print("valid_label_count=", valid_label_count)
print("loss=", round(float(loss.detach()), 4))
print("activation_shapes=", activation_shapes)
print("grad_norms=", grad_norms)
print("nonfinite_report=", nonfinite_report)
print("elapsed_ms_positive=", elapsed_ms > 0)
print("profiler_summary=", profiler_summary)
print("checks=", checks)

```

期望输出:

```

metadata= [{'name': 'input_ids', 'shape': (2, 4), 'dtype': 'int64', 'device': 'cpu', 'requires_grad': False, 'contiguous': True}]
valid_label_count= 5
loss= 3.1516
activation_shapes= {'norm': (2, 4, 8)}
grad_norms= {'embed.weight': 0.2852, 'norm.weight': 0.2617, 'norm.bias': 0.3396, 'head.weight': 1.7006, 'head.bias': 0.5000}
nonfinite_report= {'name': 'bad_logits', 'all_finite': False, 'nan': True, 'inf': True}
elapsed_ms_positive= True
profiler_summary= {'enabled': False, 'event_count_positive': False, 'has_addmm': False}
checks= {'loss_is_scalar': True, 'shift_flat_match': True, 'valid_labels_exist': True, 'loss_finite': True, 'grads_present': True}

```

这段输出要检查几件事:

1. shift_logits 和 shift_labels 第一维展平后能对齐, 并且有效 label 数不是 0。
2. loss_finite=True 说明当前 batch 的 loss 没有 NaN / Inf。
3. grad_norms 覆盖了所有可训练参数, 说明 loss 确实连到了参数。

4. `nonfinite_report` 能发现手工构造的 NaN / Inf, 说明 `finite` 检查函数有效。
5. `hook_removed=True` 避免 forward hook 长期挂在模型上。
6. `elapsed_ms_positive=True` 说明至少可以做基础 step timing; 真实 GPU timing 还需要 CUDA `synchronize`。
7. `profiler_summary.enabled=False` 是默认稳定运行模式; 需要真实 profiler 时打开环境变量再看 CPU 算子事件。

8.20 面试官会怎么问

问题一: loss 不下降你怎么排查?

回答模板:

我会先看数据和 label 是否正确, 再看 logits 和 labels 的 shape 是否对齐, loss 是否有限且依赖参数。然后检查梯度是否为 None。

问题二: 训练出现 NaN 怎么排查?

回答模板:

先定位 NaN 出现在输入、logits、loss、梯度还是参数。常见原因包括学习率过大、fp16 溢出、softmax 或 mask 产生异常、loss 分

问题三: GPU 利用率低怎么排查?

回答模板:

先区分是 DataLoader 慢、CPU-GPU 拷贝慢、模型计算小、频繁同步还是分布式通信慢。可以单独测试 DataLoader 迭代速度, 用 `cuda`

问题四: OOM 怎么排查?

回答模板:

先确认是训练还是验证 OOM, 再看 batch size、sequence length、是否开启 AMP、是否保存了带图 tensor、验证是否关闭梯度、是否

问题五: torch.profiler 能看什么?

回答模板:

`torch.profiler` 可以统计 CPU 和 CUDA 算子的耗时、调用次数、shape 和显存信息, 帮助定位耗时最多的算子、CPU/GPU 时间分布、D

8.21 常见误区

1. 一看到 loss 不下降就改模型结构, 不先查数据和 loss。
2. 只打印 loss, 不打印 shape、dtype、device。
3. 把中间 tensor 存进 list, 导致显存泄漏。
4. 验证时忘记 `no_grad`, 误以为模型太占显存。
5. CUDA 异步计时不加 `synchronize`, 导致性能判断错误。
6. profiler 一直开, 导致训练本身变慢。
7. 分布式日志不带 rank, 导致错误信息无法定位。
8. 用 `find_unused_parameters=True` 掩盖模型分支 bug。

8.22 小练习

1. 写一个 `debug_tensor` 函数, 打印 shape、dtype、device 和 contiguous 状态。
2. 构造一个 shape mismatch 的 loss 例子, 并修复它。
3. 构造一个 device mismatch 的例子, 并修复它。
4. 写一个 `check_finite` 函数, 检查 logits、loss 和 grad。
5. 写一个 forward hook, 保存某层输出并在使用后 remove。
6. 对同一个训练 step 用 `torch.cuda.synchronize()` 做分段计时。

7. 用 `torch.profiler` 跑 5 个 step, 打印耗时最多的算子。
8. 写一份 OOM 排查 checklist。
9. 写一份 loss 不下降排查 checklist。

8.23 本章总结

Debug 和 profiling 的核心是把问题分层。先确认数据、shape、dtype、device, 再确认 loss、梯度和参数更新, 然后看 NaN、OOM 和分布式同步, 最后才做性能 profiling。

需要记住:

1. 大多数训练 bug 可以先从一个 batch 的 shape、dtype、device 查起。
2. loss 不下降要先查数据、loss、梯度和 optimizer。
3. NaN 要定位到输入、logits、loss、梯度或参数中的哪一层。
4. OOM 要区分显存峰值过高和持续泄漏。
5. CUDA 计时要考虑异步执行, 必要时 `synchronize()`。
6. `torch.profiler` 用于定位性能瓶颈, 但不要长期打开。
7. 分布式 debug 要按 rank 打日志, 确认所有 rank 进入相同同步点。

下一章会进入实现 Transformer 组件, 用前面学到的 tensor、Module、训练循环和 debug 方法手写 attention、MLP、block、mask 和最小 decoder-only 模型。

第九章：实现 Transformer 组件

前面几章讲了 PyTorch 工程基础: tensor、autograd、Module、DataLoader、训练循环、混合精度、分布式和 debug。本章把这些能力集中到一个目标上: 手写 Transformer 的核心组件。大模型算法岗面试里, 最常见的代码题和工程追问往往不是让你完整复现一个大模型, 而是让你能讲清并写出 embedding、attention、causal mask、MLP、LayerNorm、RMSNorm、RoPE、KV Cache 和 decoder block 的最小实现。

本章代码以教学清晰为第一目标, 不追求 FlashAttention、PagedAttention、张量并行或极致性能。真实大模型会有更复杂的 kernel、并行策略和缓存管理, 但只要能稳定手写本章组件, 你就能理解大多数 decoder-only LLM 的主体结构。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前, 已核对 Transformer 原论文中的 scaled dot-product attention / multi-head attention 口径、PyTorch 官方 `scaled_dot_product_attention`、`nn.MultiheadAttention`、`nn.Embedding`、`nn.LayerNorm`、`nn.RMSNorm` 和 `nn.Transformer` 文档, 以及 RoPE / RoFormer 论文中旋转位置编码的基本定义。

本章聚焦面试和个人项目中最常需要手写、解释和 debug 的 decoder-only Transformer 组件: token embedding、LM head 权重绑定、RMSNorm、causal mask、scaled dot-product attention、multi-head self-attention、SwiGLU MLP、Pre-Norm decoder block、RoPE、causal LM shift loss 和教学版 KV Cache。

本章不展开 FlashAttention kernel、PagedAttention、MQA / GQA、tensor parallel、pipeline parallel、FSDP / ZeRO、`torch.compile`、生产级 serving scheduler、block table、prefix cache 或复杂长上下文外推策略。这些内容会在训练、部署、架构演进和推理框架章节中继续展开。

9.1 Transformer 组件总览

一个简化 decoder-only Transformer 通常包含:

1. Token embedding。
2. 位置编码或 RoPE。
3. 多层 Transformer block。
4. 每个 block 包含 causal self-attention、MLP、norm 和 residual。
5. 最后的 norm。
6. LM head, 把 hidden states 投到词表 logits。

数据流:

```
input_ids [B, T]
-> embedding [B, T, d]
-> block 1
-> block 2
-> ...
-> final norm
-> lm_head logits [B, T, V]
```

面试回答:

Decoder-only Transformer 的主干是 token embedding、多层 causal self-attention 和 MLP block、final norm 以及 LM head。每个 norm、self-attention、residual、pre-norm、MLP、residual。训练时输出 [B,T,V] logits, 用当前位置预测下一个 token。

9.1.1 关键公式与 Transformer 组件速查

第一, decoder-only LM 的输入输出 shape 主线是:

$I \in \mathbb{Z}^{B \times T}, \quad X = E[I] \in \mathbb{R}^{B \times T \times d}, \quad Z = XW_{\{\mathrm{lm}\}} \in \mathbb{R}^{B \times T \times V}$

其中 B 是 batch size, T 是序列长度, d 是 hidden size, V 是词表大小。

第二, 多头拆分要求:

$d = H D_h, \quad Q, K, V \in \mathbb{R}^{B \times H \times T \times D_h}$

如果 $\text{hidden_size} \% \text{num_heads} \neq 0$, 就无法把 hidden state 均匀拆成多个 head。

第三, scaled dot-product attention 是:

$S = \frac{QK^{\top}}{\sqrt{D_h}} + M, \quad A = \mathrm{softmax}(S), \quad O = AV$

其中 S 的 shape 是 [B,H,T_q,T_k], A 的最后一维应逐行求和为 1。

第四, 普通 causal mask 可以写成:

```
M_{t,s}=
\begin{cases}
0, & s \leq t \\
-\infty, & s > t
\end{cases}
```

如果使用 KV Cache, 当前 query 只包含新 token, key/value 包含历史 token, 此时更稳的形式是:

```
M_{t,s}=
\begin{cases}
0, & s \leq p+t \\
-\infty, & s > p+t
\end{cases}
```

其中 p 是已经缓存的历史 key/value 长度。这个细节能避免 decode 阶段 query_len=1, key_len>1 时 mask 方向错。

第五, RMSNorm 的核心是按最后一维均方根缩放:

$\mathrm{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}, \quad y_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}$

它不减均值, 计算比 LayerNorm 更简单。

第六, SwiGLU MLP 可以抽象为:

$\mathrm{SwiGLU}(x) = W_d \left(\mathrm{SiLU}(xW_g) \odot xW_u \right)$

输入输出 shape 都是 [B,T,d], 中间维度通常是 intermediate_size。

第七, Pre-Norm decoder block 的主线是:

$$x' = x + \mathrm{Attn}(\mathrm{Norm}(x)), \quad x_{\mathrm{out}} = x' + \mathrm{MLP}(\mathrm{Norm}(x'))$$

这也是现代 decoder-only LLM 中最常见的 block 结构。

第八，RoPE 在每个二维子空间上做旋转：

```
\begin{bmatrix}
q'_{2i} \\
q'_{2i+1}
\end{bmatrix}
=
\begin{bmatrix}
\cos\theta_{m,i} & -\sin\theta_{m,i} \\
\sin\theta_{m,i} & \cos\theta_{m,i}
\end{bmatrix}
\begin{bmatrix}
q_{2i} \\
q_{2i+1}
\end{bmatrix}
```

同样的旋转也作用到 K 。因为旋转矩阵保持向量范数，所以 RoPE 改变位置信息，不应改变每个二维子向量的长度。

第九，causal LM loss 需要右移：

$$L = -\frac{1}{N} \sum_{b,t} \log p_{\theta}(y_{b,t+1} \mid y_{b, \leq t})$$

实现时通常是 `shift_logits = logits[:, :-1, :]`, `shift_labels = labels[:, 1:]`，并用 `ignore_index=-100` 屏蔽 padding 或 prompt-only 位置。

第十，KV Cache 的单层显存量级可以粗略写成：

$$M_{\{\mathrm{kv}, \mathrm{layer}\}} = 2 \ B \ H_{\{\mathrm{kv}\}} \ T_{\{\mathrm{cache}\}} \ D_h \ b$$

其中因子 2 来自 K 和 V， b 是每个元素字节数。

9.2 配置对象

为了避免到处传参数，先定义一个简单配置。

```
from dataclasses import dataclass
```

```
@dataclass
class TransformerConfig:
    vocab_size: int = 32000
    hidden_size: int = 512
    num_layers: int = 6
    num_heads: int = 8
    intermediate_size: int = 2048
    max_position_embeddings: int = 2048
    dropout: float = 0.0
```

要求：

```
assert config.hidden_size % config.num_heads == 0
```

因为每个 head 的维度是：

```
head_dim = hidden_size // num_heads
```


9.3 Token Embedding 和 LM Head

Token embedding 把离散 token id 变成连续向量。

```
import torch
from torch import nn
```

```
embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)
```

```
input_ids = torch.randint(0, config.vocab_size, (2, 8))
x = embed_tokens(input_ids)
print(x.shape) # [B, T, d]
```

LM head 把 hidden states 投到词表大小：

```
lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)
logits = lm_head(x)
print(logits.shape) # [B, T, V]
```

很多语言模型会做权重绑定：

```
lm_head.weight = embed_tokens.weight
```

含义是输入 embedding 和输出词表投影共享同一份权重。优点是减少参数量，并让输入输出 token 表示空间更一致。

9.4 手写 RMSNorm

现代 LLM 经常使用 RMSNorm。它和 LayerNorm 的区别是：RMSNorm 不减均值，只按均方根归一化。

公式直觉：

```
rms = sqrt(mean(x * x) + eps)
y = x / rms * weight
```

PyTorch 实现：

```
class RMSNorm(nn.Module):
    def __init__(self, hidden_size, eps=1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.eps = eps

    def forward(self, x):
        input_dtype = x.dtype
        x = x.float()
        variance = x.pow(2).mean(dim=-1, keepdim=True)
        x = x * torch.rsqrt(variance + self.eps)
        return (self.weight * x).to(input_dtype)
```

这里把 x 临时转成 fp32，是为了数值更稳。最后再转回原 dtype。

Shape：

```
x: [B, T, d]
weight: [d]
output: [B, T, d]
```

9.5 Causal Mask

Causal mask 防止当前位置看到未来 token。

最小实现:

```
def make_causal_mask(seq_len, device):
    mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool, device=device))
    return mask[None, None, :, :] # [1, 1, T, T]
```

使用:

```
scores = torch.randn(2, 8, 16, 16)
mask = make_causal_mask(16, scores.device)
scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)
```

注意: 不要在 fp16 中随便写很大的负数, 比如 $-1e30$, 可能溢出。更稳妥的是使用:

```
torch.finfo(scores.dtype).min
```

9.6 Scaled Dot-Product Attention

输入:

```
q: [B, H, T, D]
k: [B, H, S, D]
v: [B, H, S, D]
```

输出:

```
context: [B, H, T, D]
```

实现:

```
import math
```

```
def make_causal_mask(query_len, key_len, device, past_key_values_length=0):
    query_positions = torch.arange(query_len, device=device)[: , None]
    key_positions = torch.arange(key_len, device=device)[None, :]
    return query_positions <= query_positions + past_key_values_length
```

```
def scaled_dot_product_attention(q, k, v, causal=True, past_key_values_length=0):
    head_dim = q.size(-1)
    scores = q @ k.transpose(-2, -1)
    scores = scores / math.sqrt(head_dim)

    if causal:
        query_len = q.size(-2)
        key_len = k.size(-2)
        mask = make_causal_mask(query_len, key_len, q.device, past_key_values_length)
        scores = scores.masked_fill(~mask[None, None, :, :], torch.finfo(scores.dtype).min)

    attn = torch.softmax(scores, dim=-1)
    context = attn @ v
    return context
```

这里的关键是:

1. $q @ k.transpose(-2, -1)$ 得到 attention scores。
2. 除以 $\sqrt{\text{head_dim}}$ 稳定 softmax。
3. causal mask 禁止看未来。
4. softmax 后乘 v 得到上下文表示。

这里额外传入 `past_key_values_length`，是为了兼容 KV Cache decode：当 `q` 只有当前 token、`k/v` 包含历史 token 时，当前 query 应该能看到全部历史 key，而不是只看到 key 序列的第 0 个位置。

9.7 Multi-Head Self-Attention

把输入 `[B, T, d]` 投影成多头 Q/K/V。

```
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.num_heads = config.num_heads
        self.head_dim = config.hidden_size // config.num_heads

        self.q_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.k_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.v_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.o_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)

    def _shape(self, x, batch_size, seq_len):
        x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
        return x.transpose(1, 2)  # [B, H, T, D]

    def forward(self, x):
        batch_size, seq_len, _ = x.shape

        q = self._shape(self.q_proj(x), batch_size, seq_len)
        k = self._shape(self.k_proj(x), batch_size, seq_len)
        v = self._shape(self.v_proj(x), batch_size, seq_len)

        context = scaled_dot_product_attention(q, k, v, causal=True)

        context = context.transpose(1, 2).contiguous()
        context = context.view(batch_size, seq_len, self.hidden_size)
        return self.o_proj(context)
```

Shape 主线：

```
x: [B, T, d]
q/k/v after projection: [B, T, d]
q/k/v after reshape: [B, H, T, D]
context: [B, H, T, D]
merged context: [B, T, d]
```

9.8 MLP 和 SwiGLU

Transformer block 里的 MLP 负责逐 token 的非线性变换。

普通 MLP：

```
class MLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.fc1 = nn.Linear(config.hidden_size, config.intermediate_size)
        self.act = nn.GELU()
        self.fc2 = nn.Linear(config.intermediate_size, config.hidden_size)
```

```
def forward(self, x):
    return self.fc2(self.act(self.fc1(x)))
```

现代 LLM 常用 SwiGLU 变体:

```
class SwiGLUMLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.gate_proj = nn.Linear(config.hidden_size, config.intermediate_size, bias=False)
        self.up_proj = nn.Linear(config.hidden_size, config.intermediate_size, bias=False)
        self.down_proj = nn.Linear(config.intermediate_size, config.hidden_size, bias=False)

    def forward(self, x):
        gate = torch.nn.functional.silu(self.gate_proj(x))
        up = self.up_proj(x)
        return self.down_proj(gate * up)
```

Shape 不变:

input: [B, T, d]
output: [B, T, d]

9.9 Decoder Block

现代 decoder-only LLM 通常用 Pre-Norm 结构。

```
class DecoderBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.input_norm = RMSNorm(config.hidden_size)
        self.self_attn = MultiHeadSelfAttention(config)
        self.post_attn_norm = RMSNorm(config.hidden_size)
        self.mlp = SwiGLUMLP(config)

    def forward(self, x):
        residual = x
        x = self.input_norm(x)
        x = self.self_attn(x)
        x = residual + x

        residual = x
        x = self.post_attn_norm(x)
        x = self.mlp(x)
        x = residual + x
        return x
```

核心结构:

```
x = x + Attention(Norm(x))
x = x + MLP(Norm(x))
```

面试中要能解释: attention 负责 token 间通信, MLP 负责每个 token 表示的非线性加工, residual 帮助深层训练和信息保留, norm 稳定激活尺度。

9.10 RoPE 的最小实现

RoPE 作用在 Q/K 上, 通过旋转注入位置信息。

先构造频率:

```
def build_rope_cache(seq_len, head_dim, device, base=10000):
    assert head_dim % 2 == 0
    inv_freq = 1.0 / (base ** (torch.arange(0, head_dim, 2, device=device).float() / head_dim))
    positions = torch.arange(seq_len, device=device).float()
    freqs = torch.outer(positions, inv_freq) # [T, D/2]
    cos = freqs.cos()[None, None, :, :] # [1, 1, T, D/2]
    sin = freqs.sin()[None, None, :, :] # [1, 1, T, D/2]
    return cos, sin
```

旋转函数:

```
def apply_rope(x, cos, sin):
    x1 = x[..., 0::2]
    x2 = x[..., 1::2]
    out = torch.stack((x1 * cos - x2 * sin, x1 * sin + x2 * cos), dim=-1)
    return out.flatten(-2)
```

输入输出:

```
x: [B, H, T, D]
cos/sin: [1, 1, T, D/2]
output: [B, H, T, D]
```

在 attention 中使用:

```
cos, sin = build_rope_cache(seq_len, self.head_dim, x.device)
q = apply_rope(q, cos, sin)
k = apply_rope(k, cos, sin)
```

RoPE 的重点不是背代码, 而是知道它通过对 Q/K 按位置旋转, 让 attention score 包含相对位置信息。

9.11 加入 RoPE 的 Attention

```
class RoPEMultiHeadSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.num_heads = config.num_heads
        self.head_dim = config.hidden_size // config.num_heads
        self.q_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.k_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.v_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.o_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)

    def _shape(self, x, batch_size, seq_len):
        x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
        return x.transpose(1, 2)

    def forward(self, x):
        batch_size, seq_len, _ = x.shape
        q = self._shape(self.q_proj(x), batch_size, seq_len)
        k = self._shape(self.k_proj(x), batch_size, seq_len)
        v = self._shape(self.v_proj(x), batch_size, seq_len)

        cos, sin = build_rope_cache(seq_len, self.head_dim, x.device)
        q = apply_rope(q, cos, sin)
        k = apply_rope(k, cos, sin)

        context = scaled_dot_product_attention(q, k, v, causal=True)
```

```

        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.hidden_size)
        return self.o_proj(context)

```

真实工程里通常会缓存 cos 和 sin，而不是每次 forward 都重新构造。

9.12 最小 Decoder-only LM

把组件组合起来：

```

class MiniDecoderLM(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)
        self.layers = nn.ModuleList([DecoderBlock(config) for _ in range(config.num_layers)])
        self.norm = RMSNorm(config.hidden_size)
        self.lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)
        self.lm_head.weight = self.embed_tokens.weight

    def forward(self, input_ids, labels=None):
        x = self.embed_tokens(input_ids)

        for layer in self.layers:
            x = layer(x)

        x = self.norm(x)
        logits = self.lm_head(x)

        loss = None
        if labels is not None:
            shift_logits = logits[:, :-1, :].contiguous()
            shift_labels = labels[:, 1:].contiguous()
            loss = torch.nn.functional.cross_entropy(
                shift_logits.view(-1, shift_logits.size(-1)),
                shift_labels.view(-1),
                ignore_index=-100,
            )

        return {"loss": loss, "logits": logits}

```

测试 shape：

```

config = TransformerConfig(vocab_size=1000, hidden_size=128, num_layers=2, num_heads=4, intermediate_size=512)
model = MiniDecoderLM(config)

input_ids = torch.randint(0, config.vocab_size, (2, 16))
labels = input_ids.clone()
outputs = model(input_ids, labels=labels)

print(outputs["logits"].shape) # [2, 16, 1000]
print(outputs["loss"])

```

9.13 KV Cache 的直觉

训练时通常一次输入完整序列。推理生成时，每次只生成一个新 token。如果每一步都重新计算整个历史序列的 K/V，会浪费大量计算。

KV Cache 的思路是：

1. 第一次 prefill 时计算 prompt 的 K/V。
2. 后续 decode 每生成一个 token，只计算新 token 的 K/V。
3. 把新 K/V 拼到历史 cache 后面。
4. query 只来自当前 token，但 key/value 来自完整历史。

没有 cache：每步重复算历史。

有 cache：历史 K/V 复用。

9.14 简化 KV Cache Attention

注意：下面是教学版，只展示核心逻辑。

```
class CachedSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.num_heads = config.num_heads
        self.head_dim = config.hidden_size // config.num_heads
        self.q_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.k_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.v_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.o_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)

    def _shape(self, x, batch_size, seq_len):
        return x.view(batch_size, seq_len, self.num_heads, self.head_dim).transpose(1, 2)

    def forward(self, x, past_key_value=None, use_cache=False):
        batch_size, seq_len, _ = x.shape

        q = self._shape(self.q_proj(x), batch_size, seq_len)
        k = self._shape(self.k_proj(x), batch_size, seq_len)
        v = self._shape(self.v_proj(x), batch_size, seq_len)

        past_len = 0
        if past_key_value is not None:
            past_k, past_v = past_key_value
            past_len = past_k.size(2)
            k = torch.cat([past_k, k], dim=2)
            v = torch.cat([past_v, v], dim=2)

        context = scaled_dot_product_attention(
            q,
            k,
            v,
            causal=True,
            past_key_values_length=past_len,
        )
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.hidden_size)
        output = self.o_proj(context)

        new_cache = (k, v) if use_cache else None
        return output, new_cache
```

KV Cache shape:

k: [B, H, T_cache, D]

v: [B, H, T_cache, D]

真实推理框架还要处理 batch 内不同长度、分页管理、prefix cache、内存复用和连续批处理。

9.15 最小可运行 Transformer 组件审计 demo

下面这个 demo 用很小的配置把本章关键组件串起来。它的目标不是训练出好模型，而是帮助你在面试或项目 debug 时快速确认：shape、mask、loss shift、RoPE 范数、权重绑定和 KV Cache 长度都符合预期。

```
import math
from dataclasses import dataclass
```

```
import torch
from torch import nn
import torch.nn.functional as F
```

```
torch.manual_seed(7)
torch.set_num_threads(1)
```

```
@dataclass
```

```
class TransformerConfig:
    vocab_size: int = 32
    hidden_size: int = 16
    num_layers: int = 2
    num_heads: int = 4
    intermediate_size: int = 32
    max_position_embeddings: int = 16
    dropout: float = 0.0
```

```
class RMSNorm(nn.Module):
    def __init__(self, hidden_size, eps=1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.eps = eps

    def forward(self, x):
        input_dtype = x.dtype
        x = x.float()
        variance = x.pow(2).mean(dim=-1, keepdim=True)
        x = x * torch.rsqrt(variance + self.eps)
        return (self.weight * x).to(input_dtype)
```

```
def make_causal_mask(query_len, key_len, device, past_key_values_length=0):
    query_positions = torch.arange(query_len, device=device)[: , None]
    key_positions = torch.arange(key_len, device=device)[None, :]
    mask = key_positions <= query_positions + past_key_values_length
    return mask[None, None, :, :]
```

```
def scaled_dot_product_attention(
    q,
    k,
    v,
    causal=True,
```



```

past_key_values_length=0,
return_attn=False,
):
    head_dim = q.size(-1)
    scores = q @ k.transpose(-2, -1)
    scores = scores / math.sqrt(head_dim)

    if causal:
        mask = make_causal_mask(
            q.size(-2),
            k.size(-2),
            q.device,
            past_key_values_length=past_key_values_length,
        )
        scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)

    attn = torch.softmax(scores, dim=-1)
    context = attn @ v
    if return_attn:
        return context, attn
    return context

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        assert config.hidden_size % config.num_heads == 0
        self.hidden_size = config.hidden_size
        self.num_heads = config.num_heads
        self.head_dim = config.hidden_size // config.num_heads
        self.q_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.k_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.v_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.o_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)

    def _shape(self, x, batch_size, seq_len):
        x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
        return x.transpose(1, 2)

    def forward(self, x, return_attn=False):
        batch_size, seq_len, _ = x.shape
        q = self._shape(self.q_proj(x), batch_size, seq_len)
        k = self._shape(self.k_proj(x), batch_size, seq_len)
        v = self._shape(self.v_proj(x), batch_size, seq_len)
        context, attn = scaled_dot_product_attention(q, k, v, return_attn=True)
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.hidden_size)
        out = self.o_proj(context)
        if return_attn:
            return out, attn
        return out

class SwiGLUMLP(nn.Module):
    def __init__(self, config):
        super().__init__()

```

```

self.gate_proj = nn.Linear(config.hidden_size, config.intermediate_size, bias=False)
self.up_proj = nn.Linear(config.hidden_size, config.intermediate_size, bias=False)
self.down_proj = nn.Linear(config.intermediate_size, config.hidden_size, bias=False)

def forward(self, x):
    gate = F.silu(self.gate_proj(x))
    up = self.up_proj(x)
    return self.down_proj(gate * up)

class DecoderBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.input_norm = RMSNorm(config.hidden_size)
        self.self_attn = MultiHeadSelfAttention(config)
        self.post_attn_norm = RMSNorm(config.hidden_size)
        self.mlp = SwiGLUMLP(config)

    def forward(self, x):
        x = x + self.self_attn(self.input_norm(x))
        x = x + self.mlp(self.post_attn_norm(x))
        return x

def build_rope_cache(seq_len, head_dim, device, base=10000):
    assert head_dim % 2 == 0
    inv_freq = 1.0 / (base ** (torch.arange(0, head_dim, 2, device=device).float() / head_dim))
    positions = torch.arange(seq_len, device=device).float()
    freqs = torch.outer(positions, inv_freq)
    return freqs.cos()[None, None, :, :], freqs.sin()[None, None, :, :]

def apply_rope(x, cos, sin):
    x1 = x[..., 0::2]
    x2 = x[..., 1::2]
    out = torch.stack((x1 * cos - x2 * sin, x1 * sin + x2 * cos), dim=-1)
    return out.flatten(-2)

class MiniDecoderLM(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size)
        self.layers = nn.ModuleList([DecoderBlock(config) for _ in range(config.num_layers)])
        self.norm = RMSNorm(config.hidden_size)
        self.lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)
        self.lm_head.weight = self.embed_tokens.weight

    def forward(self, input_ids, labels=None):
        x = self.embed_tokens(input_ids)
        for layer in self.layers:
            x = layer(x)
        logits = self.lm_head(self.norm(x))
        loss = None
        if labels is not None:

```

```

    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = labels[:, 1:].contiguous()
    loss = F.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
        ignore_index=-100,
    )
    return {"loss": loss, "logits": logits}

```

```

class CachedSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.num_heads = config.num_heads
        self.head_dim = config.hidden_size // config.num_heads
        self.q_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.k_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.v_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)
        self.o_proj = nn.Linear(config.hidden_size, config.hidden_size, bias=False)

    def _shape(self, x, batch_size, seq_len):
        return x.view(batch_size, seq_len, self.num_heads, self.head_dim).transpose(1, 2)

    def forward(self, x, past_key_value=None, use_cache=False):
        batch_size, seq_len, _ = x.shape
        q = self._shape(self.q_proj(x), batch_size, seq_len)
        k = self._shape(self.k_proj(x), batch_size, seq_len)
        v = self._shape(self.v_proj(x), batch_size, seq_len)

        past_len = 0
        if past_key_value is not None:
            past_k, past_v = past_key_value
            past_len = past_k.size(2)
            k = torch.cat([past_k, k], dim=2)
            v = torch.cat([past_v, v], dim=2)

        context, attn = scaled_dot_product_attention(
            q,
            k,
            v,
            causal=True,
            past_key_values_length=past_len,
            return_attn=True,
        )
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.hidden_size)
        new_cache = (k, v) if use_cache else None
        return self.o_proj(context), new_cache, attn

config = TransformerConfig()
batch_size, seq_len = 2, 5
x = torch.randn(batch_size, seq_len, config.hidden_size)
input_ids = torch.tensor([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
labels = input_ids.clone()

```

```

rms = RMSNorm(config.hidden_size)
rms_out = rms(x)
rms_mean_square = rms_out.pow(2).mean(dim=-1)[0, 0].item()

mask = make_causal_mask(4, 4, x.device)
mask_rows = mask[0, 0].int().tolist()

q = torch.randn(batch_size, config.num_heads, seq_len, config.hidden_size // config.num_heads)
k = torch.randn_like(q)
v = torch.randn_like(q)
context, attn = scaled_dot_product_attention(q, k, v, return_attn=True)
future_mask = ~make_causal_mask(seq_len, seq_len, x.device)
future_weight_max = attn.masked_select(future_mask).max().item()

mha = MultiHeadSelfAttention(config)
mha_out, mha_attn = mha(x, return_attn=True)
mlp_out = SwiGLUMLP(config)(x)
block_out = DecoderBlock(config)(x)

cos, sin = build_rope_cache(seq_len, config.hidden_size // config.num_heads, x.device)
rope_in = q.clone()
rope_out = apply_rope(rope_in, cos, sin)
rope_norm_preserved = torch.allclose(rope_in.norm(dim=-1), rope_out.norm(dim=-1), atol=1e-5)

model = MiniDecoderLM(config)
outputs = model(input_ids, labels=labels)
tied_weights = model.lm_head.weight.data_ptr() == model.embed_tokens.weight.data_ptr()

cache_attn = CachedSelfAttention(config)
prefill_out, cache, prefill_attn = cache_attn(x, use_cache=True)
next_x = torch.randn(batch_size, 1, config.hidden_size)
decode_out, cache, decode_attn = cache_attn(next_x, past_key_value=cache, use_cache=True)
decode_can_see_all_cache = decode_attn[0, 0, 0].numel() == seq_len + 1

summary = {
    "rms_shape": tuple(rms_out.shape),
    "rms_mean_square_first": round(rms_mean_square, 4),
    "mask_rows": mask_rows,
    "context_shape": tuple(context.shape),
    "attn_row_sums": [round(v, 4) for v in attn[0, 0].sum(dim=-1).tolist()],
    "future_weight_max": round(future_weight_max, 4),
    "mha_shape": tuple(mha_out.shape),
    "mha_attn_shape": tuple(mha_attn.shape),
    "swiglu_shape": tuple(mlp_out.shape),
    "block_shape": tuple(block_out.shape),
    "rope_shape": tuple(rope_out.shape),
    "rope_norm_preserved": bool(rope_norm_preserved),
    "logits_shape": tuple(outputs["logits"].shape),
    "loss": round(outputs["loss"].item(), 4),
    "tied_weights": tied_weights,
    "cache_lengths": (prefill_attn.size(-1), cache[0].size(2)),
    "decode_attn_shape": tuple(decode_attn.shape),
    "decode_can_see_all_cache": decode_can_see_all_cache,
}

```

```

checks = {
    "rms_shape": summary["rms_shape"] == (2, 5, 16),
    "mask_direction": mask_rows[-1] == [1, 1, 1, 1],
    "attention_shape": summary["context_shape"] == (2, 4, 5, 4),
    "attention_rows_sum_to_one": all(v == 1.0 for v in summary["attn_row_sums"]),
    "future_mask_zero": summary["future_weight_max"] == 0.0,
    "mha_shape": summary["mha_shape"] == (2, 5, 16),
    "swiglu_shape": summary["swiglu_shape"] == (2, 5, 16),
    "block_shape": summary["block_shape"] == (2, 5, 16),
    "rope_norm": summary["rope_norm_preserved"],
    "lm_shape": summary["logits_shape"] == (2, 5, 32),
    "weight_tying": summary["tied_weights"],
    "cache_grows": summary["cache_lengths"] == (5, 6),
    "decode_cache_visible": summary["decode_can_see_all_cache"],
}

print("summary=", summary)
print("checks=", checks)
print("gate_pass=", all(checks.values()))

```

运行后应看到所有 checks 为 True，并且 gate_pass=True。如果某一项失败，优先按下面顺序排查：先看 shape，再看 mask 方向，再看 contiguous().view(...)，最后看 loss 是否正确右移。

9.16 常见实现 bug

9.16.1 head_dim 不整除

```
assert hidden_size % num_heads == 0
```

否则无法把 hidden size 平均拆到多头。

9.16.2 mask 方向错

Causal mask 应该允许当前位置看自己和过去，禁止看未来。

长度为 4 的 mask 应该类似：

```

1 0 0 0
1 1 0 0
1 1 1 0
1 1 1 1

```

KV Cache decode 时还要检查 query_len 和 key_len 不相等的情况。当前 token 的 query 应该能看见完整历史 cache，否则输出会只依赖很短的历史。

9.16.3 transpose 后忘记 contiguous

```
context = context.transpose(1, 2).contiguous().view(B, T, d)
```

如果直接 view，可能因为内存 layout 不连续而报错。

9.16.4 loss 没 shift

语言模型训练需要：

```

shift_logits = logits[:, :-1, :]
shift_labels = labels[:, 1:]

```

9.16.5 padding 参与 loss

padding 或 prompt 中不训练的位置要设成 -100，并在 cross entropy 里使用 ignore_index=-100。

9.17 面试官会怎么问

问题一：Self-attention 的 shape 怎么变化？

回答模板：

输入 hidden states 是 $[B, T, d]$ 。经过 Q/K/V 投影后仍是 $[B, T, d]$ ，再 reshape 成 $[B, H, T, D]$ ，其中 $D=d/H$ 。计算 $Q @ K.transpose(-2, -1)$ 得到 scores $[B, H, T, T]$ ，softmax 后乘 V 得到 context $[B, H, T, D]$ ，最后合并多头回到 $[B, T, d]$ 。

问题二：为什么需要 causal mask？

回答模板：

Decoder-only 语言模型要按自回归方式预测下一个 token，当前位置不能看到未来 token。训练时虽然整段序列并行输入，但 causal

问题三：RMSNorm 和 LayerNorm 有什么区别？

回答模板：

LayerNorm 会减均值并除以标准差，RMSNorm 不减均值，只按均方根缩放。RMSNorm 计算更简单，是很多现代 LLM 常用的归一化方式。

问题四：RoPE 作用在哪里？

回答模板：

RoPE 通常作用在 attention 的 Q 和 K 上，对它们按位置做旋转。这样 Q 和 K 的点积会自然包含相对位置信息，因此模型可以在 at

问题五：KV Cache 为什么能加速推理？

回答模板：

自回归推理每步只新增一个 token，历史 token 的 K/V 不变。KV Cache 缓存历史 K/V，后续 decode 只计算新 token 的 K/V，并和历

9.18 小练习

1. 手写 RMSNorm，并验证输入输出 shape 不变。
2. 写出长度为 5 的 causal mask。
3. 手写 scaled dot-product attention，输入 $[B, H, T, D]$ 。
4. 写一个 MultiHeadSelfAttention，并打印每一步 shape。
5. 把普通 MLP 改成 SwiGLU MLP。
6. 写一个 DecoderBlock，确认 residual 前后 shape 一致。
7. 给 attention 加 RoPE。
8. 写一个最小 decoder-only LM，并跑一次 forward 和 loss。
9. 给 attention 加一个教学版 KV Cache。
10. 故意去掉 causal mask，解释为什么训练 loss 可能异常偏低。
11. 修改本章审计 demo，让 seq_len=1 的 decode token 使用长度为 6 的 KV Cache，并验证 attention 权重长度也是 6。

9.19 本章总结

本章把 PyTorch 基础能力落到了 Transformer 组件实现上。需要掌握的主线是：embedding 把 token id 转成 hidden states，attention 负责 token 间通信，MLP 负责逐 token 非线性变换，norm 和 residual 保证深层训练稳定，RoPE 注入位置信息，KV Cache 加速自回归推理。

核心记忆点：

1. 输入 `input_ids` 是 $[B, T]$, embedding 后是 $[B, T, d]$ 。
2. 多头 attention 中 Q/K/V 是 $[B, H, T, D]$ 。
3. attention scores 是 $[B, H, T, T]$ 。
4. causal mask 禁止看未来。
5. decoder block 常见结构是 $x = x + \text{Attention}(\text{Norm}(x))$ 和 $x = x + \text{MLP}(\text{Norm}(x))$ 。
6. RoPE 作用在 Q/K 上。
7. KV Cache 缓存历史 K/V, 减少 decode 阶段重复计算。

下一章会进入工程面试题, 集中整理 PyTorch 工程、训练循环、显存、分布式和 Transformer 实现相关的高频问答。

第十章：工程面试题

本章把第十四册的 PyTorch 与深度学习工程知识整理成面试问答。重点不是背答案, 而是训练一种工程表达方式: 先讲核心机制, 再讲常见坑, 最后给出排查路径或实现要点。很多面试问题表面是 PyTorch API, 实际考的是你是否真的写过训练代码、是否能定位训练异常、是否理解大模型组件的 shape 和系统约束。

建议使用方法: 先自己口述, 再对照参考回答。每题尽量控制在 1-3 分钟内, 如果面试官追问, 再展开代码细节或 debug 清单。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前, 已核对 PyTorch 官方 tensor view / reshape / contiguous、autograd mechanics、`nn.Module`、`torch.utils.data`、AMP、`DistributedDataParallel`、`DistributedSampler`、checkpoint 和 profiler 相关文档资料边界。

本章定位是第十四册的工程面试复盘章: 把前 1-9 章的 tensor、autograd、Module、DataLoader、training loop、AMP / 显存、DDP、debug / profiling 和 Transformer component 串成可口述、可追问、可自评的问答框架。

本章不展开完整 PyTorch API 手册、CUDA kernel、`torch.compile` 编译器内部、生产级分布式 checkpoint、NCCL 网络调优、FSDP wrap policy、模型并行系统或真实集群运维。这些内容应放到训练系统、AI Infra 和推理框架专题中继续展开。

10.0.1 工程面试公式与排查速查

第一, 语言模型 batch 的 shape 主线是:

$$I \in \mathbb{Z}^{B \times T}, \quad H \in \mathbb{R}^{B \times T \times d}, \quad Z \in \mathbb{R}^{B \times T \times V}$$

其中 `I` 是 `input_ids`, `H` 是 hidden states, `Z` 是 logits, `B` 是 batch size, `T` 是序列长度, `d` 是 hidden size, `V` 是词表大小。

第二, causal LM 的 shift loss 展平后行数应一致:

$$N_{\text{row}} = B(T-1), \quad Z_{\text{shift}} \in \mathbb{R}^{N_{\text{row}} \times V}, \quad Y_{\text{shift}} \in \mathbb{R}^{N_{\text{row}} \times V}$$

面试里如果只说“算 cross entropy”, 但没有说清 logits[:, :-1, :] 和 labels[:, 1:], 通常还不够。

第三, 梯度累积要缩放 loss:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^K g_i$$

其中 `K` 是 accumulation steps。实现上常见做法是每个 micro-batch 的 loss 除以 `K` 后再 backward。

第四, global batch size 是:

$$B_{\text{global}} = B_{\text{device}} \cdot W \cdot K$$

其中 `B_device` 是单进程 batch size, `W` 是 world size, `K` 是 accumulation steps。从单卡切到多卡时, 学习率、warmup、日志步数和评估频率都要重新核对。

第五，训练显存可粗略拆成：

$$M_{\{\mathrm{train}\}} \approx M_{\{\mathrm{param}\}} + M_{\{\mathrm{grad}\}} + M_{\{\mathrm{opt}\}} + M_{\{\mathrm{act}\}} + M_{\{\mathrm{tmp}\}}$$

其中 AdamW optimizer state 常包含一阶矩和二阶矩，长序列训练中 activation 也可能成为主要瓶颈。

第六，DDP 梯度同步的核心是平均梯度：

$$\bar{g} = \frac{1}{W} \sum_{r=0}^{W-1} g_r, \quad \theta_{t+1} = \theta_t - \eta \bar{g}$$

如果不同 rank 没有同步到同一个 $\bar{g}$ ，参数会逐步漂移。

第七，多头 attention 的 score shape 是：

$$Q, K, V \in \mathbb{R}^{B \times H \times T \times D_h}, \quad S = QK^{\top} \in \mathbb{R}^{B \times H \times T \times T}$$

面试中要同时说清 shape、mask 方向、softmax 维度和合并多头时的 contiguous() / reshape() 问题。

第八，KV Cache 单层显存量级是：

$$M_{\{\mathrm{kv}, \mathrm{layer}\}} = 2 \cdot B \cdot H_{\{\mathrm{kv}\}} \cdot T_{\{\mathrm{cache}\}} \cdot D_h \cdot b$$

其中因子 2 来自 K 和 V，b 是每个元素字节数。长上下文和高并发服务中，KV Cache 往往是显存和带宽瓶颈。

第九，一个 PyTorch 工程面试回答可以用门禁检查：

```
G_{\mathrm{eng}} = \mathbf{1}[
C_{\mathrm{topic}} \geq \tau_c,
C_{\mathrm{formula}} \geq \tau_f,
C_{\mathrm{debug}} \geq \tau_d,
R_{\mathrm{flag}} = 0
]
```

其中 C_{topic} 是主题覆盖率， C_{formula} 是公式和 shape 覆盖率， C_{debug} 是排查路径覆盖率， R_{flag} 是空泛、绝对化或夸大表述的风险数。这个门禁不是招聘标准，而是自我复盘工具。

10.1 Tensor 与 Shape

问题 1：PyTorch tensor 最重要的元信息有哪些？

参考回答：

我会优先看 shape、dtype、device、requires_grad 和 contiguous 状态。shape 决定矩阵乘法、loss 和 mask 是否对齐；dtype 决定

问题 2：view 和 reshape 有什么区别？

参考回答：

view 要求 tensor 的内存布局能按目标 shape 解释，通常要求 contiguous 或 stride 兼容；reshape 更宽松，如果不能返回 view，可

问题 3：为什么 transpose 后经常要 contiguous()？

参考回答：

transpose 通常只是改变 stride，不会真的重排底层内存，因此结果可能不是 contiguous。后面如果直接 view 合并维度，可能因为

问题 4：Broadcasting 规则是什么？

参考回答：

Broadcasting 从右往左对齐维度，每一维要么相等，要么其中一个是 1，要么其中一个不存在。它常用于 bias、LayerNorm 参数、at

10.2 Autograd 与梯度

问题 5: PyTorch autograd 是怎么工作的?

参考回答:

PyTorch 使用动态计算图。每次 forward 时, 如果输入或参数需要梯度, 相关操作会记录反向函数和必要中间值。调用 `loss.backward()`

问题 6: 为什么每个 step 要 `zero_grad()`?

参考回答:

因为 PyTorch 的梯度默认累积, 不会在 backward 前自动清零。普通训练中如果不清梯度, 当前 batch 的梯度会叠加到历史梯度上, 导致

问题 7: 为什么中间 tensor 的 `.grad` 是 None?

参考回答:

默认只有叶子 tensor 的 `.grad` 会被保存, 模型参数就是叶子 tensor。中间 tensor 虽然有梯度流过, 但为了省显存, PyTorch 不默认

问题 8: `detach()` 和 `no_grad()` 有什么区别?

参考回答:

`detach` 是对某个 tensor 切断它和当前计算图的连接, 常用于日志、缓存或 `stop-gradient`。`no_grad` 是上下文管理器, 在作用域内不

10.3 nn.Module 与模型组织

问题 9: `nn.Module` 的作用是什么?

参考回答:

`nn.Module` 是 PyTorch 组织模型的核心抽象。它不仅封装 forward, 还负责注册 Parameter、子模块和 buffer, 并支持递归遍历参数、

问题 10: 为什么子模块不能放普通 list?

参考回答:

普通 Python list 里的 Module 不会被 `nn.Module` 自动注册, 因此 `parameters()`、`state_dict()`、`to(device)`、`train/eval`、`DDP` 都

问题 11: Parameter 和 buffer 的区别是什么?

参考回答:

Parameter 是模型可训练权重, 通常会被 optimizer 更新; buffer 是不训练但属于模型状态的 tensor, 比如 BatchNorm running sta

问题 12: `state_dict` 保存什么?

参考回答:

`state_dict` 保存注册的 Parameter 和持久化 buffer, 不保存 Python 类定义、forward 代码和 optimizer 状态。训练恢复时还要额外

10.4 Dataset、DataLoader 与训练数据

问题 13: Dataset 和 DataLoader 有什么区别?

参考回答:

Dataset 定义单个样本如何读取和返回, DataLoader 负责把样本组织成 batch, 并支持 shuffle、sampler、多进程加载和 collate_fn

问题 14: 为什么变长序列需要自定义 `collate_fn`?

参考回答:

默认 `collate` 会尝试把样本 `stack` 成规则张量, 但变长 `token` 序列长度不同, 不能直接 `stack`。自定义 `collate_fn` 可以在 `batch` 级 `100` 和其他复杂字段整理。

问题 15: `padding token` 为什么不能直接参与 `loss`?

参考回答:

`padding` 只是为了把变长样本组成 `batch`, 不代表真实训练目标。如果 `padding token` 参与 `loss`, 模型会被训练去预测无意义的 `pad`, `100`, 并在 `cross entropy` 中用 `ignore_index` 跳过。

10.5 训练循环

问题 16: 一个标准训练 `step` 包含哪些步骤?

参考回答:

标准训练 `step` 包括 `batch` 到 `device`、`zero_grad`、`forward`、`loss` 计算、`backward`、可选梯度裁剪、`optimizer.step`、`scheduler.step`。

问题 17: 梯度累积为什么要除以 `accumulation steps`?

参考回答:

因为梯度默认累积。如果累积 `N` 个 `micro-batch` 但不缩放 `loss`, 最终梯度尺度大约是平均梯度的 `N` 倍, 相当于放大学习率。把 `loss`

问题 18: 验证循环和训练循环有什么区别?

参考回答:

验证时要 `model.eval()`, 并用 `no_grad` 或 `inference_mode` 关闭梯度, 不做 `backward` 和 `optimizer step`。验证结束后要切回 `model.train()`。

问题 19: `checkpoint` 应该保存什么?

参考回答:

如果只是推理, 保存模型 `state_dict` 就够了; 如果要恢复训练, 应该保存 `model`、`optimizer`、`scheduler`、`epoch`、`global_step`、`co`

10.6 混合精度与显存

问题 20: 训练显存主要花在哪里?

参考回答:

训练显存主要花在模型参数、梯度、优化器状态、前向激活、临时 `buffer` 和输入 `batch` 上。`Adam` 类优化器还要保存一阶矩和二阶矩。

问题 21: `FP16` 和 `BF16` 有什么区别?

参考回答:

两者都是 16 位浮点, 但 `BF16` 的指数范围更接近 `FP32`, 因此动态范围更大, 更不容易溢出, 通常大模型训练更稳。`FP16` 在很多硬件上

问题 22: `autocast` 和 `GradScaler` 分别做什么?

参考回答:

`autocast` 负责在前向中自动选择合适算子的低精度执行, 减少手动 `dtype` 管理; `GradScaler` 主要用于 `FP16`, 通过放大 `loss`、反向后

问题 23: activation checkpointing 为什么能省显存?

参考回答:

它在 forward 时不保存所有中间激活, 而是在 backward 时重新计算一部分前向, 用额外计算换显存。适合深层模型和长序列训练, 但

10.7 分布式训练

问题 24: DDP 的工作原理是什么?

参考回答:

DDP 是数据并行。每个进程通常绑定一张 GPU, 并持有一份模型副本。每个进程处理不同 mini-batch, backward 后通过 all-reduce 同步梯度, 保证每张卡得到一致的平均梯度, 然后各自 optimizer step, 使参数保持一致。

问题 25: rank、local rank、world size 分别是什么?

参考回答:

world size 是总训练进程数, rank 是全局进程编号, local rank 是当前机器内的进程或 GPU 编号。通常用 local rank 绑定 GPU, 用

问题 26: 为什么 DDP 要用 DistributedSampler?

参考回答:

因为每个 rank 应该处理不同数据子集, 否则多张卡会重复训练同样样本。DistributedSampler 会按 rank 和 world size 切分数据, 并

问题 27: DDP 和 FSDP 有什么区别?

参考回答:

DDP 每张卡通常持有完整模型副本, 主要通过数据并行提升吞吐; FSDP 会把参数、梯度和优化器状态分片到多张卡上, 降低单卡显存, 并

10.8 Debug 与 Profiling

问题 28: loss 不下降怎么排查?

参考回答:

我会先检查数据和 label 是否正确, 再看 logits 和 labels 的 shape 是否对齐, loss 是否有限且依赖参数。然后检查梯度是否为 Non-

问题 29: 训练出现 NaN 怎么排查?

参考回答:

先定位 NaN 出现在输入、logits、loss、梯度还是参数。常见原因包括学习率过大、FP16 溢出、softmax 或 mask 产生异常、loss 分

问题 30: OOM 怎么排查?

参考回答:

先确认是训练还是验证 OOM, 再看 batch size、sequence length、是否开启 AMP、是否保存了带图 tensor、验证是否关闭梯度、是否

问题 31: GPU 利用率低怎么排查?

参考回答:

先区分是 DataLoader 慢、CPU-GPU 拷贝慢、模型计算太小、频繁同步还是分布式通信慢。可以单独测试 DataLoader 速度, 用 cuda s

10.9 Transformer 实现

问题 32: Self-attention 的 shape 如何变化?

参考回答:

输入 hidden states 是 $[B, T, d]$ 。经过 Q/K/V 投影后仍是 $[B, T, d]$ ，再 reshape 成 $[B, H, T, D]$ ，其中 $D=d/H$ 。 $Q @ K.transpose(-2, -1)$ 得到 scores $[B, H, T, T]$ ，softmax 后乘 V 得到 context $[B, H, T, D]$ ，最后合并多头回到 $[B, T, d]$ 。

问题 33: 为什么需要 causal mask?

参考回答:

Decoder-only 语言模型按自回归方式预测下一个 token，当前位置不能看到未来 token。训练时虽然整段序列并行输入，但 causal mask

问题 34: RoPE 作用在哪里?

参考回答:

RoPE 通常作用在 attention 的 Q 和 K 上，对它们按位置做旋转。这样 Q 和 K 的点积会包含相对位置信息，模型可以在 attention score

问题 35: KV Cache 为什么能加速推理?

参考回答:

自回归推理每步只新增一个 token，历史 token 的 K/V 不变。KV Cache 缓存历史 K/V，后续 decode 只计算新 token 的 K/V，并和历史

10.10 系统性开放题

问题 36: 如何设计一个可靠的训练脚本?

参考回答:

我会把训练脚本分成配置、数据、模型、optimizer/scheduler、训练循环、验证、checkpoint 和日志几部分。训练循环要支持 device

问题 37: 从单卡训练扩展到多卡训练要改什么?

参考回答:

需要用 torchrun 启动多进程，初始化 process group，用 local_rank 绑定 GPU，把模型包装成 DDP，DataLoader 使用 DistributedS

问题 38: 如何把一个教学 Transformer 改得更接近真实 LLM?

参考回答:

可以从架构和工程两侧升级。架构上加入 RMSNorm、RoPE、SwiGLU、GQA/MQA、weight tying 和更合理初始化。工程上加入高效 atten

10.11 最小可运行 PyTorch 工程面试复盘 demo

下面这个 demo 不依赖 torch，用于训练“面试回答是否覆盖关键证据”的自查习惯。它不会替代真实 PyTorch 代码，但能把本章高频问题转成可量化检查：主题覆盖、公式覆盖、debug 覆盖、风险表述和下一步复习计划。

```
from collections import Counter
```

```
required_keywords = {
    "tensor_shape": {"shape", "dtype", "device", "stride", "contiguous"},
    "autograd": {"dynamic graph", "leaf", "grad accumulation", "zero_grad", "detach"},
    "module": {"parameter", "buffer", "state_dict", "ModuleList", "train/eval"},
    "dataloader": {"Dataset", "DataLoader", "collate_fn", "padding", "DistributedSampler"},
}
```

```

    "training_loop": {"forward", "backward", "clip", "optimizer", "checkpoint"},
    "amp_memory": {"BF16", "autocast", "GradScaler", "activation", "OOM"},
    "distributed": {"rank", "world size", "all-reduce", "global batch", "no_sync"},
    "debug_profiling": {"finite", "grad norm", "DataLoader", "synchronize", "profiler"},
    "transformer": {"Q/K/V", "causal mask", "RoPE", "KV Cache", "shift loss"},
}

mock_answers = {
    "tensor_shape": "I start from shape, dtype, device, stride and contiguous before checking values.",
    "autograd": "PyTorch builds a dynamic graph; leaf parameters keep grads, grad accumulation requires zero_grad, and",
    "module": "nn.Module registers Parameter, buffer and ModuleList entries into state_dict; check train/eval and mis",
    "dataloader": "Dataset returns one sample; DataLoader batches with collate_fn, padding, and DistributedSampler; c",
    "training_loop": "A step runs forward, loss, backward, grad clip, optimizer step, scheduler, logging, and checkpo",
    "amp_memory": "BF16 often has safer range; autocast chooses dtypes, GradScaler helps FP16, activation memory can",
    "distributed": "DDP uses rank and world size; all-reduce averages gradients, global batch changes, no_sync reduce",
    "debug_profiling": "Check finite tensors, grad norm, DataLoader speed, CUDA synchronize timing, and torch profile",
    "transformer": "Q/K/V shape, causal mask direction, RoPE on Q/K, KV Cache decode length, and shift loss must be c",
}

formula_checks = {}

B, T, V = 2, 8, 32
formula_checks["shift_rows"] = B * (T - 1)
formula_checks["shift_logits_shape"] = (B * (T - 1), V)
formula_checks["shift_labels_shape"] = (B * (T - 1),)

per_device_batch, world_size, accum_steps = 2, 4, 8
formula_checks["global_batch"] = per_device_batch * world_size * accum_steps
formula_checks["loss_scale_per_micro_step"] = 1 / accum_steps

param_count = 1_000_000
param_bytes, grad_bytes, adamw_state_bytes = 2, 2, 8
train_bytes = param_count * (param_bytes + grad_bytes + adamw_state_bytes)
formula_checks["train_state_mib"] = round(train_bytes / 1024**2, 3)

local_grads = [0.2, 0.4, -0.1, 0.3]
avg_grad = sum(local_grads) / len(local_grads)
formula_checks["ddp_avg_grad"] = round(avg_grad, 4)
formula_checks["synced_param_after_step"] = round(1.0 - 0.5 * avg_grad, 4)

batch, heads, seq_len, head_dim = 2, 4, 5, 8
formula_checks["attention_scores_shape"] = (batch, heads, seq_len, seq_len)
formula_checks["attention_score_cells"] = batch * heads * seq_len * seq_len

layers, kv_heads, cache_len, bytes_per_elem = 12, 4, 128, 2
kv_bytes = 2 * layers * batch * kv_heads * cache_len * head_dim * bytes_per_elem
formula_checks["kv_cache_mib"] = round(kv_bytes / 1024**2, 3)

def audit_keywords(required, answers):
    missing = {}
    covered_topics = 0
    for topic, keywords in required.items():
        answer = answers.get(topic, "")
        missing_keywords = [kw for kw in sorted(keywords) if kw.lower() not in answer.lower()]

```

```

        if not missing_keywords:
            covered_topics += 1
        else:
            missing[topic] = missing_keywords
    return covered_topics / len(required), missing

def detect_red_flags(answers):
    risky_phrases = ["一定不会", "完全不需要", "只要", "肯定最优", "不用验证"]
    hits = []
    for topic, answer in answers.items():
        for phrase in risky_phrases:
            if phrase in answer:
                hits.append((topic, phrase))
    return hits

topic_coverage, missing = audit_keywords(required_keywords, mock_answers)
red_flags = detect_red_flags(mock_answers)
rubric_counts = Counter()
for answer in mock_answers.values():
    lower = answer.lower()
    rubric_counts["shape_or_formula"] += int(any(x in lower for x in ["shape", "global batch", "q/k/v", "shift"]))
    rubric_counts["debug_path"] += int(any(x in lower for x in ["check", "checking", "profiler", "resume", "finite"]))
    rubric_counts["pitfall"] += int(any(x in lower for x in ["oom", "padding", "detach", "contiguous", "rank"]))

rubric = {
    key: round(value / len(mock_answers), 3)
    for key, value in sorted(rubric_counts.items())
}

gates = {
    "topic_coverage": topic_coverage >= 0.95,
    "formula_checks": formula_checks["global_batch"] == 64
    and formula_checks["shift_logits_shape"] == (14, 32)
    and formula_checks["ddp_avg_grad"] == 0.2,
    "debug_coverage": rubric["debug_path"] >= 0.55,
    "risk_free": not red_flags,
}

revision_plan = {
    topic: [f"add keyword: {kw}" for kw in kws]
    for topic, kws in missing.items()
}

print("topic_coverage=", round(topic_coverage, 3))
print("missing=", missing)
print("formula_checks=", formula_checks)
print("rubric=", rubric)
print("red_flags=", red_flags)
print("revision_plan=", revision_plan)
print("gates=", gates)
print("gate_pass=", all(gates.values()))

```

运行后应看到 gate_pass=True。如果 missing 不为空，不要机械补关键词，而要回到对应章节补三件事：机制、shape / 公式、排查路径。面试复盘的价值不在于给自己打高分，而是把“我觉得会了”变成可检查的证据。

10.12 本章总结

PyTorch 工程面试的核心不是背 API，而是能把机制、代码和排查路径连起来。回答时建议遵循：

1. 先说概念定义。
2. 再说关键机制。
3. 再说常见坑。
4. 最后说 debug 或工程实践。

本册已经覆盖 tensor、autograd、Module、DataLoader、训练循环、混合精度、分布式、debug/profiling 和 Transformer 组件实现。掌握这些内容后，你应该能阅读大多数 PyTorch 训练代码，手写核心组件，并在训练异常时给出清晰排查路径。